



RESim User's Guide

Reverse Engineering and vulnerability analysis of software on networks of heterogeneous computers by instrumenting simulated hardware

September 1, 2026

Contents

1	Introduction	6
1.1	This Guide	6
1.2	Analysis artifacts	7
1.3	Dynamic analysis of programs executing in their environment	8
1.4	Limitations	8
1.5	Development and Availability	8
2	Notional Workflow	9
2.1	Workspace directories	9
2.2	Configuration files	9
2.3	Kernel parameter extraction	9
2.4	Find interesting processes	10
2.5	Analyze a process	10
2.6	Code coverage with AFL	11
3	RESim commands	11
3.1	General display	11
3.2	Process tracing	11
3.3	Saving state	13
3.4	Shared libraries	13
3.5	Process Analysis	14
3.6	Data tracking	16
3.7	Vulnerability detection	18
3.8	System modification	19
3.9	Code coverage	19
3.10	Fuzzing	20
3.11	Msc	22
4	Defining a target system	22
4.1	ENV section	22
4.2	Target sections	24
4.3	Target platforms	26
4.4	Network definitions	26
4.4.1	Real network connections	27
4.4.2	DHCP	27
4.4.3	Configuring VLANs	27
4.5	Driver component	28
5	Running the simulation	28
5.1	Installation	28
5.1.1	Missing Simics platform files	29
5.1.2	IDA Pro installation	30
5.1.3	Ghidra installation	30
5.1.4	AFL installation	30
5.2	Getting started	30
5.2.1	Example systems	30
5.2.2	Defining your own simulation	30
5.2.3	Kernels with ASLR	31
5.2.4	Kernel Parameters for x86 32-bit compatibility	31
5.3	Program binary locations	31
5.4	Tracing processes	32
5.5	IDA Pro	32
5.5.1	RESim IDA menus and function keys	33
5.5.2	RESim IDA windows	33
5.5.3	RESim IDA options	33
5.5.4	IDA data decoding	33
5.5.5	Helping IDA with function names	34
5.5.6	IDA Troubleshooting	34
5.6	Ghidra	34

5.7	Dynamic changes to execution control flow	35
5.8	Dynamic modifications to memory, registers, topology and control flow	36
5.8.1	ReadReplace	36
5.8.2	regSet	37
5.8.3	Dmods	37
5.8.4	Dynamic modifications to multiple computers	39
5.8.5	Dmods in the background	39
5.8.6	Manual modifications to topology	39
5.9	Tracing processes	40
5.10	Sending data	40
5.10.1	Using drive-driver	40
5.11	Tracking data	41
5.11.1	Watch Marks	42
5.11.2	Sending data to be tracked	43
5.11.3	Tracking data through pipes	43
5.11.4	Tracking libraries	43
5.11.5	Suppress tracking of functions	43
5.11.6	User defined memory functions	43
5.11.7	When are you done?	43
5.11.8	Warning	44
5.11.9	Tracking manually modified data	44
5.11.10	Tracking backwards	44
5.11.11	Removing unwanted traffic	44
5.12	Tracking injected data	44
5.12.1	Where to inject data?	45
5.12.2	Injecting data into the driver	45
5.12.3	Preparing for data injection	46
5.12.4	Injecting into a TCP conversation	47
5.12.5	Injecting data	47
5.13	Code coverage	47
5.13.1	Branches not taken	47
5.13.2	Hits in libraries	47
5.14	Generating new input data sets	48
5.15	Selecting checkpoints	48
6	Fuzzing with AFL	48
6.1	Seeds	49
6.2	Injecting fuzzed data	49
6.3	Fuzzing code coverage	50
6.3.1	BNTs from fuzzing	50
6.4	Data filters	51
6.5	Address jumpers	51
6.6	Fuzz another library	51
6.7	Fuzz another process	51
6.8	Fuzz using the driver	51
6.9	Thread isolation and skipping breakpoints	52
6.10	Crash analysis	52
6.11	Blacklist crashing instructions	52
6.11.1	False crash reports	52
6.12	Update code coverage	52
6.13	Fuzzing performance	53
6.14	Parallel fuzzing	53
6.14.1	Fuzzing with multiple computers	54
6.15	Fuzzing in background	54
6.16	Fuzzing application notes	54
6.16.1	Why fuzz with full system simulation?	54
6.16.2	TCP timing and packetizing	55
6.16.3	State of the target system	55
6.16.4	False paths	55
6.16.5	CADET01	55

7	Example workflows	55
7.1	Analysis of a specific service behavior	55
7.2	Watch consumption of a UDP packet	56
7.3	Reverse engineer a service	56
7.4	Find source of log messages	57
7.5	Observe changes in outputs	57
7.6	Track buffer accesses	57
7.7	Find code divergence	57
7.8	Branches not taken	58
7.9	Packaged example with public images	58
7.10	Fuzzing a web service	58
8	Example targets	58
8.1	CADET01 Example	58
8.2	Network File Service	59
8.3	Cyber Grand Challenge Services	59
8.4	ARM Cortex A9 Sample	59
8.5	ARM5 Sample	59
8.6	ARMv8-A FVP Foundation	59
8.7	Custom devices	59
9	Implementation strategy	59
10	Troubleshooting	60
11	Data Stores	60
12	IDA data and hits files	61
12.1	Coverage files	61
13	Utility scripts	61
14	Testing	63
15	License	63
	Appendices	65
	Appendix A Analysis strategies	65
A.1	Analysis on a custom stripped kernel	65
A.2	Detecting SEGV on a stripped Linux Kernel	65
A.2.1	Faults on ARM	65
A.3	External tracking of shared object libraries	66
A.4	Analysis of programs with crude timing loops	66
A.5	Stupid close calls	66
A.6	Breakpoints can be complicated: Real and virtual addresses	67
	Appendix B Divergence Between Physical Systems and RESim Simulations	68
B.1	Overview	68
B.2	Timing	68
B.3	Model Limitations	68
B.3.1	VLANs	68
B.3.2	File descriptors	68
	Appendix C What FD is this?	68
	Appendix D Context management implementation notes	69
	Appendix E What is different from Simics?	69
	Appendix F IDA Pro issues and work arounds	69
	Appendix G Simics issues and work arounds	69

Appendix H Performance tricks	70
Appendix I New disk images	70
Appendix J Implementation notes	70
J.1 HAPs	70
J.2 Thread tracking	70
J.3 Fuzzing anther process	71
Appendix K ToDo	71
K.1 Msc	71
K.2 Data Watch and Watch Marks	72
K.2.1 Data Watch ntohl	72
K.2.2 Data watch ad-hoc copies	72
K.2.3 Watch Marks and Parsing	72
K.2.4 Tracking values returned in registers	72
K.3 I/O via threads	72
K.4 Tracing library calls	73
Appendix L Dynamically loaded Linux programs	73
L.1 Watching process exit whilst jumping around time	73
L.2 Defining new targets	73
L.3 Fuzzing: too many crashes	73
L.4 Branches not taken	73
L.5 Skipping over kernel calls	73
L.6 More on kernel data buffers	74
L.7 Computed data (e.g., CRC) considerations	74
L.8 UDP without headers	74
L.9 TCP conversations and fuzzing	74
L.10 Fuzzing crash detection	75
L.11 BNTs and compares	75
L.12 Shared libraries and jumpers	75
Appendix M Driver platforms	75
M.1 Using the driver component	76
M.2 Notes on updating the driver	77
M.3 Notes on driver-server.py	78
Appendix N Simics user notes	78
Appendix O Injecting to kernel buffers	79
Appendix P Troubleshooting	79
P.0.1 Missing syscalls in logs	79
P.1 Real networks: WARNING	79
P.2 Failed cat to /dev/udp/localhost	79
P.3 Real networks and UDP	79
P.4 Backtracing malloc'd addresses	80
P.5 Fork exit	80
P.6 Fuzzing	80
P.7 Fuzzing TCP	80
P.8 Multipacket workflows	80
P.9 Your target process is not scheduled in the snapshot	80
P.10 Backstops	80
Appendix Q Windows	81
Q.1 Windows program names	81
Q.2 Windows Kernel Parameters	81
Q.3 PIDs and Threads	82
Q.4 CR3 and Windows	82
Q.5 Windows syscall parameters	83

Appendix R CADET01 networking	83
Appendix S More code coverage	84
S.1 Branches not taken	84
S.2 Ad-hoc code coverage	84
S.3 Data injection and system calls	85
Appendix T VxWorks DKM	85
T.1 File system	85
T.2 Legacy binaries	85
T.3 Execution context	85
T.4 VxWorks Performance	86
Appendix U Simics 7	86
Appendix V Progressive fuzzing	86
V.1 Overview	86
V.2 Limitations	86
V.3 Workflow and artifacts	87
Appendix W ROP gadgets	87
Appendix X PowerPC	87
X.1 current task pointer	88
X.2 syscall jump table	88
X.3 Page tables	88
X.4 instructions	88
X.5 Simics breakage	89
Appendix Y Windows XP (32 bit)	89

1 Introduction

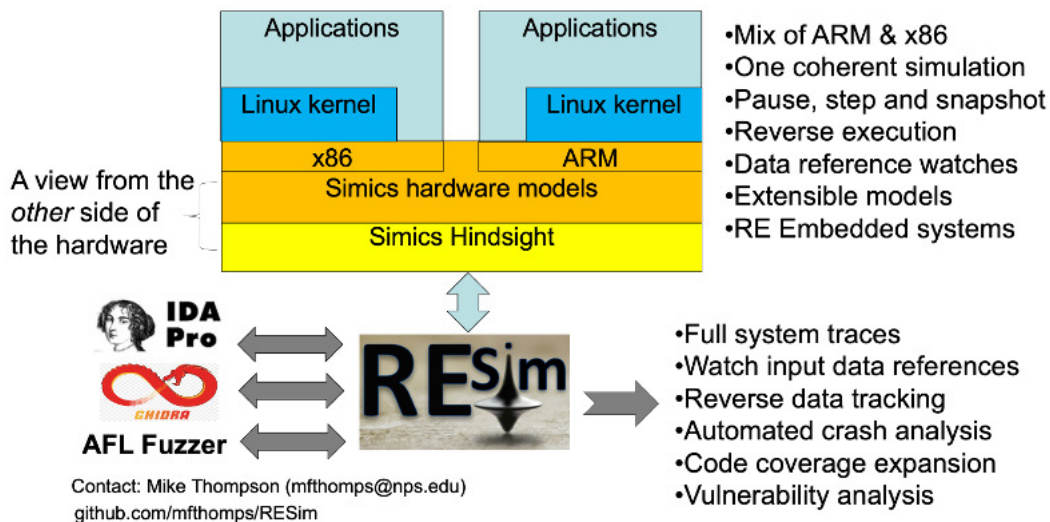
Imagine you would like to dynamically analyze, (e.g., perform cyber testing on), the processes on embedded networked computers, and assume you'd like to perform this analysis without altering the systems to add instrumentation and without having a shell on the systems. RESim is a dynamic software analysis tool that provides detailed insight into processes, programs and data flow within networked computers. RESim simulates networks of computers through use of the Intel Simics¹ platform's high fidelity models of processors, peripheral devices (e.g., network interface cards), and disks. The networked simulated computers load and run targeted software copied from disk images extracted from the physical systems being modeled. Insight into software behavior is obtained by instrumenting the simulated hardware rather than instrumenting the software.

RESim aids reverse engineering of networks of Linux-based systems² by inventorying processes in terms of the programs they execute and the data they consume. Data sources include files, device interfaces and inter-process communication mechanisms. Process execution and data consumption is recorded through dynamic analysis of a running simulated system without installation or injection of software into the simulated system, and without detailed knowledge of the kernel hosting the processes. The simulation can be paused for inspection, e.g., when a specified process is scheduled for execution, and subsequently continued, potentially with altered memory or register state. The analyst can explicitly modify memory or register content, and can also dynamically augment memory content based on system events, e.g., change a password file entry as it is read by the `su` program (see the `Dmod` function described in 5.8).

RESim also provides interactive analysis of individual executing programs through use of either the IDA Pro or the Ghidra disassembler/debugger to control the running simulation. The disassembler/debugger allows setting breakpoints to pause the simulation at selected events in either future time, or past time. For example, RESim can direct the simulation state to reverse until the most recent modification of a selected memory address. During a RESim session, any point within the simulation can be *bookmarked*, and that execution state can later be restored.

The *American Fuzzing Lop* (AFL) fuzzer is integrated with RESim, which injects fuzzed data generated by AFL directly into simulated memory. Instead of constructing custom test harnesses for each target program, RESim enables fuzzing of programs as they execute in their native environments, which may include substantial interaction with other processes or computers prior to reaching the state at which fuzzing is to commence. RESim creates a memory-based snapshot of that state, and returns the system to that state after each fuzzing iteration.

The analyst can generate reloadable checkpoints at any point during system execution, and these checkpoints can then be shared with other analysts and used as the starting point of future RESim sessions. These checkpoints include the full target system state (e.g., similar to a VM snapshot) as well as RESim context information such as information about currently running processes.



1.1 This Guide

The remainder of this introduction provides an overview of RESim features and its limitations and availability. Following the introduction:

¹Simics is a full system simulator sold by Wind River and owned by Intel, which holds all relevant trademarks.

²And preliminary support for analyzing Windows applications, see [Q](#).

- Section 2 describes a notional workflow, highlighting functions and features of RESim and how an analyst might use them to reverse engineer a network of computers.
- The set of commands supported by RESim are listed in section 3.
- Section 4 details the elements of RESim configuration files (ini files) that define the system to be simulated.
- The mechanics of installing and running RESim are described in section 5. This includes descriptions of data tracking (forward and reverse) and code coverage.
- Example workflows for RESim are provided in section 7.
- Example disk images and targets are listed in section 8.
- Use of AFL to expand code coverage and identify vulnerabilities is described in section 6.
- Section 9 describes the implementation strategy and some of its consequences.
- Section 10 provides some trouble-shooting tips.
- The Appendices provide a set of notes and hints that have not yet been integrated into the body of the guide.

1.2 Analysis artifacts

RESim generates system traces of all processes on a computer, starting with system boot, or from a selected checkpoint. Trace reports include two components:

1. A record of system calls, identifying the calling process and selected parameters, e.g., names of files and sockets and IP addresses.
2. A process family history for each process and thread that has executed, identifying:
 - (a) Providence, i.e., which process created the process (or thread), and what programs were loaded via the `execve` system call.
 - (b) Files and pipes that had been opened (including file descriptors inherited from the parent), and those that are currently open.
 - (c) Linux socket functions, e.g., `connect`, `accept`, `bind`, etc. Socket connect attempts to external components are highlighted, as are externally visible socket accepts.
 - (d) Mapped memory shared between processes

The system trace is intended to help an analyst identify programs that consume externally shaped data. Those programs can then be dynamically analyzed by RESim and the disassembler/debugger.

Artifacts associated with individual processes (and their associated threads) include:

- Maps of shared object libraries loaded by each thread, including their load addresses
- Records of references to input data, and references to copies of such data, as the target program consumes the data. These *Watch Marks*, (see 5.11), can be dynamically loaded to skip the simulation to the point of the reference.
- Reverse data tracks that trace sources of memory or register values in terms of exchanges between memory and registers, potentially leading back to initial ingest of the data, e.g., via a `recv` system call.
- Data written to selected files or file descriptors, (see the `traceFile/traceFD` commands).
- System traces as previously described, but constrained to actions taken by threads within the process being analyzed.

1.3 Dynamic analysis of programs executing in their environment

RESim couples a disassembler/debugger client with the Simics simulation to present a dynamic view of a running process. The analyst sets breakpoints and navigates through function calls in both the forward and reverse execution directions. This facilitates tracking the sources of data. For example, if a program is found to be consuming data at some location of interest, reverse execution might identify a system call that brings the data into the process's address space.

Analysis is performed entirely through external observation of the simulated target system's memory and processor state, without need for shells, software injection, or kernel symbol tables. The analysis is said to be *external* because the observation functions do not affect the state of the simulated system. For example, when viewing code with the IDA Pro or Ghidra debugger, addresses that are not yet paged in will appear as containing zeros. In most other systems, the debugger or its agent would run on the target, and thus the mere act of viewing the address with a debugger would cause the kernel to page in the memory containing the referenced code.³

A key property that distinguishes RESim from other RE strategies is that analysis occurs on processes as they execute in their native environment, and as they interact with other processes and devices within the system. Consider an example process that communicates with a remote computer via a network while also interacting with a local process via a pipe. When the analyst pauses RESim for inspection, the entire system pauses. The simulation can then be resumed (or single-stepped) from the precise state at which it was paused, without having to account for timeouts and other temporal-based discontinuities between the process of interest and its environment.

1.4 Limitations

RESim analyzes Linux-based systems (and some Windows based systems) for which copies of bootable media or root file systems can be obtained. Analysis does not depend on a system map of the kernel, i.e., it works with stripped kernel images. The current version of RESim supports 32-bit and 64-bit X86 ARM platforms. It can also be extended to support alternate architectures, e.g., PowerPC, supported by Simics processor models⁴. RESim is currently limited to single-processor (single core) models. Simics supports multi-processor simulations (at reduced performance), but RESim has not yet been extended to monitor those. See section B for information on potential divergence of simulations from real systems.

RESim and Simics are not “record-and-replay” tools (though Simics has that features to support that.) We run, instrument and observe simulations of real systems. These simulations can and will diverge from real world results and multiple runs of what seem to be the same simulation will diverge from each other (as will multiple runs of real systems, even though they consume the same data.) Sometimes the divergence is minimal, e.g., different timestamps within data. Other times the divergence can be substantial. The point is that you should not always expect the same results across multiple runs. For example, full system traces may contain different data even though they commence from the same snapshot.

1.5 Development and Availability

RESim is derived from the “Cyber Grand Challenge Monitor” (CGC), developed by the Naval Postgraduate School in support of the DARPA CGC competition. It is implemented in Python, primarily using Simics breakpoints and callbacks, and does not rely on Simics “OS Awareness” or Eclipse-based interfaces.

- Simics is available as a commercial product from Wind River. A free version for Intel processors is available at <https://www.intel.com/content/www/us/en/developer/articles/tool/simics-simulator.html>. RESim has been extended to work with this free version. RESim requires Simics 6. Preliminary support for Simics 7 has been implemented along with our own module for managing reverse execution, which was deprecated in Simics 7.
- IDA Pro extensions are implemented using IDAPython, and are included within the RESim repo at <https://github.com/mfthomps/RESim/simics/ida>
- The Ghidra plugins for RESim are available at: <https://github.com/mfthomps/RESimGhidraPlugins>. A fork of gdb needed for use with the Ghidra plugin is at <https://github.com/mfthomps/binutils-gdb>.
- A fork of the AFL fuzzer integrated with RESim is available at: <https://github.com/mfthomps/AFL>.
- Preliminary Windows support is provided as described in Q.

³See A.6 for another example of the implications

⁴A summary of Simics device models is at: <https://www.windriver.com/products/simics/simics-supported-targets.html>

- In addition to running on a local Simics installation, RESim can be offered as a network service to users running local copies of IDA Pro and an SSH session with a RESim console. See the *RESim Remote Access Guide*.

2 Notional Workflow

This section provides an overview of how RESim can be used by an analyst to reverse engineer a system. Details are provided elsewhere in this guide. The general steps include:

- Extract software images from target systems.
- Identify Simics models to simulate the target hardware.
- Create a RESim workspace directory with which to run the simulation
- Construct a RESim configuration file to identify system image file locations and RESim parameters.
- Use RESim to extract a set of kernel parameters if not already extracted for the kernel images being used.
- Run the simulation to produce full system traces and processing reports for the simulated systems.
- Identify processes of interest, and use the RESim IDA Pro debugging client to analyze their behavior, e.g, the protocols they consume.
- Use the integrated AFL fuzzer to improve code coverage within the targeted process.
- Analyze AFL and RESim artifacts to identify unexplored code paths and identify input data to reach those paths, potentially feeding those back to AFL.
- Automatically assess any crashes generated by AFL to identify potentially exploitable vulnerabilities.
- Use RESim to analyzing the vulnerabilities and craft proof of concept exploits.

2.1 Workspace directories

Simics simulations are run from workspace directories prepared using the `resim-ws.sh` command, which creates a Simics workspace within a new empty directory and populates it with selected RESim files. The workspace is where simulation-specific configuration files and scripts are to reside. Snapshot data is also stored within workspace directories. You may share snapshots between workspaces by using symbolic links.

2.2 Configuration files

Simulated systems are defined within RESim configuration files (see 4 that parameterize pre-defined Simics scripts to identify processors and interface devices, e.g., network cards, disks and system consoles. RESim currently includes the following platforms:

- A general purpose X86 platform with disk, multiple ethernet and serial ports;
- A generic ARM Cortex A9 platform with disk, multiple ethernet and serial ports.
- An ARMV5 platform based on the ARM926EJ-S processor. This is a partial implementation of the ARM Integrator-CP board. It currently only supports the initial RAM disk, one ethernet and serial ports.

Other platforms can be modeled via Simics, the details of which are beyond the scope of this manual. Once a system is modeled and referenced by a RESim configuration file, a RESim script is run, naming the configuration file.

2.3 Kernel parameter extraction

RESim analysis requires about twenty parameters that characterize the booted kernel instance, e.g., offsets within task records and addresses of selected kernel symbols. Many of these parameters are defined within the Linux sched.h file, however offsets within structures vary by kernel versions and configurations. RESim uses heuristics to locate the needed offsets. The `CREATE_RESIM_PARAMS` directive within the RESim configuration file directs the tool to automatically analyze the running kernel and extract the desired parameters. This allows RESim to analyze disparate Linux kernels without a priori knowledge of their versions or configurations⁵. See section 5.2.

⁵Not to be confused with the similar function included with Simics Analyzer product. RESim uses an alternate strategy for OS-Awareness.

2.4 Find interesting processes

Once the kernel parameters have been extracted, the RESim configuration file is modified to reference the parameter file, and the simulation is restarted. The user is presented with a command line interface console. This console manages the simulation via a combination of RESim and Simics commands, including commands to:

- Start or stop (pause) the simulation
- Run until a specified process is scheduled
- Run until a specified program is loaded, i.e., via `execve`
- Generate a system trace
- Inspect memory and component states
- Enable reverse execution, i.e., allow reversing to events from that point forward
- Set and run to breakpoints, either in the future or in the past.
- Target RESim to focus on a specific process thread group, e.g., for dynamic analysis using IDA Pro.

See section 3 for details of RESim commands.

A typical strategy with RESim is to initially perform a full system trace on a system as a means of identifying programs of interest, e.g., which can then be further analyzed using the interactive IDA Pro client. The `traceAll` command described below in 3 generates such a trace. Note that a typical Linux-based system can perform tens of thousands of system calls during initial boot processing. It may save considerable time to use the `toProc` command to run forward to some event such as creation of `rsyslogd` before issuing the `traceAll` command. Note that `toProc` tracks processes creation and some system configuration settings such as IP addresses set using `ip` or `ifconfig` commands, which can be seen using the `showNets` command. Use the `writeConfig` command to create a checkpoint, and then update your ini file to begin at that checkpoint using the `RUN_FROM_SNAP` directive. Trace files are created in the `./logs` directory. During a trace, if you use the Simics stop and run commands, the tracing will continue. While stopped, you may use the `tasks` commands to see which processes are currently running. Or the `showBinders` command to see network ports being listened to. See the 3.2 for more information about process tracing.

Running wireshark from the Simics prompt will often tell you IP addresses used by the target.

2.5 Analyze a process

When RESim is targeted for a given process it runs until that process is scheduled, after which the user starts IDA Pro with a suite of custom plugins that interact with the simulation. If a binary image of the target program is available, standard IDA Pro analysis functions are performed. If no program image is available, IDA Pro will still present disassembly information for the program as it exists in simulated system memory.

RESim extends IDA Pro debugger functions and the analyst accesses these functions via menus and hot keys. The disassembler/debug client can be used to:

- Single-step through the program in either the forward or reverse direction
- Set and run to breakpoints in either direction
- Run to the next (or previous) system call of a specified type, e.g., `open`.
- Run to system calls with qualifying parameters, e.g., run until a socket connect address matches a given regular expression.
- Track references to data read in via a given FD, as well as references to copies of that data, e.g., via `mempcpy`.
- Reverse-trace the source of data at a memory address or register.
- Modify a register or memory content
- Switch threads of a multithreaded application
- Set and jump to bookmarks

2.6 Code coverage with AFL

Using RESim to understand the protocols handled by a process, (and associated vulnerabilities), typically involves providing the process with crafted data and observing its behavior and modifying data to achieve greater code coverage. AFL contributes to code coverage expansion and vulnerability identification through automated guided fuzzing. AFL provide RESim with randomized inputs that it continually augments based on feedback that RESim generates reflecting code paths reached by each input.

A set of RESim functions can then be used to replay AFL sessions to automatically identify and display unexplored code paths within the IDA client. This allows the analyst to identify and generate inputs for additional code paths, and feed these inputs back into new AFL sessions. See section 6 for information on the use of AFL.

See 7 for detailed examples of RESim workflows.

3 RESim commands

The following RESim commands are issued at the Simics command prompt, naming the commands as methods of the `cgc` python module, e.g., "`@cgc.tasks()`". Interfaces for all of these commands are implemented in the python script at `simics/monitorCore/genMonitor.py`. If this documentation falls out of sync, or you need to fix broken stuff, refer to that file (and the dependent python classes, which are all in that directory.)

There are also a set of utility commands that run from a bash prompt (rather than from Simics). See section 13 for a list of those.

3.1 General display

- `show` – Show current process information for the currently monitored target.
- `tasks(filter=None)` – List currently executing process names and their PIDs for the currently monitored target. Use a simple string filter to view information on a specific program.
- `showThreads` – List threads of process currently selected for analysis, e.g., via `debugProc`.
- `showTargets` – List the target cells that may be monitored.
- `setTarget` – Set the currently monitored target.

3.2 Process tracing

The tracing commands described below are applied to the currently selected target system. If you wish to trace multiple simulated computers, select each using `setTarget` and issue the `traceAll` command. Note traces are not necessarily repeatable, i.e., there is usually execution divergence between different runs, even when starting at the same snapshot. Trace artifacts are written to the logs subdirectory of the workspace.

Within a given simulated computer the scope of which processes are traced has four forms:

1. All processes are traced as they are scheduled and run.
2. When debugging a selected program, only the threads within the associated process are traced.
3. If `ONLY_PROGS` is set in the ini file, then only the programs listed in the named file are traced.
4. If `SKIP_PROGS` is set in the ini file, then all programs other than those listed in the named file are traced.

A common use of process tracing is to create artifacts describing network usage, e.g., which ports are bound by which processes. Note that RESim tracks such things by watching system calls. Thus, if you only start tracing a process *after* it has bound to a port, then RESim will not tell you that the process is bound to the port.

When tracing is selected, RESim will track loading of shared program libraries.

- `traceAll` – Begin tracing all system calls, storing the results in the logs subdirectory of the workspace. If a program was selected using `debugProc` as described below, limit the reporting to that process and its threads. To limit tracing to the current thread, first issue the `debugThis` command. See the `saveTraces` command below, and the `traceReport.sh` script which will parse system call logs and create reports on file, network and IPC (System V) usage. Also see the `showFDs.ph` utility as a way to find open file descriptors at a given cycle, e.g., to answer question such as: "The trace shows a message being read on from an FD, what is that FD associated with at this time?".

- **traceProcesses** – Begin tracing the following system calls as they occur: vfork; clone; execve; open; pipe; pipe2; close; dup; dup2; socketcall; exit; group_exit Tracing continues until the stopTrace command is issued. **Note** Generally use traceAll instead of this command. This command has some value in that it results in breakpoints on calculated entry points vs sysenter type entry points. This may aid in some RESim debugging.
- **saveTraces** – Combines the next 4 items and saves the results in the workspace/logs directory. This is typically done after doing a traceAll on either an entire component, or a program. After running **saveTraces**, exit Simics, cd to the logs directory and run **traceReport.sh** to parse the trace artifacts and create summaries, e.g., a summary of all services accessing networks.
 - **showProcTrace** – Generate a process family summary of all processes that executed since the trace-Process (or traceAll) command.
 - **showNets** – Display network configuration commands (e.g., **ifconfig** collected from process tracing and the use of toProc.
 - **showBinders** – Display programs that use bind and accept socket calls – intended for use during process tracing to identify processes that listen on externally accessible sockets.
 - **showConnectors** – Display programs that use connect to open sockets – intended for use during process tracing to identify processes that connect to externally accessible sockets.
- **traceFile(logname)** – Copy all writes that occur to the given filename. Intended for use with log files. Output is in ./logs/[basename(logname)]. The **trackIO** command (see 5.11) can be directed to include traced file output in the watch marks, e.g., to see where log messages occur within the time line of watch marks.
- **traceFD(FD)** – Copy all writes that occur to a given FD, e.g., stdout. Output is in ./logs/output-fd-[FD].log Assumes traceAll is set. Use **raw=True** to capture raw binary output. If the given FD is for a bind, then subsequent calls to accept will cause I/O to the new FD to be captured. Use **web=True** to insert delimiters between the data from each read. The resulting file can then be provided to genJson.py with the **-w** option go generate a json file to provide drive-driver. Provide the **all=True** argument to trace output from all processes on the given FD, e.g., to watch stdout.
- **buffer tracing** – Use the **TRACE_BUFFERS** environment variable to report on use of debug buffers. Syntax of each line is:

```
<command> <addr> <reg> <outfile>
```

The *command* can be either **call_reg** or **string_reg**. if **call_reg**:

- *addr* is the address of a call instruction, typically a call to some some sprintf function within a debug routine. This is of the form prog:addr where prog is the program or library name and addr is the static address within that binary. (Use origProgAddr to get the static load address from the dynamic address.)
- *reg* names the register that containing the address of where the called function will write the debug message.

If the command is **string_reg** then:

- *addr* is the address of some instruction at which we know the desired buffer is populated and we have a register whose current address points to that buffer. This is also of the form prog:addr.
- *reg* names the register that containing the address that contains debug message.

The *outfile* is the name of the file into which the messages should be written.

- **flushTrace** - Flush trace output to the trace log files.
- **toProc** – Continue execution until the named program is either loaded via execve or scheduled. Intended for use prior to tracing processes, e.g., to get to some known point before incurring overhead associated with tracing. This function will track processes PIDs and names along with network configuration information, and will save that data if a writeConfig function is used. Use an optional **run=False** to prevent RESim from continuing the simulation, e.g., so that you can run commands such as ptime. NOTE: for windows, if the process is not running, it runs until it is created. A second call to toProc will run until scheduled in user space.

- **autoMaze** – Avoid being prompted when tracing detects a crude timing loop or other events that are repeated many times in a loop. See section [A.4](#).
- **noExitMaze** – Do not check for tight timing loops. See section [A.4](#).
- **instructTrace(file, all_proc=False, kernel=False, watch_threads=False)** -- Generate an instruction trace and save it into the named file. Use the kernel flag to observe traces within the kernel. Use watch_threads to trace all threads of the current process. The file name is relative to /tmp.
- **traceFuns** - Generate trace messages when each function within a target is entered. Currently these message go to the resim log and assume ARM32.
- **traceSO** - Generate trace messages when each shared object is entered. One message is generated for each shared object, and then the corresponding breakpoint is removed. An intended use is determining which DLLs process data input, e.g., runToIO and then traceSO. While trackIO watchmarks also identify the SO file, trackIO can fail to track all data movement. TBD messages currently go to the resim.log.

3.3 Saving state

A *snapshot* of the system state can be created at any time and then named within the ini file using the `RUN_FROM_SNAP` environment variable.

- **writeConfig** – Uses the Simics write-configuration command to save the simulation state for later loading with read-configuration. This wrapper also saves process naming information, shared library object maps and network configuration commands for reference subsequent to use of the read-configuration function.
- Also see `prepInject`.

3.4 Shared libraries

RESim tracks loading of shared libraries within processes depending on the current scope of tracing or whether the program is being debugged. If a program is to be analyzed, then it is critical that the program's use of shared libraries be tracked. When system state is saved to a snapshot, the shared library information is saved and then restored when the snapshot is loaded.

When **traceAll** is used, all traced processes will have their shared libraries tracked. If a process is being debugged, e.g., through use of **debugProc**, then its shared library loading will be tracked both as it loads, and then as it is run forward. If processes of interest are not being traced or debugged, then the **trackThreads** command will cause library loading to be tracked.

- **trackThreads** Causes RESim to track shared library loading for all processes on the selected target, filtered by either the `ONLY_PROGS` or `SKIP_PROGS` directives in the ini file.
- Use of **traceAll** or **traceWindows** will cause tracking of libraries.
- **showSOMap(filter=None)** will display the SO map of the current process. Use the filter to select based on string content.
- **getSO(addr, show_orig=False)** Display the shared library file and address range within which the given address falls. If the `show_orig` option is given, then the original program base address is displayed as well.

If a target system includes multiple interacting processes, use **trackThreads** prior to creation of any of the processes of interest. Let the system run until all such processes have been created and generate a snapshot. You can then use **debugProc** on any of those processes and the system will reflect all of the shared libraries. This strategy is also consistent with the goal of letting the system settle out prior to per-process analysis to ensure that observed behavior is not simply a consequence of incomplete initialization.

Thread tracking will also record initial stack pointer values for each Linux thread as an optimization for stack frame generation. For optimization on windows, we use a heuristic driven by stack analysis that first occurs without a base.

When **debugSnap** or **debugIfNot** is invoked, RESim finds all of the recorded shared libraries and parses them to create shared library maps. On large systems with many DLLs, this can be slow. Use the `ONLY_SO` environment variable to name a file that lists SO files you care about.

3.5 Process Analysis

These functions support interactive analysis of the threads of a process, e.g., to run until some system call is made. Also see the *Tracking data* functions in the following subsection. Analysis functions depend on shared library data gathered while the target process is being loaded and initialized. In general, you must use the `debugProc` function to capture that information. You may then create a checkpoint. Subsequent sessions started from that checkpoint, or subsequent checkpoints, can then use alternate functions to select debugging, e.g., the `debugPidGroup` function. RESim gathers shared library information on all processes started while processing the `debugProc` command, so it is possible to create checkpoints from which multiple processes can be selectively debugged. To put this another way, you cannot choose to debug some process that is already running on the simulated system unless you took steps to gather its shared library data prior to the process being loaded.

- `debugProc(program_name, trace_threads=True, new=False)` – Initiate the debugger server for the given program name. If a process matching the given name is executing, system state advances until the process is scheduled. If no matching process is currently executing, execution proceeds until an `execve` for a matching program. If a copy of the named program is found on the RESim host, (i.e., to read its ELF header), then execution continues until the text segment is reached. RESim tracks the process as it maps shared objects into memory (see Appendix C). The resulting map of shared object library addresses is then available to the user to facilitate switching between IDA Pro analysis and debugging of shared libraries and the originally loaded program.

The process name is searched against the comm names of running processes, which are fifteen characters. RESim will truncate input names to 15 characters.

If the program is not yet running, consider providing the full program path, (e.g., `/usr/bin/foo`), because some systems resolve PATH searches by trying to exec each path, leading to many failed `execve` calls.

Subsequent to the `debugProc` function completion, IDA Pro can be attached to the simulator. Most of the commands listed below have analogs available from within IDA Pro, once the RESim `rev.py` plugin is loaded.

If execution transfers to a shared object library of interest, the associated library file can be found via the `origProgAddr` command described below. Load that file into IDA Pro and rebase to the SO address found via `showSOMap` prior to attaching the debugger. If you have run `reTrack` or `injectIO`, you must re-run the command in order for the Watch Marks to detect and report mem operations, e.g., `memcpy` – and then refresh the IDA Data Watch window.

If you prefix the given process name with `sh` , (sh followed by a space), RESim will look for a shell invocation of the script name that follows the sh. The `trace_threads` flag controls whether shared libraries of other processes will be tracked while looking for the given process. The `new` flag ignores any currently running process and looks for a new creation. This is useful for processes that start, die, and start again for whatever reason.

Note that the initial execution of a target process may not be the ideal state for a snapshot from which to interact with it, i.e., you may want to let the rest of the system initialize and settle out. To achieve that, use `noReverse()` to disable reversing and then run forward until the Simics `ptime` command reflects that the system is not bogged down with initialization. Then create your snapshot. The state of the snapshot may not have the target process scheduled, but when you subsequently run `debugSnap` (see below), it will run forward to your process. If your process is blocked in the kernel waiting for input, you may need to provide that input to see your process scheduled so that you can issue commands.

- `debugTidGroup(tid)` – Initiate the debugger server for a given TID. Intended for use after starting a session from a checkpoint created using `writeConfig`. This assumes process information had been generated in a previous session that was saved.
- `debugSnap` – Assumes the current session was loaded from a snapshot, it use `debugPidGroup` to debug the process being debugged when the snapshot was made. **NOTE** If you use `debugSnap` after running for a while, you may well lose process state, specifically system call information about threads that are waiting in the kernel. If you want to run ahead to let the system settle out, first use `debugSnap` and then use `noReverse` to improve speed.
- `debugIfNot` – Will call `debugPidGroup` for the currently scheduled process.
- `debugThis` – Limit debugging to the current thread, ignoring other threads.

- **stackTrace** – shows the call stack as seen by the monitor. The Ida client uses this to maintain its view of the callstack. The monitor uses the IDA/Ghidra-generated function databases generated using `idaDump.sh` (or Ghidra) to aid in determining if data references (watch marks) are made within known libc functions, e.g., `strcpy`. The monitor-local **stackTrace** command displays a stack trace that uses the IDA function database to resolve names. Be sure to have generated function databases for any libraries that appear in stack traces. This function **may not always be reliable**, e.g., phantom frames may appear based on calls that occurred previously. ARM stack frames may be difficult to determine due to its myriad ways of calling and returning. If the stack frame is not correct, and you rely on it, e.g., to reverse to some function call you see in a stack frame, you can become seriously misled. Consider using `runToText` or similar to find where you came from.
- **runToSyscall(call number)** – Continue execution until the specified system call is invoked. If a value of minus 1 is given, then any system call will stop execution. The simulation stops after the return from the call. If the debugger is active, then the simulation only halts when the debugged process makes the named call.
- **runToCall(call name)** – Continue execution until the specified system call is invoked. The simulation stops at the system call kernel entry. If the debugger is active, then execution only halts when the debugged process makes the named call. Use `stop_on_call` to stop at the start of the call, otherwise execution stops on return of successful calls.
- **revToSyscall()** – Reverse to previous syscall made by the current process.
- **runToConnect(search pattern)** – Continue execution until a socket connection to an address matching the given search pattern.
- **runToBind(search pattern)** – Continue execution until a socket bind to an address matching the given search pattern. Alternately, providing just a port number will be translated to the pattern `.*:N$` where N is the port number.
- **runToAccept(FD)** – Continue execution until a return from a socket accept to the given file descriptor.
- **runToIO(fd, nth=1)** – Continue execution until a read, write, select, ioctl, accept or close with the given file descriptor. If nth is greater than 1, will run until the nth recv call.
- **runToInput(fd)** – Like `runToIO`, but only finds reads, receives, etc.
- **runToOpen(file)** – Stops at the open of a given filename.
- **runToWrite(substring)** – Stops when a process writes the given substring via a system call.
- **clone(nth)** – Continue execution until the nth clone system call in the current process occurs, and then halt execution within the child.
- **runToText()** – Continue execution until the text segment of the currently debugged process is reached. DO NOT use for Windows. Use `runToSO` instead. This, and `revToText`, are useful after execution transfers to libraries, or Linux linkage functions, e.g., references to the GOT.
- **revToText()** – Reverse execute the current process until the text segment is reached. Also useful when you get lucky and execution is down a nop sled. This function can be slow, e.g., if there is a kernel system call between you and the text segment.
- **showSOMap(tid, filter=none)** – Display the map of shared object library files to their load addresses for the given pid (along with the main text segment). Use a simple string filter to look for a particular library.
- **origProgAddr(pid, addr)** – Display the program file name (e.g., library) and load address of the shared object containing the given address. Also displays the original program address, e.g., to include a jumper file for use over multiple load instances.
- **revInto** – Reverse execution to the previous instruction in user space within the debugged process.
- **revOver** – Reverse execution to the previous instruction in the debugged process – without entering functions, e.g., any function that may have returned to the current EIP. Note that ROP may throw this off, causing you to land at the earliest recorded bookmark. Use `revInto` to reverse following a ROP.
- **uncall** – Reverse execution until the call instruction that entered the current function. This may be unreliable with some ARM programs.

- `revToWrite(address)` – Reverse execution until a write operation to the given address within the debugged process. Execution is stopped immediately following execution of the instruction that causes the write.
- `revToModReg(reg)` – Reverse execution until the given register is modified.
- `toPid(pid)` – Run forward until the given pid is scheduled. Use `-1` to indicate any of the threads being debugged.
- `runToUser()` – Continue execution until user space of the current process is reached.
- `reverseToUser()` – Reverse execution until user space of the current process is reached.
- `setDebugBookmark(mark)` – set a bookmark with the given name.
- `goToDebugBookmark` – jump to the given bookmark, restoring execution state to that which existed when the bookmark was set. The `listBookmarks` command will list bookmarks, displaying index numbers. These numbers can be provided to the `goToDebugBookmark` command instead of the bookmark string. Note these bookmarks are separate from the data watch bookmarks, which can be viewed using the `showWatchMarks` command.
- `runToKnown` Continue execution until a text range known to the SOMap (see `showSOMap()`). Intended for use if execution stops in `got/plt` or other loader goo.
- `runToOther` Continue execution until a text range known to the SOMap (see `showSOMap()`) – but not the current text range – is entered. Useful if you are in some library called by some other library, and you want to return to the latter.
- `runToSO` Continue execution until a given program, or library is reached. The optional `threads=True` will stop when any thread of the process enters the library.
- `runToMainSO` Continue execution until a SO or DLL library named within the file defined by `MAIN_LIBS`.
- `doBreak` Sets a breakpoint on a given address. This command can be useful when wishing to break while running other commands. For example, using `doBreak` prior to running `injectIO` may be more reliable than simply setting a Simics break and it will clean up other breaks and haps that might interfere with your subsequent analysis.
- `runToWriteNotZero(addr)` – Run forward until a non-zero value is written to the given memory address.

3.6 Data tracking

Also see [5.11](#)

- `trackIO(FD, reset=False, count=1, mark_logs=False, kbuf=False, run=True, commence=None, max_marks=None, track_calls=False)` – Combines the `runToIO` and the `watchData` functions to generate a list of data watch bookmarks that indicate execution points of relevant IO and references to received data. This list of data watch bookmarks is displayed using `showWatchMarks` or in the IDA client `data watch` window (right click and refresh). If an `accept` call with the given FD is detected, the FD being tracked will be changed to that returned by the `accept` call. The `trackIO` function will break simulation after `BACK_STOP_CYCLES` with no data references. If the debugged processes are in the kernel waiting to read on the given FD, `RESim` will ensure that call is tracked. The `count` parameter can be used to track the Nth read or recv. Data watches persist after the call, e.g., to support `retrack` described below. Each use of `trackIO` will reset all data watches, but does not clear watch marks, i.e., you can still skip to those simulation cycles. Also see the `tagIterator` command and `addIterator`. The `reset` parameter will cause the reverse time origin to be reset if the kernel was waiting on a read/recv. See [P.1](#) Data to be consumed by the target can originate from a simulated computer, e.g., a driver, or via a real network using something like:

```
cat foo.io > /dev/udp/127.0.0.1/6060
```

Note that sending multiple packets via real networks will cause the origin to reset on each packet, thereby limiting the range of reverse execution. It is generally simpler to use a driver computer to send multiple packets, using a program such as `simics/bin/drive-driver` (see [5.10.1](#)). The `-d` option of that program executes the Simics magic instruction to cause `RESim` to reset its origin just prior to the initial data

transmission. This also causes RESim to disconnect the real network to ensure that the reversible range does not receive any real-world leakage.

Use of the `mark.logs` option will include corresponding log entries from any use of `traceFile` or `TRACE.BUFFERS` as watch marks. This may help display diagnostics within your watch mark list as parsing encounters errors.

The `kbuf` option is used in conjunction with `prepInjectWatch` to record addresses of kernel buffers used when receiving the data. Those kernel buffer addresses are then referenced when injecting data into the kernel as described in [5.12.3](#)

Use an optional `run=False` to prevent RESim from continuing the simulation, e.g., so that you can run commands such as `ptime`.

Use an optional `commence=string` to delay data tracking until the received data starts with a given string. This can be useful when a socket receives a lot of traffic and you wish to focus on traffic that you are sending. Use the `commence_offset` parameter to skip N characters before testing for `commence`. Do not use this along with `kbuf`.

Use `max_marks` to stop the simulation after a given number of watch marks are generated.

Use `track_calls` to generate watch marks for each syscall.

- `trackProgArgs` Track references to arguments provided to the program as argv values.
- `trackCGIArgs` Track references to arguments provided to the program as packaged by cgi-bin.
- `retrack` – Intended for use after modifying content of an input buffer in memory. This will track accesses to the input buffer. Note that this function does record additional IO operations, BUT WILL reflect subsequent access to the existing watch buffers. (TBD, terminate on access to remembered FD?)
- `trackFile` – Similar to `trackIO`, but provide a substring of a file name instead of a FD. It will track references to data read from the file.
- `trackXMLFile` – Currently works only with files opened by `xmlParseFile`. The function notes all memory malloc'd within `xmlParseFile` and then tracks its access, e.g., via `xmlGetParam`, adding watch marks as it goes.
- `showWatchMarks(old=False)` Display a list of watch marks (see [5.11.1](#)) created by `trackIO` or `injectIO`. Use the `old` flag to view stale watch marks, i.e., those that occurred prior to the origin (which gets reset on each data injection.)
- `goToDataMark(watch_mark)` Skip to the simulation cycle associated with the given `watch_mark`, which is an index into the list of data watch bookmarks generated by `trackIO` or `injectIO`.
- `findRef(offset)` Display a list of each watch mark index value corresponding to an application read of data at the given offset into an input file, e.g., either an `injectIO` file or a drive-driver file.
- `getWatchMarks()` Return a json list of watch marks created by `watchData` or `trackIO`, used by the IDA client.
- `tagIterator(index)` Tag a watch mark as being an iterator. The associated function is added to a file stored along with IDA functions. The intent is to avoid data watch events on each move, or access, e.g., a crc generator.
- `addIterator(address)` Tag a function as being an iterator. The address can be any address within the function. Use this if your `tagIterator` turns out to not reduce watch marks because it is repeatedly called by the calling function, which you would set as the iterator. Note however that you will lose any other watch marks that the iterator function might have generated.
- `trackFunctionWrite(fun)` Record all write operations that occur between entry and return from the named function.
- `goToWriteMark(write_mark)` Skip to the simulation cycle associated with the given `write_mark`, which is an index into the list of write watch bookmarks generated by `trackFunctionWrite`.
- `getWriteMarks()` return a json list of write marks created by `trackFunctionWrite`.
- `revTaintReg(reg)` – Back trace the source of the content the given register until either a system call, or a non-trivial computation (for evolving definitions of “non-trivial”).

- **revTaintAddr(addr)** – Back trace the source of the content the given address until either a system call, or a non-trivial computation
- **injectIO(iofile, stay=False, n=1, target=None, targetFD=None, cover=False, break_on=None, trace=False, mark_logs=False, no_reset=False, max_marks=None, trace_all=False, trace_fd=None, malloc=False)** – Assumes you have previously used **prepInject** to create a snapshot (see 5.12.3 and that snapshot has been loaded. The content of the given **iofile** is written into the read buffer, and the register reflecting the returned count is modified to reflect the size of the file. If **stay** is **False**, the **retrack** function is then invoked. If **keep_size** is set, then the size register will not be altered, which is intended for use when replaying files trimmed by AFL. This **injectIO** function intended use is to rapidly observe execution paths for variations in input data. The command will automatically run the **debugPidGroup** command on the current TID. The optional **n** parameter causes the data file to be treated as **n** different UDP packets and will be split based on the **AFL_MAX_LEN** value. The optional **count** value supresses generation of watchmarks until the given number of read/recv calls were made. The optional **target** and **targetFD** parameters name a different process and its FD whose data references are to be tracked. For example, if the original receiving process does some processing and sends derivative data to a pipe, the target could be the process at the read side of the pipe. If the **cover** parameter is true, code coverage will be tracked and saved in a hits file. This may take a while – wait for the Simics prompt to display a message such as **Coverage saveCoverage to....** Other options:
 - **break_on** will break on execution of the given address.
 - **trace** option will generate a system call trace instead of watch marks, and the **trace_all** to trace system calls from all processes.
 - **mark_logs** to include log entries generated using **traceFile** or **traceFD** within the watch marks.
 - **trace_fd** to name a FD to be traced and include it in watch marks per **mark_logs** (avoids having to separately issue the **traceFD** command).
 - **no_reset** to stop the simulation rather than resetting the origin, e.g., to adjust a return value from a read.
 - **max_marks** to cause the simulation to stop after generated that number of watch marks. This value includes any marks that become stale due to origin resets.
 - **malloc** Generate watch marks for each malloc call
 - **track_malloc** Track data within malloc buffers. May be helpful if complex data movement from a buffer to a malloc'ed buffer is not caught by **memsomething** functions.
- **traceInject(iofile)** – (DEPRECATED) See the **trace_all** options to **injectIO** and **playAFL**.
- **traceMalloc** – track calls to the malloc and free functions and includes those events in the list of watch marks. Use **showMalloc** to then list by pid, block address and size. And use **saveMalloc** to save the open mallocs as a json, e.g., for use by **watchMallocMem**.
- **watchData** – run forward until a specified memory area is read. Intended for use in finding references to data consumed via a read system call. Data watch parameters are automatically set on a read during a debug session, allowing the analyst to simply invoke the **watchData** function to find references to the buffer. List data watches using **showDataWatch**. Note however, that data watches are based on the **len** field given in the read or recv, and thus data references are not necessarily to data actually read (e.g., a read of ten bytes that returns one byte would break on a reference to the fifth byte in the buffer.)
- **watchMallocMem** – Reads malloc addresses from a file created by **watchMallocMem** and sets a watch on each block, reporting on reads. Also highlights the reading of any word that is a code address, e.g., a function pointer.

3.7 Vulnerability detection

- **catchCorruption** – Watch for events symptomatic of memory corruption errors, e.g., SEGV or SIGILL exceptions resulting from buffer overflows. This is automatically enabled during debug sessions. Refer to A.2 for information about what we mean by SEGV, and how we catch it.
- **watchROP** – Watch for return instructions that do not seem to follow calls. This is available while debugging a process.

3.8 System modification

- **writeReg** – Write value to a register (Note: deletes existing bookmarks.)
- **writeWord** – Write word to an address. This function will delete existing bookmarks and create a new origin. If there are data watch marks, i.e., from a trackIO, these are deleted except for any that are equal to the current cycle. The upshot of this is that if you want to modify memory and then rerun a trackIO, do so while at the first datamark.
- **writeString** – Write a string to an address. Use double escapes, e.g., two backslashes and an n for a newline. (Note: deletes existing bookmarks.)
- **writeBytesFromFile** – Write the bytes from a file into a given address. Provide the **watch=True** option to set a data watch on the buffer.
- **runToDmod(dmod_file)** – Run until a specified read or write system call is encountered, and modify system memory such that the result of the read or write is augmented by a named Dmod directive file. See [5.8](#)
- **rmAllDmods()** – Remove all Dmods. Use this if the Dmods are needed to bring the system to a suitable state and that state has been reached, i.e., you do not anticipate the need for further modifications going forward. The intended use is primarily to remove noise from the logs and potentially speed up the simulation.
- **showDmods()** – Show all Dmods currently loaded.
- **readReplace(readreplace_file)** – Replace memory content with a given hex string when that memory address is first read by the program. See [5.8.1](#).
- **modFunction(fun, offset, word)** Write the given word at an offset from the start of the named function. Intended for use in setting return values, e.g., force eax to zero upon return. Bring your own machine code. TBD accept an assembly statement?
- **showLinks** – Display symbolic names of network connections. These may then be used in DMOD directives, or manually using Simics **connect** and **disconnect** commands.
- **jumper(from, to)** – Alter control flow by causing execution to jump to a given address whenever a given from address is hit. See the **EXECUTION_JUMPERS** ini environment value.

3.9 Code coverage

The following functions are intended for manual, ad-hoc code coverage analysis. Also see the functions listed under [3.10](#) and the discussion of fuzzing in [6](#).

- **mapCoverage(backstop=False, dead_zone=False)** – Set breakpoints on all basic blocks in the text segment and use them to track code coverage. The basic block coverage is saved in a <prog>.hits file in IDA db directory, and will be read by the IDA client when the **color** option is used. Subsequent uses of **mapCoverage** will reset the coverage tracking. Use **stopCoverage** to stop basic block tracking. The basic blocks database is read from the .blocks file created by the **idaDump.py** script or the Ghidra plugin's dump artifacts function. See section [5.13](#) for information on how the IDA client represents code coverage and highlights branches that have not been taken. The **backstop** option causes execution to stop when **BACK_STOP_CYCLES** pass without hitting a basic block. The **dead_zone** option will record all basic blocks into a file that can later be used to ignore those basic blocks. This is intended to remove background noise, e.g., behavior of threads of a process having nothing to do with your input data. See the **IGNORE_BB_LIST** ini parameter.
- **showCoverage** – display summary of basic block coverage. Also see the IDA client **color** and **clear** options.
- **goToBasicBlock** – Skip the simulation state to the first hit of the block named by an address (requires use of **mapCoverage**..)
- **addJumper(from, to)** – Define a jumper to cause execution to skip from a from block to a to block. These are intended for use with code coverage and fuzzing (see [6.5](#)). Use **jumper(from, to)** for general execution jumpers.

3.10 Fuzzing

This list below describes RESim commands related to fuzzing. See section 6 for a fuller discussion of fuzzing with AFL and RESim

- **afl(fname=None, target=None, targetFD=None, dead=False, count=1 test_file=None, commence_params=None, primer=None)** – Typically the **runAFL** utility (from a bash shell in your workspace directory) is used instead of this command, which can be used for testing with the **test_file** option. Uses a network connection to communicate AFL server github.com/mfthomps/AFL. The simulation is assumed to loaded from a checkpoint created using **prepInject**. Provide **fname** to name a library, i.e., if the program's main text blocks is not what is to be instrumented. Use **tt target** and **targetFD** to name some other process and the FD from which it reads if the fuzzing target is other than the consumer of the AFL-generated data. The **count** option will delay commencement of code coverage tracking until the given number of successful read/recv calls have occurred on the FD from the **prepInject**, or **targetFD**. The **commence_params** option names a file containing parameters computed for for the count. If the file does not exist, it is created and populated, and then used on subsequent runs of **afl** or **playAFL** as an optimization and in lieu of the **targetFD** and **count** fields. The **dead** option creates a filter to avoid instrumenting basic blocks hit by other threads. Use **test_file** to test RESim's AFL fuzzing portion on a selected file without starting AFL. Use the **primer** option if the **target** option is provided and there are constraints on inputs that lead to the target being scheduled so code coverage can be mapped. The primer should be an input file that leads to the scheduling of the target. See section 6 for details.
- **prepInject(fd, configfile, count=1, commence=string)** – Prepares a simulation for injectIO or fuzzing by running to an input system call for the given FD. It saves the post-call state in a snapshot file with the given name. Saved state includes the syscall call and return instruction addresses for use in multi-packet UDP fuzzing, and buffer address and size. Note using this with TCP and real networks can lead to undefined behavior, and thus you should use a driver computer instead of real networks. Use the count to stop the simulation subsequent to the nth input system call, e.g., to get an initial state for fuzzing that is predicated on a preamble exchange. This may require use of the **simics-magic** program on a driver computer. See 5.12.3 for details.

Note that the data you provide for **prepInject** should not include multiple UDP packets because you want the **recv** to not see more data after consuming your subsequent data injections.

Consider providing **prepInject** with largish amounts of data to ensure buffers that may cross page boundaries are populated.

Use the **commence=string** option to ignore input that does not start with the given string.

For TCP traffic, one approach is to use **runToAccept** and then use **prepInject** on the resulting FD.

The **prepInject** function works by observing buffers provided for data read via kernel system calls, e.g., **recv**. And this only works if when RESim has recorded the system call, e.g., while debugging a process. If a process enters the kernel, e.g, via **select**, during a period in which the process is not being debugged, then RESim will not note its return to user space and thus will observe the location of its associated data. This may not be an issue with TCP traffic in which you wish to inject data subsequent to the **attach** because the kernel will be blocking on a **select** or the **attach** itself and thus RESim will see the subsequent calls to **recv**.

For UDP traffic, if you have a system state in which you've run forward subsequent to debugging, e.g., you used **debugProc** on the process and then subsequently ran forward without debugging, you can insure that RESim sees your data by sending multiple items to the target and using the count option.

- **prepInjectWatch(watchmark, configfile)** – For use when preparing to inject data directly into kernel buffers. This call is intended for use when the application makes multiple kernel read/recv calls, e.g., character-at-a-time reads. Similar to **prepInject** but takes the index of a watch mark that is a call to **read/recieve**. Generate the data marks by using **trackIO**, and providing the maximum amount of data you expect to read from kernel buffers. Use of the resulting snapshot with AFL or **injectIO** will cause the data to be injected into a kernel buffer. The snapshot is made just prior to the kernel commencing to move data from its buffer to the user space buffer. Note this command does not support running forward past the application references to the input data, e.g., the kernel may detect corruption on the **recv** following the end of the current data. Do not try to run these checkpoints forward to establish new checkpoints. It is important that received data all appears within the kernel buffer before the application is returned to from its **recv** call. Otherwise, **injectIO** and AFL sessions will have kernel buffers overwritten by data on the wire. Consider using **drive-driver** command as described in 5.10.1.

- **prepInjectAddr(addr, configfile)** – For use when preparing to inject data directly into arbitrary application memory, e.g., when fuzzing argv parameters to a program. Consider priming the prepInject state with enough data to cross any page boundaries that might be part of the buffer.
- **playAFL(dfile, afl_mode=False, trace_all=False, fname=None, target=None, targetFD=None, primer=None)** – Play data input files discovered by AFL. The **dfile** parameter is a single file, a relative directory, or names a subdirectory of an output directory relative to the **AFL_DATA** environment variable). If a relative directory, then each file in that directory is played and coverage output files are created in local directory named **coverage**. If a directory relative to **AFL_DATA** then each file in the target's **queue** subdirectory(ies) will be played with code coverage tracked with new coverage files generated in the **AFL_DATA** coverage subdirectory. If a single file, the coverage will be stored in the **/tmp** directory. **Note:** While this command is running, you may see a static **simics>** prompt. That does not mean that Simics is not running – it is an artifact of the implementation. When running the command, you'll know it has finished when it displays where it stored the hits file. Tail the log if you suspect it has hung. Provide a library name as **fname**, e.g., **libc.so.6**, to record coverage of a library. Note if **afl_mode** is set, no hits file is generated. Use the **primer** option if the **target** option is provided and there are constraints on inputs that lead to the target being scheduled so code coverage can be mapped. The primer should be an input file that leads to the scheduling of the target.

If the given **dfile** is a file, then only that file will be played. The **afl_mode** switch attempts to reproduce how the file was played in AFL, e.g., without temporary breakpoints and with the AFL backstop cycles. However, if you wish to test the RESim portion of AFL fuzzing, use the **afl** command with the **test_file** option. The **trace_all** switch will generate a system call trace. These are intended for use in discovering why AFL behaves in unexpected ways, e.g., lots of hangs, long sessions.

Use the **target** parameter if the process whose coverage is to be tracked differs from the process into which data is to be injected. If that occurs on a different cell, preface the name of the target process with the cell name followed by a colon. If you do not want to start tracking coverage until the target has performed some number of read calls, provide the optional **targetFD** and **count** parameters. For example, if you want to commence coverage tracking after the target has returned from two read, provide a count of 2.

The **playAFL** command generates a coverage file for each AFL queue file and those are stored in a **coverage** directory alongside the **queue** directory. Those individual coverage files are intended to be read by the **findBB.py** utility to find sessions that lead to BNTs. The **runPlay** stand-alone command launches parallel instances of RESim running the **playAFL**, for use in replaying sessions created by parallel fuzzing. Note however this does not generate a new total hits file, it only creates coverage files.

If processes exit during replay, those will be reported after **playAFL** completes replaying the sessions.

See the **IGNORE_BB_LIST** and code coverage discussion at [5.13](#) to understand how to ignore some basic blocks.

- **crashReport(dfile)** Generate crash reports for each crash discovered by AFL. The given **dfile** is as defined in the **playAFL** command and may be a path to a single file. The recommended method for processing multiple crashes found by AFL is to use the **crashReport.py** utility (see section [13](#) to launch RESim multiple times, one for each data file. The reports are written to **/tmp/crash_reports**.
- **replayAFL(target, index, FD, [instance=N], cover=False, afl_mode=False)** – Replay a specific AFL session named by a target and index identifier, e.g., **0012**. Use the optional **instance** if parallel AFL is used. This function assumes a driver component. The driver will use the **sendDriver.sh** script to send the named file and a client program to the target using an address and port per the ini file **TARGET_IP** and **TARGET_PORT** values. For UDP sessions, RESim will use the **trackIO** function on the given **FD**. Use this to analyze multipacket processing. For TCP, use **replayAFLTCP** and the **FD** will be used to catch an **accept** system call. RESim will then use the returned **FD** for **trackIO**. Set **cover** to **True** to generate a coverage hits file. You can then start IDA with the **color** option, which will read that hits file. You can then double click on entries in the IDA BNT window to skip execution to the first hit of the selected BNT source block. Set **afl_mode** to report on blocks hit over 256 times in a session.

Note that function uses a driver to send data, and this it should not be used with a **prepInject** snapshot.

- **aflInject(target, index)** – Replay an AFL session, similar to **replayAFL**, except the **injectIO** function is used without a driver component. Thus, only single-packet processing can be analyzed. Use **aflInjectTCP** for TCP sessions.
- **trackAFL(target)** – Use **injectIO** to replay all AFL queue files for a given target, and store the set of resulting watch marks as json files in the AFL output directory. Only the first packet is tracked. If the

AFL sessions ran in parallel, or include multiple packets, then use the `runTrack` utility to start new Simics sessions for each queue file within each parallel workspace. Use the `--trace_all` option to create system call traces instead of watchmarks.

- `injectToBB(bb, fname, target, targetFD)` – Find an AFL input file that leads to the given BB and play it using `injectIO`, breaking at the BB. The AFL target is assumed to be the current RESim workspace directory name. Note the simulation breaks at the BB, and thus the watch marks do not reflect subsequent processing – it they will not reflect any watch mark in the target BB. (TBD, modify to process/track everything, and then skip back to the first hit of the BB.) Provide a target and targetFD if the process is other than that scheduled in the snapshot. Provide an fname if the bb is in a lib. The bb value is per the program addresses, and RESim will adjust it to reflect the lib load address.
- Also see utilities listed in section 13, e.g., to find branches not taken and AFL sessions that reach a given basic block.

3.11 Msc

- `runToCycle(cycle)` – Run to an absolute cpu cycle on the current cpu. Useful for moving simulation forward to prepare to debug some process that is not created until after much processing. Cycles reported in syscall logs can help get you close to the point in time.
- `runToSeconds(seconds)` – Run to a simulation time in seconds .
- `satisfyCondition` – (experimental) Assess the comparison instruction at a given address and attempt to satisfy it by altering input data. Initially handles simple ARM `cmp reg, <value>` instructions. The reverse track function is used to determine the source of the register content, and if that is a receive, it alters the data to satisfy the comparison and then uses `retrack` to run forward and track the new execution flow.
- `idaMessage` – display the most recent message made available to IDA. For example, after a `runToBind`, this will display the FD that was bound to.
- `setTarget` – Select which simulated component to observe and affect with subsequently issued commands. Target names are as defined in the RESim configuration file used to start the session.
- `saveMemory(addr, size, fname)` Write a byte array of the given size read from the given addr into a file with the given name.
- `pageInfo(addr)` Use saved kernel page table addresses to get the physical address of a given linear address. This is useful when the current context is not that of the typical kernel state.

4 Defining a target system

This section assumes some familiarity with Simics. RESim is invoked from a Simics workspace that contains a RESim configuration file. This configuration file identifies disk images used in the simulation and defines network MAC addresses. The file uses ini format and has at least two sections: and ENV section and one section per computer that will be part of the simulation. An example RESim configuration file is at:

`$RESIM/simics/workspace/mytarget.ini`

A full example target is available as described in section 8.

4.1 ENV section

The following environment variables are defined in the ENV section of the configuration file:

- `RUN_FROM_SNAP` The name of a snapshot created via the `@cgc.writeConfig` command or the Simics write-configuration command. The simulation will be started from this snapshot.
- `RESIM_TARGET` Name of the host that is to be the target of RESim analysis. Currently only one host can be analyzed during a given
- `CREATE_RESIM_PARAMS` If set to YES, the `getKernelParams` utility will be run instead of the RESim monitor. This will generate the Linux kernel parameters needed by RESim. Use the `@gkp.go()` command from the Simics command prompt to generate the file. For ARM platforms, use the optional `_64=True` to only look for arm64 entry points. Also see the `PLATFORM` env variable.

- **DRIVER_WAIT** Causes RESim delay boot of target platforms (i.e., those other than the driver) until the user runs the `@resim.go()` command. Intended to allow you (or scripts) to configure the driver platform after it boots, but before other platforms will boot.
- **BACK_STOP_CYCLES** Limits how far ahead a simulation will run during `trackIO`, `injectIO` or `playAFL` without the `afl_mode` flag. Defaults to 9,000,000. after the last data watch event, or the last code coverage hit.
- **AFL_BACK_STOP_CYCLES** Limits how far ahead a simulation will run after the last code coverage hit during an AFL session.
- **HANG_CYCLES** Limits how many cycles a simulation will run under AFL or `playAFL` until it is considered hung.
- **MONITOR** If set to NO, monitoring is not performed.
- **INIT_SCRIPT** Simics script to be run using `run-command-file`. For example, use this to attach real networks to avoid doing so after enabling reverse execution (which should be avoided due to Simics foibles). Do not use the name `init.simics`, it seems to confuse Simics.
- **PRE_INIT_SCRIPT** Simics script to be run prior to initializing the system.
- **STOP_ON_EXIT** Stops simulation when debugged process exits.
- **AFL_PAD** Cause data received from AFL to be padded to this minimum packet size, intended for services having a minimum udp packet size. This value will also be used to determine the size of writes to receive buffers when multi-packet AFL is used.
- **AFL_UDP_HEADER** Used with multi-packet AFL, will split data received from AFL at these strings. If single-packet UDP sessions are desired, leave this undefined. NOTE: initial data, e.g., AFL seeds should include the UDP header, (vice relying on filters) because the system will split input data into datagrams at the headers *prior* to applying filters. The default maximum number of UDP packets is 10. A maximum is needed, otherwise AFL gets carried away by duplicating the header. (TBD, add an env override.)
- **AFL_PACKET_FILTER** A data filter to modify or weed out data received from AFL for purposes of satisfying protocols or constraining fuzzing sessions to particular paths, e.g., to fuzz a single command dictated by a byte at a fixed offset. The value should be a .py file name, less the .py extension. The file must be relative to the Simics workspace and it must have a `filter(data, packet_num)` method that returns the desired string. For example, if the packet is to be rejected, return a byte array of zeros. Filters can also be used to compute and store CRCs within data and conform to other protocol requirements to guide AFL.
- **AFL_OUTPUT** – Optional path to an AFL afl-output directory that will be referenced when running `playAFL` and `crashReport` commands. Typically this should not be used, and the system will use the value of `AFL_DATA` defined in the `.bashrc`.
- **AFL_MAX_LEN** – Maximum size of a data injection for AFL, into either kernel or user buffers. Note the trimming occurs just prior to injection, and thus queue files and records of recent data, e.g., the `ihung` file, will not yet be trimmed.
- **AFL_MAX_PACKETS** – Maximum number of UDP packets that RESim will extract from a unit of fuzzed data from AFL.
- **AFL_STOP_ON_READ** – Cause AFL and replays to stop once the original read/recv call (or select) is hit.
- **AFL_STOP_ON_CLOSE** – Stop fuzzing or tracking if the FD is closed. AFL queue files will be given a suffix of `closed`.
- **AFL_NO_CALL_HAP** – An optimization applicable only to TCP sessions (kernel buffers) that prevents an AFL session from using a callHap until the buffer is consumed. This can speed up AFL about 30 percent, depending on the application.
- **AFL_SKIP_READ_N** – Experimental... Cause every Nth read/recv to cause RESim to skip over that kernel call. The intended use is odd cases where a TCP session does a series of reads, then a read with a large length, but then goes back and reads a deterministic amount of data.
- **IOCTL_COUNT_MAX** – Intended for use with data injection into TCP conversations, limits the number of calls to `ioctl` on the tracked FD before the session is stopped.

- **TARGET_IP** – IP address to use on driver fuctions such as **replayAFL**
- **TARGET_PORT** – Port to use on driver fuctions such as **replayAFL**
- **READ_LOOP** – What to do if dataWatch detects a read loop, e.g., a vary large CRC calculation. The read loop trigger defaults at 10,000 and is overridden by the **READ_LOOP_MAX**. A value of *quit* will cause the RESim session to exit (useful for crashReport). Otherwise, the data watch is stopped and execution continues, e.g., to arrive at a page boundary fault.
- **READ_LOOP_MAX** – Limit on dataWatch reads from a single buffer. a jumper will reset the reverse execution origin, and thus may not be suitable when using injectIO or crash analysis.
- **RESIM_LOG_LEVEL** – Set to INFO to suppress debug messages in the resim log.
- **SUSPEND_COVERAGE** – Names a file having a function start and end address, intended to supress code coverage tracking while processing a function.
- **FUNCTION.BACKSTOP** – Names a file having a function start addresses which, when hit, will cause an AFL iteration to stop.
- **CONFIG.COMMMAND** – Execute this Simics command after all components are loaded.
- **ADD_FROM_SNAP** The name of a snapshot created via the Simics write-configuration command. Intended for special case usage where a slow-loading component is started into a quiecient state, and then other components are added to the simulation.
- **INIT_FROM_SNAP** Like **RUN_FROM_SNAP**, but does not restore any of the RESim data from pickle files. Intended for use when a target is rebooted. Create a snapshot when shutdown seems completed and then run from that with this setting so that ASLR and other boot-dependent changes get reassessed.
- **ALLOW_REAL_EXTERNAL** If the **create_network** component parameter is used to create a dhcp service node, setting this value to TRUE will allow the service node to access external networks via dns and napt”

4.2 Target sections

Each computer within the simulation has its own section. The section name will be assigned as the name of the platform. The section items listed below that have a \$ prefix represent Simics CLI variables used within Simics scripts. If you define your own simics scripts (instead of using the generic scripts included with RESim), you may add arbitrary CLI variables to this section.

- **\$use_disk2** Whether a second disk is to be attached to computer.
- **\$use_disk3** Whether a third disk is to be attached to computer.
- **\$disk_image** Path to the boot image for the computer.
- **\$disk_size** Size of disk_image
- **\$disk2_image** Path to the 2nd disk
- **\$disk2_size** Size of 2nd disk_image
- **\$disk3_image** Path to the 3nd disk
- **\$disk3_size** Size of 3nd disk_image
- **\$mac_address_0** Enclose in double quotes
- **\$mac_address_1** Enclose in double quotes
- **\$mac_address_2** Enclose in double quotes
- **\$mac_address_3** Enclose in double quotes
- **\$eth_device** Alternet ethernet device, see [4.4](#) below.
- **\$create_network** Create a network service node and attach it to the named switch, intended to provide DHCP services to targets that require it. This overwrites any connections to ETH0. See [4.4.2](#). The service node will have NAPT and real DNS disabled.

- `$service_node_ip_address` If service node creation is requested, assign the node this ip address, which should be the gateway address and/or dhcp server address used by the target.
- `SIMICS_SCRIPT` Path to the Simics script file that defines the target system. This path is relative to the `target` directory of either the workspace, or the RESim repo under `simics/simicsScripts/targets`. For example,

`SIMICS_SCRIPT=x86-x58-ich10/genx86.simics`

would use the generic X86 platform distributed with RESim. See ?? for available scripts.

- `OS_TYPE` Either LINUX or LINUX64 RESim session.
- `RESIM_PARAM` Name of a parameter file created by getKernParams utility.
- `RESIM_UNISTD` Path to a Linux `unistd*.h` file that will be parsed to map system call numbers to system calls. For 64 bit Linux, use `RESIM_DIR/linux/ia64.tbl`
- `RESIM_ROOT_PREFIX` Path to the root of a file system containing copies of target executables. This is used by RESim read elf headers of target software and to locate analysis files generated by IDA Pro.
- `RESIM_ROOT_SUBDIRS` An optional semicolon separated list of subdirectories off of the `RESIM_ROOT_PREFIX` to limit the search for executable images.
- `BOOT_CHUNKS` The number of cycles to execute during boot between checks to determine if the component has booted enough to track its current task pointer. The intent is to keep this value low enough catch the system shortly after creation of the initial process. The default value is 900,000,000, which is too large for some ARM implementations. While components are booting, RESim uses the smallest `BOOT_CHUNKS` value assigned to any component that has not yet completed its boot.
- `DMOD` Optional file to pass to the `runToDmod` command once the component has booted. See 5.8.
- `READ_REPLACE` Optional file to cause memory values to be changed. See 5.8.1.
- `REG_SET` Optional file to cause register values to be changed. See 5.8.2.
- `PLATFORM` One of the following: `arm5`; `armMixed` (analyze 32 and 64 bit apps); `arm64` (analyze only 64 bit apps); `arm32` (analyze only 32 bit apps).
- `INTERACT_SCRIPT` Simics command script to be run after this component is loaded, which only occurs during boot, i.e., these are not run with snapshots.
- `ALWAYS_SCRIPT` Simics command script to be run on each startup, regardless of whether the run is from a snapshot.
- `ONLY_PROGS` Optional name of a file that lists the programs that are to be watched or traced. All others are ignored.
- `SKIP_PROGS` Optional name of a file that lists the programs that ignored.
- `ONLY_SO` Optional name of a file that lists the SO files to be parsed during a debugSnap function. See 3.4.
- `DLL_SKIP` Optional name of a file that identifies DLLs whose system calls are to be skipped. See Q in the Windows discussion.
- `TRACE_BUFFERS` – Name of a file with buffer trace directives for use in capturing data written to debug buffers (regardless of whether those buffers ever find their way to any program output). See 3.2
- `FUNCTION_NO_WATCH` – Name of a file listing functions whose data references are not to be tracked, e.g., to reduce noise generated by debug loggers. See 5.11.5
- `EXECUTION_JUMPERS` – Name of a file with jumpers (see the `jumper` command in 3.8), expressed as from/to addresses in hex. Syntax of the file is described in 5.7 Use `#` for comments.
- `LOAD_JUMPERS` – Load all defined jumpers automatically.
- `WORD_SIZES` – Name of a file that defines the word sizes used by programs. Each entry is a comm followed by either 4 or 8. This can be helpful for Windows systems that include 32 and 64 bit applications. Also see the `winProgSizes.py` utility.

- **IGNORE_BB_LIST** – File name to use when recording basic blocks that are to be ignored when running AFL or other coverage-related functions. If this parameter is not present, then the basic blocks will be recorded in a file named by the current snapshot with a suffix of `.dead`.
- **MAIN_LIBS** – File name of a list of DLLs or SOs. The `runToMainSO` command will run forward until one of these is entered.
- **USER_FUNS** – Name of a file listing user-defined functions to be recognized by the data tracking as memory copy functions when generating watch marks.
- **STUPID_CLOSERS** – Name of a file listing programs that make bulk calls to close for purposes of suspending tracing on a thread until done. See [A.5](#)
- **REVERSE_SPAN** – Number of cycles between micro checkpoints when reverse execution is enabled for Simics 7 See [5.11.6](#)

4.3 Target platforms

Example ini files for these target platforms are described in section [8](#).

Targets platforms from the RESim `simics/simicsScripts/targets` include:

- `x86-x58-ich10/genx86.simics` – An x86 platform with legacy BIOS that will boot many different x86 bootable disk images. Note the `launchRESIM.py` may select a different script in this directory, or the `resim-x86` script below.
- `resim-x86/resim-x86.simics` – An x86 platform with legacy BIOS used with Simics 7
- `arm` – The Simics ARM QSP platform. By default this boots the Simics-provided QSP kernel. Alternately you can set the `kernel_image` parameter to point to a different kernel. See the directory at: `simics/image_build/arm-qsp` for information on rebuilding the ARM QSP kernel. The `root_disk_image` should point to a root file system. A stock root file system can be found at: <https://nps.box.com/v/resim-arm-qsp-rootfs> The `user_disk_image` appears as `/dev/qspdc` when the stock QSP kernel boots. If it contains a partition table and file system, then the first partition is mounted on `/mnt` by the stock root fs as `/dev/qspdc1`.
- `integrator-cp` – A 32-bit ARM platform modeled after the Integrator-cp. A kernel and sample initrd image are at: <https://nps.box.com/v/resim-arm5-images>
- `armv8a/fvp.simics` – The Simics ARMv8-A Fixed Virtual Platform. See the READMEs at `simics/image_build/arm_fvp/` for information on building a kernel and ram disk.
- `armv8aJun26/fvp.simics` – The Simics ARMv8-A Fixed Virtual Platform that boots newer linux kernels, e.g., 7.1. See the READMEs at `simics/image_build/arm_fvp_jun26/README` for information on building a kernel
- `ppc32` – The Simics 32 bit PowerPC QSP platform. See the README at `simics/image_build/ppc-qsp` for information on building a kernel. Root file systems disks can be simple ext2 file system images.

4.4 Network definitions

Configuring the networks for a simulated system can require some trial and error and use of Wireshark to determine which logical ethernet devices are communicating on which simulated devices. If you are using the predefined driver component, see the Appendix [M](#) for mappings of addresses and device names. You may also start Wireshark from the Simics command line to observe which interfaces are connected to the different switches, e.g., `wireshark switch0` will display traffic on switch0. Take care to note MAC addresses and don't be fooled by packets routed through computers to other switches.

Four network switches are created, named `switch0`, `switch1`, `switch2` and `switch3`. And two network hubs, named `hub0` and `hub1`, are created. Each generic RESim computer has one or more network interfaces, depending on the type of platform. These are named `eth0`, `eth1` and `eth2`, and are assigned corresponding MAC addresses from the RESim configuration file. By default, each computer ethernet interface is connected to its correspondingly numbered switch. This topology may be modified via entries in computer sections of the RESim configuration file. For example, an entry of:

```
ETH0_SWITCH=switch2
```

would connect the eth0 device to switch2. A switch value of NONE prevents the ethernet device from being connected to any switch. Note there is no error checking or sanity testing. In the above example, you would also need to re-assign the eth2 device or it will attempt to attach two connections to the same switch port. Alternately you can connect the ethernet devices to the hub, e.g.,:

```
ETH0_SWITCH=hub0
```

Ethernet devices on the generic x86 platform default to the i82559 device defined by Simics on Simics 7 and the i82543gc on Simics 6. Use of the `$eth_dev=` entry lets you pick one of the following alternate ethernet devices:

```
i82559
i82546bg
i82543gc
dec21041
```

If using a VLAN, select the 82559. The i82543gc does not properly support VLANs. Note the 82559 device changes ethernet names, e.g., ens11 becomes ens11f0 and ens12 becomes ens12f0. Also, Simics 7 has trouble with the i82543gc, but seems to work with the 82559. Windows XP seems only to work with the dec21041 (not available on Simics 7).

Simics CLI variables are assigned to each computer ethernet link using the convention `$TARGET_eth0` where `TARGET` is the value of the configuration file section header for that component. Similarly, connections from the computer to the switches are named using the convention `$TARGET_switch0`. These CLI variable names may be referenced in user-supplied scripts, or in Dmod directives of type `match_cmd`. See 5.8.

The `eth1` cli name is assigned to the motherboard ethernet slot. the eth0, eth2 get northbridge slots and eth3 gets a southbridge pci slot.

The `showLinks` command displays symbolic names of network connections. These can be used in DMOD directives, or manually to alter the network topology using the Simics `disconnect` command, e.g.,

```
disconnect $box1_eth0 $box1_switch0
```

Each i82546bg device has two ethernet connections, with the 2nd having a MAC address one greater than the first. Currently, RESim only connects to the first one, though this should be fixed to allow both to be use (see `simicsScripts/targets/x86...`

4.4.1 Real network connections

Refer to the Simics *Ethernet Networking Technology Guide* for details on connecting the simulation to real networks. Also see the section in this guide on driver platforms (M and see the limitations described in P.1. By default, the Simics `connect-real-network` command creates a service node that provides NAPT forwarding of traffic from targets that can reach the service node. If you want the target to be able to reach external networks (e.g., to install packages), use the `ALLOW_REAL_EXTERNAL` environment value, otherwise real dns and napt are disabled.

4.4.2 DHCP

If you have an ARM or x86 target that requires DHCP to get an IP address, use the `create_network` environment variable, providing a value naming one of the switches to which a service node will be created. The first IP address by default to the device will be 10.10.0.100. That service node will have NAPT and real DNS disabled. Note you must set the switch value to NONE for the named switch. An alternative is to add a DHCP server to the driver.

4.4.3 Configuring VLANs

Create a VLAN switch by adding a `VLAN_N` to the ENV section of the ini file, where N identifies the switch. For example:

```
VLAN_1=3
```

will create a VLAN switch named `vswitch1`, and it will create a VLAN group id of 3 on the switch. Subsequent entries naming the same switch can be used to add additional VLAN groups to the switch. Then, when linking ethernet devices to the switch in the platform section, append the desired group number to the switch name. For example, the following will create a trunk connector for VLAN group 3 on switch `vswitch1`:

```
ETH0_SWITCH=vswitch1-3
```

To create a non-trunk connection (i.e., one that will not have VLAN group indicators in the ethernet frames), preface the group number with *nv*, e.g.,

```
ETH0_SWITCH=vswitch1-nv3
```

4.5 Driver component

Each simulation can have an optional driver component – designated by assigning the string **driver** to the corresponding section header. This component will be created first, and other components will not be created until the driver has caused a file named **driver-ready.flag** to be created within the workspace directory. Use the Simics Agent to create that file from the driver computer. This requires you copy the Simics agent onto the target and get it to run upon boot. It is intended that the agent will load scripts to generate traffic for the target computers. An example driver is described in Appendix M. See the **mapdriver.simics** script in the **simics/workspace** directory for an example of how to connect the driver to a real network, e.g., to use **drive-driver** script in a manner that disconnects the service node from the simulation.

5 Running the simulation

RESim sessions are started from the Simics workspace using the **resim** command (the path to which should be in your **PATH** environment variable).

RESim is controlled either through the **simics** command line, or using IDA Pro as described below. Interaction with the simulated computers, e.g., ssh'ing to a driver computer, requires that the simulation be running and not paused.

5.1 Installation

RESim assumes you have installed and are somewhat familiar with Simics. Versions 6 and 7 are supported as is the free version of Simics 7 from Intel. It also assumes you have IDA Pro or Ghidra. RESim can be used without IDA Pro or Ghidra, though you are limited to the command line and don't benefit from function analysis provided by IDA or Ghidra.

Note if you are using an NPS-provided RESim server, this RESim installation will have already been done. But see the IDA Pro installation below for information on installing IDA. See the *RESim Remote Access Guide* for information on remote access to RESim servers.

- Get RESim from the git repo:

```
git clone https://github.com/mfthomps/RESim.git
```

- Install python-magic. If on Simics 6, install the standard python3-magic. If on Simics 4, install it from gz file: `pip install <path>`

```
sudo pip install /mnt/re_images/python_pkgs/python-magic-0.4.15.tar.gz
```

- Install xterm

```
apt-get install xterm
```

- Define environment variables. In your **.bashrc**, define the following (place prior to the non-interactive return so they are picked up by ssh):
 - **RESIM_DIR** Path to your git clone.
 - **SIMDIR** Path to the Simics directory, e.g., `.../simics6/install/simics-6/simics-6.0.89`. The parent of this path must exactly match the *Installation Directory* setting in your *Intel Simics Package Manager* used to install Simics.
 - **IDA_DIR** Path to your IDA installation directory (define on system where you will run IDA).
 - **RESIM_IDA_DATA** Path to directory to hold data use by RESim and IDA. Typically `$HOME/ida_data`
 - **AFL_DIR** Path to directory containing the AFL executable (only needed to support fuzzing.)
 - **AFL_DATA** Path to directory containing the AFL seeds and output data.
 - **RESIM_IMAGE** Path to directory containing root file systems. It may also contain disk images as a convenience for canned example ini files, You may place disk images anywhere and name them from the ini file. See 8 for images that you may wish to download and place in this directory.

- IDA_ANALYSIS Path to where IDA (or Ghidra) will store analysis artifacts, and where RESim will look for them. Place this such that the parent directory is the same as the RESIM_IMAGE path parent directory, e.g.,

```
export RESIM_IMAGE=/some/path/to_your/images
export RESIM_ANALYSIS=/some/path/to_your/analysis
```

Consider placing these on a network share to facilitate sharing of images and analysis artifacts.

- Add \$RESIM_DIR/simics/bin and fuzz_bin to your path, e.g. by adding to your .profile,

```
export RESIM_DIR=[path to your git clone]
export PATH=$RESIM_DIR/simics/bin:$RESIM_DIR/simics/fuzz_bin:$PATH
```

- Install the RESim gdb-remote package which is needed to support with Ghidra. Use the Intel Simics Package Manager that you used to install Simics as follows: For Simics 7:

```
./intel-simics-package-manager-1.18.0/ispm install $RESIM_DIR/simics/packages/simics-pkg-200000-7.0
```

For Simics 6:

```
./intel-simics-package-manager-1.8.3/ispm install $RESIM_DIR/simics/packages/simics-pkg-200000-6.0
```

- Download the *driver* disk image, which is used to interact with targets from:

https://nps01-my.sharepoint.com/:u:/g/personal/mfthomps_nps_edu/IQCV_MiNKTuQULQgny7z3A0RAc51gxY

should be copied to:

```
$RESIM_IMAGES/driver/driver_22.disk0.hd_image.craff
```

- For analysis of Windows applications, install the *pev* package to get the readpe function for reading coff headers.
- Install the Capstone disassembler engine and python bindings to handle cases where RESim needs to analyze instructions that have not yet been paged in. For Ubuntu 22:

```
apt install -y capstone-tool pip
pip install capstone
```

For Ubuntu 24:

```
apt install python3-capstone
```

RESim assumes the python bindings are at /usr/local/lib/python3.10/dist-packages. Set a symlink if needed.

5.1.1 Missing Simics platform files

The following files may be missing from some versions of Simics 6:

```
(install directory)/simics-6/simics-qsp-arm-6.0.?.targets/qsp-arm/images/qsp.dtb
(install directory)/simics-6/simics-qsp-arm-6.0.?.targets/qsp-arm/images/uImage
```

Contact Wind River to get those files (or contact mfthomps at nps.edu).

5.1.2 IDA Pro installation

The following steps augment a standard IDA installation (which is not covered here.)

- The IDA Pro installation should be configured to use Python 2.7, which must be installed, e.g., using `apt-get install python2-dev`.
- If IDA and RESim run on different computers, e.g., remote access to RESim:
 - Clone the RESim git repo onto your local computer (i.e., where IDA will run)
 - Configure the `RESIM_DIR`, `RESIM_IDA_DIR` and `IDA_DIR` variables as described above.
 - Extend your `PATH` to include the `RESIM_DIR/simics/bin` as described above.
- In your `$IDA_DIR/cfg` directory, there are a number of xml files that need to be replaced with those found in `$RESIM_DIR/simics/ida/cfg`. Backup the original xmls first.
- Start Ida and configure the debugger to use remote gdb, and configure the Debugger process options to use localhost and port 9123. Alternately, if IDA is run on a different computer than Simics, and the Simics machine can bind to an external 9123 port, set the host to name the server on which Simics is running. Otherwise, communication between the IDA computer and the Simics computer will be via an ssh tunnel, which can be vary slow when large amounts of GDB data are exchanged.

5.1.3 Ghidra installation

See section 5.6 for information on installation and use of Ghidra with RESim.

5.1.4 AFL installation

Clone the AFL repo (<https://github.com/mfthomps/AFL.git>) Older environments may need to update binutils. Use `make` to build the executable. Confirm your bash shell environment variables include `AFL_DIR` and `AFL_DATA` Path per the installation instructions at 5.1. The shell from which you ssh to the Simics server that will run AFL must support X11. If you have OSX, install XQuartz.

5.2 Getting started

This section describes steps for creating and running RESim simulations. It is best to start with one of the examples.

5.2.1 Example systems

RESim includes a few examples to help you get familiar with the tool. These are described below in 8. The simplest example is CADET01.

5.2.2 Defining your own simulation

Steps to define and run your own RESim simulation are listed below. It is assumed you are familiar with basic Simics concepts and have a computer upon which Simics is installed and you've set your `.bashrc` to include environment variables defined above.

1. Create a Simics workspace using the `resim-ws.sh` script:

```
mkdir mywork; cd mywork
resim-ws.sh
```

2. Copy any desired files from `$RESIM/simics/workspace` or other workspaces into the new workspace. You may also create links to checkpoint directories in other workspaces.
3. Rename the `mytarget.ini` file and modify it as follows:
 - Set the `disk_image` entry to name paths to your target disk image.
 - Obtain the `unistd.32.h` or equivalent, for your target's kernel – this is used match system call numbers to calls. Name the file in the `RESIM_UNISTD` parameter.
 - Copy the target systems root file system, or a subset of the file system containing binaries of interest to the local computer and name that path in the `RESIM_ROOT_PREFIX` parameter. These images are used when analyzing specified programs, and are given to IDA Pro for analysis.

- Set the `CREATE_RESIM_PARAMS` parameter to `YES` so that the first run will create the kernel parameter file needed by RESim (unless the param file already exists, in which case you are done with initial setup).
4. Launch RESim using `resim mytarget`. That will start Simics and give you the Simics command prompt.
 5. Continue the simulation until the kernel appears to have booted, then stop. Try stopping before the initial processes complete, i.e., while output is still spewing on the boot console.
 6. Use the `@gkp.go()` command to generate the parameter file. This may take a while, and may require nominal interaction with the target system via its console, e.g., to schedule a new process. If it displays a message saying it is not in the kernel, try running ahead a bit, e.g., `r 10000` and try the `gkp.go` command again. For ARM platforms set the `PLATFORM` env value.
 7. After the parameters are created, quit Simics and remove the `CREATE_RESIM_PARAMS` parameter.
 8. Restart using the `resim` command. RESim will begin to boot the target and pause once it has confirmed the current task record. You may now use RESim commands listed in [3](#).

5.2.3 Kernels with ASLR

The kernel parameter management mostly accounts for ASLR by tracking differences between addresses recorded when the parameters were first captured, and addresses during the current boot, e.g, using the FS base address. However, if the disk image containing the kernel is modified, the addresses may change in a manner that is not accounted for. Repeat the kernel parameter creation steps to mitigate that.

5.2.4 Kernel Parameters for x86 32-bit compatibility

If a 64-bit x86 Linux environment includes 32-bit applications, there are different strategies for extending the param file to include the 32-bit entry points.

For older kernels, e.g., prior to Linux 4.0?, start by creating kernel parameters per the above, and then remove the `CREATE_RESIM_PARAMS` setting. Then run RESim until one of the 32-bit applications is about to be scheduled (e.g., invoke a 32 bit program via the command line). Then use `@cgc.writeConfig` to save the state. Modify the ini file to restore that state and set `CREATE_RESIM_PARAMS` to `YES`. Then start the monitor and use `@gkp.compat32()`. This will modify the kernel parameters in the .param file to include those needed to monitor 32-bit applications.

For newer kernels, it depends on whether there are significant 32-bit applications running to be seen by `getKernelParams`. If so, set the `PLATFORM` to `x86Mixed`. Then run `@cgc.record32BitEntry()` and cause the 32 bit application to run. Otherwise, create the param file as usual and save a backup of it. Then, boot the system and run to the point of a 32-bit application starting via `toProc`. You will now use the `record32BitEnter` function. Assuming the application first uses `int 128` system calls and then `sysenter` system calls, use `record32BitEnter('int 128')`. After that reports the param file has been saved, use `record32BitEnter('sysenter')`, which will also save the kernel's exit address for `sysexit`. The param file is not updated. If you run into trouble, always restore the origin param file before trying again to ensure you are starting from a known state.

5.3 Program binary locations

Files containing binaries of programs to analyze will exist in two or three locations:

1. Within the file system on a virtual disk image named in the ini file, e.g., in the root file system image.
2. Beneath the `RESIM_IMAGE` directory (defined in your .resimrc file) will be a subdirectory whose path is named in the RESim ini file as `RESIM_ROOT_PREFIX`, and the binaries will be beneath that, typically as a copy of the root file system.
3. If your IDA or GHIDRA computer is other than the one that runs Simics, then you must also have a `RESIM_IMAGE` directory on that computer, with a copy of the binaries for IDA or Ghidra to access (or an NSF mount of the files shared with the Simics host.)

The directory structure under the `RESIM_IMAGE` directory should look like:

```
RESIM__IMAGE
  project name 1
    root_fs    (named in RESIM_ROOT_PREFIX)
      etc bin ...
```

```

project name 2
  root_fs_xyz    (named in RESIM_ROOT_PREFIX)
  etc bin ...

```

5.4 Tracing processes

RESim provides a range of tracing functions. A typical starting point is to trace all system calls made by all of the processes executing on a target. The resulting artifacts identify network bindings and connections that may be of interest in vulnerability analysis. Once you've identified processes that interact with networks, you can send data to the target as described in 5.10 and observe the resulting system calls. In addition to tracing system calls, you can also trace every instruction executed by a process. Simics allows you to record all data read or written to files or devices. And it supports generating traces of all function calls made by a target. See the Process tracing section in 3.2 for commands that trace processes.

System call tracing on some target systems can be very slow due to massive quantities of system calls. The following steps can be taken to improve tracing:

- Experiment with tracing to determine where slowdowns might be occurring.
- Use the SKIP_PROGS or ONLY_PROGS to limit which programs are traced.
- Use autoMaze() to instruct RESim to automatically suppress system calls while within what appear to be intensive loops, e.g., many calls to get time.
- Use dll.skip to suppress call tracing while executing within a DLL, e.g., one that performs cryptographic functions.
- If you are simply looking for BINDs; use runToCall with `linger=True` instead of traceAll

5.5 IDA Pro

Once you have identified a program to be analyzed, e.g., by reviewing a system trace, open the program in IDA Pro as described below. Alternately, use Ghidra as described in 5.6. Note you must have the binaries organized as described in 5.3.

Get a separate terminal shell and cd to the location named by the RESIM_ROOT_PREFIX path named in the RESim configuration (ini) file. If you will run IDA on a different computer than the one running Simics, you will need access to that directory or a copy of it.

Use the `idaDump.sh <path to program>` script (which should be in your path) from a shell to create the IDA database and dump function and block information needed by RESim. Those artifacts will be stored in a directory relative to your IDA_ANALYSIS environment variable. If you are running in a shared environment, e.g., at NPS with analysis artifacts on a shared drive, then check that path to see if the function, block, imports and mangle artifacts were already created. If they were, you need not create them again.

Start IDA using the `runIda.sh` command (the path to which should be part of your PATH environment variable). Start IDA from the RESIM_ROOT_PREFIX directory, providing the path to the executable relative to that root prefix directory.

If IDA will run on a different computer than where Simics is running, and that remote computer can only be reached via SSH, include the host name as a command line option to open a ssh tunnel. See section 5.1.2 for additional information.

The first time you start IDA, use the **Debugger / Process options** to ensure your host is either localhost, or the name of the remote host on which Simics is running, and the port is 9123. Save those as the default. Then go the **Debugger setup** and select **Edit exceptions**. Change the SIGTRAP entry to pass the signal to the application; and to only log the exception. Save the settings. You should only need to do this step once per binary.

From the Simics command line (after starting RESim), run the `@cgc.debugProc<program>` command, naming the program of interest. RESim will continue the simulation until the program is `exec'd` and execution is transferred to the text segment, at which point it will pause. Assuming you started IDA with the `runIDA.sh` command, you may now press `shift-r`, which will cause IDA to attach to the process and load its RESim plugin. Alternately you can manually attach the process and run the IDAPython script at:

```
$RESIM/simics/ida/rev.py
```

Note: When you use `shift-r` or attach the debugger, IDA may generate a few pop-ups informing you of this or that. When that happens, select the *do not show again* checkbox. Then, save the IDA state, kill it and restart it. The goal is to suppress those pop-ups, which can corrupt the loading of IDA plugins when the debugger is attaching.

You can now run the commands found in the debugger help menu. Those commands generally invoke RESim commands listed in 3.

The RESim IDA client is not a robust debug environment in the sense that you can easily cause Simics to leave your intended execution context. There are attempts to catch the termination of the process being debugged. But in general, you should consider defining bookmarks to allow you to return to a known state.

There are situations where it is most productive, or necessary, to engage with the Simics command line directly. If you change the execution state via the command line, you can get IDA back in synch via the **Debugger / Resynch with server** menu selection. Or you can double-click a bookmark or watch mark.

RESim commands are available in IDA via the Debugger menu item, and via right clicking on addresses.

Setting IDA breakpoints requires that the current context be within the thread family being debugged. For example, if execution is currently stopped in some other process, use *Run to user space* to get execution back into the debugged process. You may then use IDA breakpoints.

The IDA step-over function will usually handle page faults and return you to the point after which the page is swapped in. If you find yourself in the kernel, use *Run to user space*.

5.5.1 RESim IDA menus and function keys

The IDA *Debugger* menu contains several custom RESim entries, including a RESim submenu at the bottom. A number of function keys are mapped to RESim functions, including some the over-ride IDA functions to provide similar functionality integrated with RESim. These include:

- F7 – step into
- F8 – step over
- shift F4 – run to cursor (the unshifted F4 may fail when running to the cursor within a loop, or may hit some other task.)
- alt-shift-F7 – reverse into
- alt-shift-F8 – reverse over

5.5.2 RESim IDA windows

The IDA plugin creates several tabs along side of the IDA *Stack view* window.

- Bookmarks – List of bookmarks created by the user (e.g., via right click), or created automatically when reverse tracking data. Double-clicking on a bookmark will skip execution to the associated cycle.
- stack trace – The stack trace, as determined by RESim. You may need to right-click / refresh to see this. Double-clicking on a item will take the *IDA View-EIP* to that address, but has no effect on the current execution cycle. Note, your previous focus must be in the *IDA View-EIP* window, otherwise you may end up changing the address of a *Hex View* window.
- data watch – List of watch marks generated by a trackIO or injectIO (which would be run from the Simics command line). Use right-click / refresh to view these. Double-click will skip the execution to the associated cycle.

5.5.3 RESim IDA options

Provide the `color` option to the `runIDA` command to cause IDA's graph display to be colored based on the content of the programs hits file found in the `$IDA_DIR`. Note that after running the `runPlay` or similar functions, you must copy the newly created `program.target.hits` file to the `program.hits` file.

The last option provided to the `runIDA` command can name a server on which RESim is running. The script will create an ssh tunnel to that server using port 9123. Note that some programs require a lot of data exchange between the gdb server and the client, and using an ssh tunnel can result in poor response times. Consider using the IDA Debugger/Process options to set the host explicitly.

5.5.4 IDA data decoding

A few data decoding options are available on the right-click on a register or memory address.

- Structure Field – interpret the data as an IDA structure
- Show Ptr – Read an address from the address found in a register and then set the *IDA Hex View-1* window to display the content at that address

- Show Addr – Read an address from the highlighted fields in a hexview window and jump the window to that address.
- Show Ascii Map – Interpret the next 256 bytes as a regx-type map and display characters corresponding to positions having a value of zero. Intended for use within regx type parsers that refer to compiled expressions.

5.5.5 Helping IDA with function names

Consider the following analysis from IDA of a ARM64 program.

```
v:0x0000007f8f7785a8 p:0x000f54d85a8  adrp x4, 0x7f8f793000
v:0x0000007f8f7785ac p:0x000f54d85ac  ldr x4, [x4, #3848]
v:0x0000007f8f7785b0 p:0x000f54d85b0  adrp x1, 0x7f8f77c000
v:0x0000007f8f7785b4 p:0x000f54d85b4  add x1, x1, #0xd6
v:0x0000007f8f7785b8 p:0x000f54d85b8  blr x4
```

Prior to re-basing, IDA knows that the ldr memory reference is to an imported function symbol, e.g., *strdup*, and will display that if hovered over. However, there is no comment indicating the function name, and the information is lost upon re-basing or debugger attachment. The RESim `idaDump.sh` utility uses IDA's xref database to generate a mapping of these type of function calls. This mapping is then used when creating stack traces, and it is used by the RESim `armBlrComments.py` script to add comments to IDA's display of blr call instructions indicating the called function name. The `armBlrComments.py` runs automatically when the plugin is loaded, and it can be run manually from file scripts, e.g., without debugger attachment.

Another function naming aide makes use of program logging subsystems. If a program binary is stripped of symbols, calls to loggers can sometimes reveal the name original name of the function. The RESim `nameFunsFromLog.py` script looks for some instances of that, and sets function names. After running the script, rerun the `idaDump.sh` utility so that the artifacts used in stack traces reflect the revised function name.

5.5.6 IDA Troubleshooting

IDA will sometimes get confused about memory regions, especially when debugging libraries. Use *Debugger / Manual Memory Regions* to help it. Symptoms include clicking on a register not resulting in the memory display in Hex View.

Use

```
set IDALOG=ida.log
then -z10000 tacked onto the ida command
```

to generate debug output for IDA

5.6 Ghidra

A preliminary RESim plugin for the Ghidra debugger is available at <https://github.com/mfthomps/RESimGhidraPlugins>. See that repo's README for installation instructions. It is assumed that the user has experience with the Ghidra platform.

Use of the Ghidra plugin requires a modified version of gdb, available at <https://github.com/mfthomps/binutils-gdb>. The modification causes gdb to display responses from gdb "monitor" commands using the same FD as used for other gdb command results. This is needed for Ghidra to see the results of monitor commands, which the plugin uses to interact with RESim.

Ghidra also requires a modified version of the Simics gdb-remote program if Windows targets are to be analyzed. Run the script at

```
$RESIM_DIR/simics/setup/add-gdb.sh
```

to add the modified package to your Simics installation.

Once the RESimGhidraPlugins extension is installed (per the repo README):

- Start Ghidra using `runGhidra.sh` script from a shell while in the target application root per the RESim ini file. Note you must have the binaries organized as described in 5.3.
- Create a Ghidra project and import the target binary into the project.
- Start the Ghidra debugger and open the project module.

- Click the menu item at **RESim / Dump artifacts** to dump function and block information needed by RESim. Those artifacts will be stored in a directory relative to your `IDA_ANALYSIS` environment variable. If you are running in a shared environment, e.g., at NPS with analysis artifacts on a shared drive, then check that path to see if the function, block, imports artifacts were already created. If they were, you need not create them again.
- Organize the RESim windows as desired, e.g., by dragging them to other tabbed windows. If RESim windows do not appear, try going to the menu **Window / Debugger** and select the missing windows. RESim windows have the icons that look like a spinning top or a bomb when Ghidra is sick..
- Use the RESim / Configure menu options to set the path to your customized gdb; the path to the file system root of the target binary, the host:port of your Simics host, and to set the ARM architecture if needed.
- Assuming you have the target debugging in RESim, attach to the GDB server by clicking the menu item at **RESim / Attach Simulation**.
- If the Dynamic list does not align with the analysis listing, select the *Modules* tab and click the red map button (2 headed arrow).
- The windows described for the IDA plugin are duplicated within the Ghidra plugin.
- Navigation and functions are available in the RESim menu, and by right clicking. For example, to reverse track the source of a memory location, put the cursor over the operand and right click...
- Ghidra provides logs, typically at `$HOME/.ghidra/.ghidra.11xx.PUBLIC/application.log` Or, if a dev build of Ghidra the logs may be in `HOME/.config/ghidra...`
- Ghidra stores its analysis in "project" directories, typically in home.
- To rebase Ghidra programs, open the Memory Map windows and press the house icon at the upper right.
- The RESim plugin for Ghidra includes *Memory reference Operand Hover*, which displays addresses and their content when hovering over an operand.
- Make sure the Dynamic window is set to "Track Program Counter (by Register)"
- Use **Modules / red-bidirectional-arrow** to ensure the listing matches the Dynamic.

5.7 Dynamic changes to execution control flow

RESim includes **jumpers** to cause execution to skip from a given source address to a destination address.

The execution jumpers are different from jumpers intended for use with code coverage and fuzzing, which are described in 6.5. Execution jumpers are defined in a file named by the `EXECUTION_JUMPERS` environment variable. The file may have multiple entries Each includes an optional `comm` parameter that names the process, which is intended to differentiate processes that share a library containing the jump addresses. Otherwise, the jumper will be active for all processes that execute that code in that library. Each entry also includes an optional 'break' keyword that causes the simulation to stop at the destination address. The break keyword must come last. The format is:

```
prog:addr addr [comm] [break] [patch] [replace]
```

Use `getS0` with `show_orig=True` to get the original load address of a program or library.

The jumper file is processed either as the result of the `loadJumpers` command, or when a program is debugged, e.g., via `debugSnap`. Jumpers use physical addresses. If a named program or library is not loaded when the jumper file is processed, a callback is set to handle it when it becomes loaded.

NOTE: Use of execution jumpers requires debugging and/or track threads to be enabled.

Use `comm` to only activate the jumper when a given `comm` program is scheduled. Use `break` to stop the simulation at the jumper. The `patch` and `replace` options can only be used alone. The former replaces the instruction at the from address with a branch to the destination. It does this when RESim starts and currently only works with VxWorks. The `replace` option replaces the instruction with the value given as the destination, which must be a hex string with or without a leading 0x. For example replacing an instruction with an ARM nop would be:

```
prog:addr e1a00000 replace (for arm)
prog:addr d503201f replace (for arm64)
```

Or, to set x0 to zero and return on arm64:

```
prog:addr D2800000 replace
prog:addr+4 D65F03C0 replace
```

5.8 Dynamic modifications to memory, registers, topology and control flow

RESim includes functions that cause system events to trigger dynamic modification of modeled elements and connections.

For example, a script that loads selected kernel modules could be augmented in memory to load alternate modules, e.g., those for which you have modeled devices. Modifying such a script on the volume image itself is not always convenient, e.g., *tripwire* functions might manage checksums of configuration files. It is therefore sometimes preferable to dynamically augment the software's perception of what is read.

There are three mechanisms for dynamic modifications: one that modifies system memory content the first time an address is read (known as a *ReadReplace*; the *regSet* function that sets a register to a given value whenever an execution address is hit; and a *Dmod* that modifies the simulated topology or results of file read, write or open operations.

Additionally, there is an experimental **modFunction** to write a given word at an offset from the start of a named function. Intended for use in setting return values, e.g., force *eax* to zero upon return. Bring your own machine code.

5.8.1 ReadReplace

The RESim **readReplace** function triggers on the first reading of selected memory addresses by named programs and replace memory content with a given hexadecimal string. The function takes as an input the name of a file containing one or more directives of the format:

```
operator lib:addr hexstring [comm] [writable]
```

The operator identifies how the target memory address is determined, and when the replacement should occur. It may be:

- **data** The target address is the given address. When that address is first read, the replacement is performed.
- **immediate** Similar to **data**, but the replacement is performed as soon as the target memory is mapped.
- **write** Similar to **data**, but the replacement is performed when the target memory address is written to. The replacement value is substituted for whatever was to be written by the program.
- **code** The target address is determined by dereferencing a pointer in the instruction at the given address when the instruction is reached. The **code** operator is used when the address is not relative to any particular program load address.

The **lib** field is the name of a program or shared object and value is a hexadecimal string that will be unhexified and written to the given address the first time the address is read. The **comm** value is optional. If given, the replace will only occur if the current thread matches the **comm** when the address is read. The **comm** value can have an optional address following a colon for use with **data** operators to limit the action to data references made from a specific code address. The **writable** option is only allowed with the **comm** option on operations other than **code**. The value defaults to *true*, in which case the replace will not occur until the page of memory is mapped as writable, e.g., after copy-on-write. Set this value to false if the target memory is a read-only segment.

Use the **READ_REPLACE** environment variable to automate execution of this function.

The **readReplace** is generally simpler than using *Dmods*. Consider an example in which a program repeatedly opens and reads from a file and compares the data read with a constant string in memory. And assume you'd like the compare operation to reflect a match despite the fact that the content of the file does not match the memory constant. One approach is to use a *Dmod* to alter the value of what is read from the file each time the file is read. However, if the program repeatedly re-reads the file, then the *Dmod* will occur repeatedly, resulting in a reset of the reverse execution origin (if enabled). An alternative is to use **readReplace** to replace the constant value the first time it is read.

When analysis reveals that a program would generate debugging messages if configuration values were suitably set, the **readReplace** can often alter configuration values accordingly. However, fuzzing performance may be better those debugging alterations are not included in the **prepInject** snapshot, particularly if the debugging results in rescheduling.

5.8.2 regSet

The RESim **regSet** function triggers on execution of a given program address by named programs and replace register content, or content of memory pointed to by a register, with a given hexadecimal value. The function takes as an input the name of a file containing one or more directives of the format:

```
comm addr keyword=value...
```

Where "comm" is the process comm name; addr is the program address; and keyword can be:

- **reg** name of a register whose value is to be set
- **reg_indirect** The register reg is indirect and the value will be written into the address within the register
- **value** Value to write into the reg register or into the address contained in the reg_indirect register.
- **jmp** Optional, Either lr or the address to jump to.

A legacy format without keywords is also supported. Each entry must be:

```
comm addr reg value
```

Use the **REG.SET** environment variable to automate execution of this function.

5.8.3 Dmods

The **runToDmod** function triggers on the reading or writing of a specified regular expression via the **write** or **read** system calls, or due to a failure when opening a file. The **runToDmod** function includes a parameter that names a file containing Dmod directives. In the *read* and *write* subfunctions listed below, the **match** string identifies the read or write operation that triggers the action. The format of directive files depend on the subfunction. Each subfunction also identifies whether it is triggered on a **read** or **write** operation or **open** operation, and may include an optional count value to control when the Dmod is removed from processing.

Dmod directive files start with a line that identifies the kind of dmod, the operation and optional values. The first field is the kind, which can be:

- **sub_replace** – Replace a substring within a read or write buffer.
- **script_replace** – Similar to **sub_replace** but intended for use on scripts. See the examples below.
- **full_replace** – Replace an entire read or write buffer.
- **match_cmd** – Execute a series of Simics commands when a given string is read or written.
- **open_replace** – Mask a failure to open an named file, and spoof subsequent reads from that file with content of a named file, or, if files are written to, create those files in a parallel file system on the host and use those in subsequent accesses, e.g., to support **/sys** device interfaces.
- **syscall** – Change return value of a system call.

The second field in the first line of the directive is the operation type, e.g., **read**, **write**, **open**. The operation can be followed by the following keyed options, e.g., **key=value**

- **count** – The Dmod will be removed after performing this number of operations.
- **comm** – Semicolon separated list of names of programs (comm) to filter on. Only processes with a matching comm will be considered for the Dmod.

Use the **DMOD** environment variable in the ini file to automate execution of Dmods.

- **sub_replace <operation>** – Replace a substring within a read or write buffer (specified by the **<operation>**), with a given string. The directives file includes one or more sets of directives. The directives use regular expression syntax. An example directives file looks like:

```
sub_replace read
#
# match
# was
# becomes
root:x:0:0:root
root:x:
root::
```


This example might be run when the `su` command is captured in the debugger.

- **script_replace** `<operation>`– Replace a substring within a script buffer with a given string. The intended use is to dynamically modify commands read from script files. Some implementations read 8k from the script file, operate on the next no-comment line, and then advance the file pointer and repeat. This causes your Dmod target to be read many times. With a `script_replace` Dmod, the target match is only considered when it matches the start of the first non-comment line of a read buffer. The directives file includes one or more sets of directives. The directives use regular expression syntax. An example directives file looks like:

```
script_replace read
#
# match
# was
# becomes
modprobe e1000e
modprobe e1000e
modprobe e100
```

This example might be run when the `su` command is captured in the debugger.

- **full_replace** `<operation>` – Replace the entire write or read buffer with a given string. The directives file includes a single directive whose replacement string may include multiple lines.

```
full_replace write
KERNEL=="eth*", NAME="eth
SUBSYSTEM=="net", ACTION=="add", DRIVERS=="?*", ATTR{address}=="00:e0:27:0f:ca:a8", \
    ATTR{dev_id}=="0x0", ATTR{type}=="1", KERNEL=="eth*", NAME="eth0"

# PCI device 0x8086:0x1001 (e1000e)
SUBSYSTEM=="net", ACTION=="add", DRIVERS=="?*", ATTR{address}=="00:e0:27:0f:ca:a9", \
    ATTR{dev_id}=="0x0", ATTR{type}=="1", KERNEL=="eth*", NAME="eth1"
```

- **match_cmd** `<operation>` Execute a list of Simics commands when the trigger string is found in a read or write buffer (per the `<operation>` field), and a separate substring is also found. If the trigger string is found, the function will terminate, i.e., no more `write` syscalls will be evaluated. Simics commands may reference CLI variables defined via the RESim configuration files, such as network connection names described in [4.4](#).

```
match_cmd write
#
# match (regx)
# was (regx)
# cmd
KERNEL=="eth\\*", NAME="eth
("00:e0:27:0f:ca:a8".*eth1|"00:e0:27:0f:ca:a9".*eth0)
disconnect $TG1_eth0 $TG1_switch0
disconnect $TG1_eth1 $TG1_switch1
connect $TG1_eth0 cnt1 = (switch1.get-free-connector)
connect $TG1_eth1 cnt1 = (switch0.get-free-connector)
```

- **open_replace** `open` If a given file or device is opened and fails (because it is missing), the return from `open` masks the failure and returns FD 99. When the application reads from that FD, the content of a data file is returned to the application. If the file is opened for writing, a host-local file is created in the `dmod_files` relative to the current workspace. That file is the access for subsequent reads and writes.

```
open_replace open
#
# fname
# becomes
/dev/mtd1
uboot_params.data
```

The *becomes* field is the name of a data file whose content is returned to the application when reading from the FD associated with the missing file. Note that `open_replace` does not affect `stat64` calls. Use `syscall dmods` if you wish to force a no-error return to the application.

- **syscall** Set the return value of a syscall. Behavior depends on the named syscall. Calls having an `filename` parameter (1st parameter) that matches a given string will return zero.

```
syscall stat64
/var/proc/net/xt_recent
```

causes the `stat64` call to return zero instead of an error when stating that file. The `stat64` kernel call returns a structure, which can be written by using the `bytes_file` option, e.g.,:

```
syscall stat64 bytes_file=stat64.bytes
```

. This is required if a script invokes `test -f`. Typically a single bytes file will work for different `stat64` calls.

```
syscall getpid comm=must-run-as-pid1
1
```

causes the the returned value of `getpid` to be 1 when called from `must-run-as-pid1`

5.8.4 Dynamic modifications to multiple computers

Dynamic modifications (Dmods) to system state are often performed early in the boot process, e.g., as network devices are being assigned addresses or kernel modules are being loaded. When simulated components boot, RESim monitors them to determine when each has booted far enough for the current task record to be of use, which is typically when the `init` process runs. RESim then pauses after all computers in the simulation have reached this initial state. Note though that the first computers to have reached their initial state will continue on while other computers are still booting. Thus, by the time RESim pauses, some computers may have executed beyond the point at which a dynamic modification was desired.

To avoid such race conditions, the RESim configuration file can optionally include the `DMOD` directive to identify Dmod files for each of the computers in a simulation. If present, the `runToDmod` command is executed as soon as the corresponding computer reaches its initial state. This ensures that dynamic modifications to those computers will occur while other computers in the simulation continue to boot to their initial states.

5.8.5 Dmods in the background

When dynamically analyzing a process or family of threads, RESim generally manages breakpoints for just those processes. This optimization can significantly speed up the analysis processes by entirely ignoring system calls and other events that occur in processes other than those under analysis. However, there are times when you need to dynamically modify some other process while debugging. Consider this example: you've directed RESim to debug some program X, and RESim has now detected the loading of X and has broken execution. You now want to observe X, e.g., tracking its I/O to some socket, however you know that at some point in the coming system execution, some other process Y will write a value that X will observe (perhaps indirectly). Use of the `background=True` option with the `runToDmod` command will cause RESim to monitor all system calls associated with the Dmod, even while you focus on the behavior of X.

5.8.6 Manual modifications to topology

Simics `connect` and `disconnect` commands may be used at any point in the simulation to modify the network topology. See the discussion in 4.4 for naming network connections. Potential uses of this include:

- Remove real network connections that had been used to provision the simulation, i.e., to avoid real-world leakage into the simulation and resulting corruption of reverse execution.
- Disconnect computers that frequently send data to network ports of interest, i.e., to reduce noise and simplify your ability to send your own data to those ports.

5.9 Tracing processes

System calls made by processes are traced using functions described in 3.2. The resulting trace file artifacts are parsed by scripts in the RESim/postscripts directory to summarize system activity, e.g., to identify which processes bind to which ports.

In addition to system calls, selected buffers written by processes can be traced, e.g., to view log messages that never find their way into a system call. See 3.2.

5.10 Sending data

There are a variety of ways to send data to a target. The quick and easy way is to use the Simics real networks, which utilizes port forwarding. For example, if the target network is connected to switch0 and the target port is 62226 and the IP is 172.16.8.100, the following will forward traffic from the localhost port 62226 to the target:

```
connect-real-network-port-in target-port = 62226 host-port = 62226 \  
ethernet-link = switch0 target-ip = 172.16.8.200
```

The use of real networks has limitations described in P.1.

It is often preferable to send data via a driver computer. Driver computers are described in M. If you use the default RESim driver computer, then the **drive-driver** script provides a convenient way to send data from your host to the target via a driver.

5.10.1 Using drive-driver

The **drive-driver** utility uses command line options and *directive* files to control the transmission of input data from the Simics host to a driver computer and from there to the target. Use of **drive-driver** requires:

- The driver is configured to have an IP address that can communicate with the target IP address. This implies IP address settings and configuration of virtual switches to provide (virtual) physical connectivity. The former can be achieved with **driver-script.sh**, or use of the **iface** directive (see below). Connections between ethernet devices and switches are settings within the RESim ini file.
- A Simics *real network* is defined to allow the Simics host to communicate with the driver via SSH. This can be achieved with a Simics command file such as the **mapdirver.simics** script found in the sample workspace.

An example directive file for sending a UDP packet to a target is:

```
FILE=some.io  
IP=10.10.0.2  
PORT=5678  
HEADER=UdPcc  
SESSION=UDP
```

This directive would cause the driver to send the content of the file named **some.io** to port 5678 at IP address 10.10.0.2 of the target. While UDP **HEADER** field is required, it is only used when the IO file, e.g., **some.io** contains multiple UDP packets with that same UDP header. The system will then transmit those UDP packets separately. Otherwise, the header is ignored (but is required.) Assuming the above directive is in a file named **my.directive**, the command for this directive would be:

```
drive-driver my.directive -d
```

The **-d** option tells RESim to disconnect the real network interface once the driver computer has received everything it needs to make the transmission to the target. This is generally necessary if **trackIO** is used. See P.1 for an explanation.

The Simics host computer uses scp and ssh to send information to the driver via the real network interface. The default port is 4022, which can be changed by including the **-p** option followed by the desired port number.

Other options for **drive-driver** include:

- Data can be sent via TCP by setting **SESSION** to TCP. TCP directives should not include UDP header fields. A TCP conversation can be driven by including multiple IO files on separate lines.
- Broadcast UDP can be sent by setting **SRC_IP** to the source IP, e.g., the device that should broadcast the message.
- Set **SESSION** to client and provide a **CLIENT** entry that names a customized client modeled after the TCP client.

- Set SESSION to UDP_JSON to send a set of UDP packets from a json file generated using the genJson.py utility.
- Set SESSION to TCP_JSON to send TCP data from a json file generated using the genJson.py utility. Each json entry is sent by the client, which then sleeps for 1 second before sending the next entry. If you have captured traffic to a web server via traceFD with the `web=True` option, provide that file name to genJson.py along with the `--web` option to generate json entries per the delimiters inserted by RESim.
- The driver can run a TCP server by setting SESSION to SERVERTCP. In this case, the server will listen on the IP and port named in the directive. When the server accepts a connection, it will send the IO file content and then perform a receive.
- TCP replay of a PCAP file occurs by setting SESSION to REPLAY. The FILE values are assumed to be PCAPs. The IFACE value should be the name of the ethernet device on the driver, e.g., `ens25` from which the data should be sent. The Unix `tcpreplay` utility is then invoked on the driver.
- Arbitrary scripts can be run on the driver by setting SESSION to COMMAND and including COMMAND tag whose values are commands to execute from the driver.
- The IFACE directive will define a new IP address for a given ethernet interface. For example:

```
FILE=z2000.io
iface=ens12:172.33.15.101
PORT=80
ip=0.0.0.0
SESSION=serverTCP
```

Use of Scapy to send UDP packets from a json file from a specified device, IP and port as the source. For example:

```
DEVICE=ens12f0
IP=10.10.0.100
PORT=20002
SRC_IP=10.10.0.100
SRC_PORT=5678
SESSION=UDP_JSON
FILE=udp_packets.json
```

In the above example, Scapy is used to spoof the source of IP and Port number in the UDP packets. The FILE is a json containing a list of UDP packets, e.g., created using genJsonIO.py

5.11 Tracking data

RESim tracks program references to data consumed by processes, generating a list of *watch marks* that represent execution points at which the process referenced input data. Each watch mark has an associated cycles value that can be skipped to, setting the execution state to that when the reference was made. If a data reference is made from within a libc function such as `strcmp`, the corresponding watch mark identifies the function, and its cycle is set at the return from the function. RESim debugger plugins include a GUI window with a list of data watch marks. Double clicking an entry in the list skips the simulation to that point of execution. The `showWatchMarks` command at the simics prompt displays the list, and `goToWatchMark` skips to the cycle. Watch marks include the information that varies depending on context, including:

- A numeric index, which can be provided to the `goToWatchMark` function.
- The execution address (pc) of the process at the time of the watch mark.
- The data address being referenced, and sometimes information about the buffer being accessed, e.g., offset 9 within buffer starting at 0x123444.
- The simulation cycle at the time of the watch mark.
- The name of the libc function and selected parameters where applicable.
- A subsequent comparison instruction if found, though these are sometimes not germane to the data reference.

- If the watch mark represents a copy of data, the address of the new buffer.
- An indication that the reference was a modification, and optionally that the target buffer will no longer be tracked.

Two RESim functions track input data:

1. `trackIO` Use this function to track data as it is read from a network, pipe, or file associated with a file descriptor.
2. `injectIO` This function tracks data that has been injected into simulated memory.

If a process copies data from an input buffer, RESim will watch memory within the copy along with the memory within the original buffer. RESim recognizes many common data copy functions, (e.g., `memcpy` and `strcpy`), and reports those as watch marks, identifying the old and new buffer addresses. RESim also recognizes *ad-hoc* data copies performed by the program directly rather than via a library call.

RESim data tracking uses a *backstop* value to determine when to stop looking for additional inputs or references to data buffers. The backstop is reset on each data reference or input via the target FD. If no references or inputs occur over the backstop duration, the simulation stops. The backstop value is in cycles, and useful values can vary wildly between environments. Use the `BACK_STOP_CYCLES` value in the RESim ini file to adjust this. When a backstop is reached and tracking stops, you have the option of continuing tracking, which will append to the list of watch marks until the next backstop is reached. Note that backstops are typically hit while not executing the process you are watching. The simulation stops in an seemingly arbitrary location and RESim does not automatically skip to any of the watchmarks. This is deliberate because a skip operation will terminate data tracking. It will also terminate *DMods*; *readReplace* and *jumper*s. If you wish to run forward past your watch marks, create a snapshot and reload from there to restore DMods, etc.

When a data watch buffer within the stack appears to be substantially overwritten, it is automatically removed from the watched buffers. The associated heuristic is intended to catch reuse of stack buffers to avoid false watch marks. The method is not perfect. Beware of false removals, i.e., you may miss accesses to buffers. And beware of missed removals, i.e., you may spend time trying to account for access to data within memory no longer retaining any semblance of your input data.

The `trackIO` function tracks data read via kernel system calls such as `recv`, and this only works if RESim has recorded the system call while debugging a process. If a process enters the kernel, e.g. via a blocking `recv`, during a period in which the process is not being debugged, then RESim will not note its return to user space and thus will not track associated data. One alternative is to send two packets with the intent of tracking references to the 2nd. This issue can also arise when handling asynchronous IO in Windows, but can usually be solved by either sending two packets, or ensuring that debugging is applied when the targeted `recv` call is made.

5.11.1 Watch Marks

Watch marks identify references to input data made by target applications, e.g., a read of a byte within an input buffer. Each mark describes the reference. For example:

```
18 Read 4 from 0x0837eb3c offset 12 into 0x0837eb30 (buf size 20) ip:0x8072c22 tid:1022
```

describes a reference that occurs at IP 0x8072c22. It says that four bytes were read from 0x0837eb3c, which is twelve bytes into some input buffer (or copy thereof) that begins at address 837eb30 and is twenty bytes long.

Copying of buffers is noted by use of *Copy* instead of *Read*. A copy is typically due to a `memcpy` or `strcpy`. If the phrase *Ad-hoc* occurs within the watch mark, that implies the copy occurred via a sequence of moves from memory to a register and then to a different memory location. Note these copies occur in the context of data processing that may exceed the copy, and thus they should be explored, e.g., to find comparisons or other conditions. A *Write* watch mark indicates the content of a watched buffer is modified. Watch marks may also reflect string comparisons, e.g., via the `strcmp` function.

There may be cases where RESim reports references to buffers that occur after the buffer was reused, e.g. a stack buffer. RESim catches some, but not all, cases of stack buffer reuse.

When injecting data, e.g., via `injectIO` (see the following subsection), the list of referable watch marks is reset on each data injection. Thus, if the `injectIO` function injects three packets into the target application simulated memory, only the watch marks recorded after the third packet will be in the list of watch marks that can be skipped to. The older, stale watch marks can be viewed using the `old` flag in the `showWatchMarks` command. If multiple injections are required, consider inject data into the driver, as described below.

Watch mark entries may include information about compare or test instructions that occur following data references. For example, this watch mark:

16 Read 1 from 0x000ce306 offset 18 into 0x000ce2f4 (buf size 97) cmp r12, r1 ip:0x35cf0 tid:1037

indicates that `cmp r12, 31` occurs within several instructions of the data reference. These indicators are generated as hints and do not necessarily reflect that the comparison is relevant to the input data reference. With ARM programs, some watch marks may reflect “CBR” (compare before reference) as the comparison, which means that the data reference occurs subsequent to a `cmp` instruction and prior to a conditional branch, i.e., the conditional branch is not dependent on the data reference.

Watch marks might suggest permutations of input data that might lead to new code paths. If there are multiple string compares, consider using the RESim automated functions for generating new input files as described in 5.14.

5.11.2 Sending data to be tracked

In general, data should be sent to the target from a driver or other simulated computer rather than via a real network. Use of real networks to send data for `prepInject` may fail because of the problems associated with real-world leakage, see P.1. Even using a real network to ssh into a driver to send the data may lead to corruption, particularly if the target process is waiting in the kernel for data. The use of ssh and scp can be made safe by executing the `simics/magic/simics-magic` binary on an x86 driver just after starting `prepInject` but prior to sending data to the target. That program executes a Simics *magic instruction* that RESim detects and uses to reset the origin for reverse execution during performance of `prepInject`. The use of this magic is currently required for all cases where the receive call is waiting in the kernel, whether or not ssh is used. See the `simics/bin/driver-driver.py` program for an example of using the magic instruction when sending data from the driver. Using the `-d` option will cause it to establish a new origin and will disconnect the driver from the network simulation.

5.11.3 Tracking data through pipes

The `traceReports.sh` script creates a `pipes.txt` file (see 3.2. RESim looks for this file in the workspace and will track data through the pipes defined therein. For example, if a thread reads data from a network and then writes that data to a pipe, RESim will then watch for a thread to read data from that same pipe.

5.11.4 Tracking libraries

The data references of interest might occur in a shared library. See section A.3 for information on opening IDA to observe library references to data.

5.11.5 Suppress tracking of functions

Tracking of data references made by selected functions can be suppressed using the `FUNCTION_NO_WATCH` environment variable in the ini file. The file named by that env variable has entries of the form:

```
lib:function
```

where each entry names a program or library and function. Data references made while within the named functions will not be tracked.

5.11.6 User defined memory functions

Program functions that copy input data can be tagged using the `USER_FUNCTIONS` value in the ini file to name a file having entries such as:

```
<comm> <function> <value_type> <param1> <param2>
```

where `comm` is the program name and `function` is the name of the function per IDA or Ghidra. Currently, the value type and parameters must be `string`, `dest`, `src`. RESim data tracking will treat the function as a strepy, generating a single watchmark rather than watch marks for each memory reference. This feature can be useful when you encounter hashing or encoding schemes.

5.11.7 When are you done?

When tracking data, you need to decide when to stop looking for data references. The primary mechanism for this is the `BACK_STOP_CYCLES` setting. When tracking data, RESim declares it is done tracking after that number of execution cycles occur with no data references. Note this can mask interesting behavior (and faults) if the number of cycles is too small. Note also that failure to account for all data buffers can lead to tracking ending prematurely. When in doubt, test with ridiculously large cycle counts.

5.11.8 Warning

Use of real networks to generate tracked data can lead to silent corruption of the simulation unless care is taken to reset the reversing origin after the real network is done interacting with the system. See [P.1](#).

Also, modifying memory content or register content will effectively redefine the origin used for reversing. Do not use direct Simics commands to modify memory or registers, use the RESim wrappers.

5.11.9 Tracking manually modified data

A more ad-hoc approach can be applied to the results of a trackIO by manually modifying memory data:

- Skip the simulation to the point at which the original data was read, e.g., following the `recv` syscall. So this by double clicking on that Watch Mark, or use the `goToDataMark` function in RESim.
- Use `modifyMemory` to change data in memory. Note this will reset the reverse execution bookmarks, preventing you from skipping to any earlier point.
- Use the `retrack` function.
- You may repeat this as many times as needed, however instead of skipping to a Watch Mark, you can simply skip to the `origin` bookmark since that is now the point at which you previously modified memory.

5.11.10 Tracking backwards

RESim can track the origins of data named by a register or memory address using the `revTaintReg` and `revTaintAddr` functions, which are also available via the IDA client. These functions do not generate data watch bookmarks, rather, they generate bookmarks viewable with `listBookmarks` or in the IDA Bookmarks window. RESim will backtrace the data until one of three events:

- It finds a kernel read or `recv` call that wrote the data into memory;
- The origin is reached, i.e., the start of reverse execution; or
- An operation on the data is not easily traced to a previous memory location. Examples are multiplication and boolean transformations.

If the origin is reached, RESim will query the Watch Marks to attempt to trace the data back through buffer copies. If successful, it will identify the initial read buffer (e.g, the 3rd read) and the offset into that buffer. This output will appear at the Simics command line, or in the Ida output window, depending on where the command originated.

5.11.11 Removing unwanted traffic

Target network ports may receive periodic traffic from other components in the simulated topology, thus interfering with your crafted data streams. Offending network connections can be disconnected as described in [5.8.6](#).

5.12 Tracking injected data

Once trackIO is performed and you've reviewed input data references, you can repeat the tracking step with modified data. The recommended way to iteratively modify input data is to use `injectIO`, which injects data from a file into simulated memory. This subsection describes methods for injecting data and observing references to the data by applications. As described in detail in [5.12.3](#), data injection requires that the system be put into a suitable state in which some program is about to consume input data and we know the addresses of buffers that are to contain injected data. The `prepInject` and `prepInjectWatch` functions automate the steps needed to achieve this state and capture necessary information about buffer addresses and execution points. The RESim AFL fuzzing functions (see [6](#)) use a similar data injection scheme and rely on injection preparation steps described in [5.12.3](#).

The `injectIO` command includes a `trace=True` option that causes it to generate system call traces rather than watch marks.

The `playAFL` command injects data similar to `injectIO`, but it tracks code coverage and generates hit lists as described in [5.13](#).

A combined report of watch marks, system calls and coverage hits is created using the `injectReport` utility.

5.12.1 Where to inject data?

Data is injected into one of three kinds of memory buffers, depending on the nature of the application being analyzed:

1. An application buffer defined by a read or receive system call.
2. A kernel buffer from which data is copied into a user buffer during processing of a read or receive system call.
3. A buffer on a driver computer that a driver utility will read when constructing data to send to a target. This buffer is typically the result of a file read, e.g., JSON data that will be extracted and used to construct data passed to `send` system calls.

Injection into application buffers is often appropriate for UDP applications and others where the application obtains data from a system call and then processes that data. Applications that consume multiple UDP packets can also make use of injection into application buffers as long as the buffer address remains the same for each `RECV` system call. In such cases, RESim will note the location of the `RECV` call and jump over the kernel for each subsequent packet.

Injection into kernel buffers is useful for applications that do not read all data available from an interface, such as TCP applications that read a specific number of bytes from the kernel on each read. Such an application may violate assumptions made when injecting data directly into application memory, e.g., the assumption that the application will read all necessary data in a single read. For example, a non-conformant application may end up hanging on a `select` call because the kernel does not know there is more data to inject into the application memory. Another problem with partial reads is that they would break the use of `injectIO` since each injection into application memory resets the origin used with reverse execution.

TCP applications such as web servers that use large reads expecting the results to contain a single transaction, e.g., `GET` or `POST` command are typically not suitable for injecting data into kernel buffers. If you inject multiple transactions into the kernel buffer, the read will return them all and the application would ignore everything past the first transaction. Inject data into application buffers for such applications. In short, if the requested number of bytes on a `RECV` on a TCP connection is larger than the expected data, inject your data into user application buffers and not into the kernel.

The application whose consumption of data is to be tracked may not be the application that initially ingests the data. For example, one process may read from a network port and perform some data validation or transformation whose analysis is completed or otherwise not of interest. However that process may then send the data via a pipe or socket to another process. The `injectIO` function accommodates this through use of the `target` parameter which names the process whose data references are to be tracked, and the `targetFD` which names a file descriptor (handle) via which the process reads the data. Note that while you could inject data directly into the 2nd process's buffer, doing so risks ingest of data that would never get through the first process.

The target application might also execute on a different computer than that into which data is injected. Use the optional `target` parameter can include a prefix (followed by a colon) to identify the computer upon which the target process executes.

5.12.2 Injecting data into the driver

There also may be situations in which you wish to inject data directly into the driver computer. Consider an example target application that consumes multiple UDP packets that each change the system state in a manner that affects processing of subsequent packets. While the `injectIO` function supports injecting multiple UDP packets, doing so causes the reverse execution bookmarks to be reset on each packet injection. Also, the `injectIO` support for multiple UDP packets assumes all packets are received into the same buffer. TCP conversations may also benefit from injecting data into the driver, e.g., to replace one portion of the conversation. Injecting data into the driver is performed in the same manner as injecting data into a target application. The `drive-driver` utility includes a `UDP_JSON` option in the directive file to send UDP packets from a JSON file and a `TCP_JSON` for TCP conversations that cannot be easily injected into the kernel buffer. The driver receives these JSON files from the host and uses them as input sources when sending data to the target. The JSON files are created using the `genJson.py` utility. Injection requires use of a filter that replaces a portion of the JSON file with your crafted data (or data from AFL).

Your ini file must first be modified to make the driver the target. Get the `driver.param` file from the `RESIM_DIR/simics/workspace` directory and refer to that `driver.ini` file for driver component parameters. Add those parameters to your ini file. The snapshot should be whatever snapshot you were using to send data to your target for use with `trackIO`. Do not boot your simulation from scratch with the driver as the target, or it may not stop before the target processes have been created.

Create a json file as large as any you expect to inject using `genJson.py`. Use the drive-driver utility to tee up reading of the JSON on the driver, after first using `debugProc('/usr/bin/python3', new=True)` to catch execution of the client script that will send data to the target. Then use `runToOpen` to catch the opening of the JSON file (provide the full path, e.g., `/tmp/foo.json`), to see its FD. You can then use `prepInject` on that FD to create a snapshot for use with injecting json data into the driver.

The `genJson.py` utility will combine multiple input data files into a json file for use with drive-driver. For TCP connectins, if you provide host and port options to that utility the resulting JSON will include those parameters and will be suitable for direct injection. This allows the same driver snapshot to be used for multiple target processes.

Use an AFL filter when injecting data. The filter should replace a portion of the JSON with your data.

Another potential advantage of injecting into the driver is the ability to generate multiple traces from the same starting cycle. It is also intended for TCP-based protocols that use `ioctl` to determine how much data to read from the socket on each receive. In particular, some applications will call `ioctl` for a zero result prior to completing processing of input. That would reset the origin if data were injected into the target and RESim had to alter the `ioctl` return value. Injecting into the driver avoids the origin reset.

NOTE: You may end up working from two checkpoints, one for injecting JSON data into the driver and another for injecting data into the target application. The former is useful when analyzing watch marks because it is not subject to origin resets. The latter is may be more efficient for TCP connections. Consider deriving the target injection checkpoint from the driver checkpoint to avoid unintended differences, e.g., one having the HANG setting and the other not.

5.12.3 Preparing for data injection

Data injection techniques depend on whether the goal is to inject data into the application buffer, or into a kernel buffer.

The `prepInject` command is used to put the simulation in a state suitable for injecting data into an application read buffer, i.e., the named by the application when making a syscall such as `recv`. Notional data is sent to the target just as you would when using `trackIO`, however instead of using `trackIO`, you use `prepInject`. The data content is not important, however it should be as large as any data that you wish to inject (bounded of course by the application design.)

For network traffic, the general steps for `prepInject` include:

- Run the simulation to a desired state after selecting the target process for debugging, e.g., using `debugProc`. This state may include completion of initialization that may occur while the target process is waiting in the kernel on the target `recv`.
- Create a snapshot.
- Prepare a means of sending sample data to the target port. The purpose of this data is to cause the target to receive it and return. The data itself will be replaced later by injected data. The data should not include multiple UDP packets, you want a single read.
- Restart RESim and issue the `prepInject` command, providing the FD of the receive socket a name for the new snapshot.
- Send the sample data.
- RESim will pause the simulation when the data is read and will create a new snapshot.

Splitting multiple UDP packets for each injection takes one of two approaches. If the application is known to have a fixed UDP header, e.g., `RrUDP`, then we can split AFL data at those strings using the `AFL_UDP_HEADER` environment definition in the ini file. Packet combinations can be concatenated to accommodate whatever packet quantity is desired. The second approach is to inject the UDP packets into the driver computer as described in 5.12.1.

The `prepInjectWatch` operation prepares snapshots for injection of data directly into the kernel buffer instead of into application memory, which is often useful when analyzing TCP conversations. If analysis is to commence with the first input data read, then `trackIO` is used with the `kbuf=True` option. The data sent to the target must be populated with a designated character, currently "Z". And there must be as much data as you expect to read during `injectIO` sessions. The designated characters are used by RESim to identify the end of individual kernel buffers, and thus are critical to determining the size and location of the different kernel buffers. It is also critical that the data be of a form that will be read by the application because RESim locates kernel buffers by backtracing data read into application buffers. However, if you find the first kernel buffer is as large as any data you might send, then a single application read will suffice. If AFL filters (see 6.4) are to be used, e.g., to generate CRCs, then apply the filter before sending the data to the target via the driver. Take care to avoid extraneous characters, e.g., newlines at the end of the test data.

5.12.4 Injecting into a TCP conversation

Consider an application service that accepts a connection and performs one or more receive operations to get the application to the point where you want to analyze it or fuzz it, e.g., you'd like a snapshot of state prior to the third `recv` call. In this example, we assume the application uses `ioctl` systems calls to determine how much data is waiting to be read. Generated a directive file to send multiple files via TCP as part of the same conversation, with the last file containing the designated character (Z). Use `trackIO` without the `kbuf` option, and use `c` to continue until the Zs are read. Identify the watch mark associated with the `ioctl` call that returned the size of the Z's data, and pass that to `prepInjectWatch`. Alternately, see [7.10](#).

5.12.5 Injecting data

Once you've created a snapshot with either `prepInject` or `prepInjectWatch`, the `injectIO` operation takes data from a file and writes it into the target memory. It then runs the simulation forward with data tracking functions from the `trackIO` function, generating watch marks. If you've defined a `AFL_UDP_HEADER` to split up UDP packets, then RESim performs multiple injections, one for each packet. Otherwise, all of the data from the file is written to the target buffer. When injecting data, you must account for buffer sizes. With user space injections, the length value given to a `recv` call can be a guide (but not always). Injecting data past the end of an application buffer is a fine way to observe artificial SEGV's.

The `injectIO` function will apply data filters defined in the ini `AFL_PACKET_FILTER` environment variable. See [6.4](#).

5.13 Code coverage

Code coverage artifacts help the analysis understand how much of the target code has been executed by test inputs, and may help to identify data inputs that will lead to previously unexplored execution paths. RESim generate two different kinds of code coverage artifacts:

- Coverage files resulting from the replay of AFL sessions using the `runPlay` script. These files are stored beneath the AFL output directory and include cpu cycles and UDP packet number for each hit.
- Hits files containing lists of all basic blocks hit in a session or cumulatively.

Analysis and expansion of fuzzing code coverage is described in [6](#). RESim also includes code coverage functions intended for use by the analyst independent of fuzzing. Those are described in this subsection.

The scope of coverage is tied either to the target program binary, or a single shared library. We refer to this as a *coverage unit*. Coverage tracking commences with use of the `mapCoverage` command which causes RESim to record each basic block hit within a coverage unit. The `showCoverage` command displays a summary of the hits and stores them in a json file known as a *hits file*. Alternately, providing the `cover` option to the `trackIO` function causes it to create a hits file.

The hits files are read by IDA and Ghidra, coloring basic blocks to highlight the code that has been executed. If the target is being debugged in RESim, use the `color` option when starting IDA, otherwise run the `colorBlocks.py` script from IDA. In Ghidra, use the `color blocks` menu option. The location and naming of hits files is described in [11](#).

5.13.1 Branches not taken

The `findBNT.py` script analyzes hits files and reports on *branches not taken*, (BNTs), i.e., unused exits from hit basic blocks within a coverage unit. These reports can be helpful when identifying data inputs that would improve code coverage. BNTs are a key part of fuzzing workflows as described in [6](#).

Note that AFL does not report crashes as queue files and thus paths that always lead to a crash will appear as a BNT. TBD highlight these.

5.13.2 Hits in libraries

The instruction addresses in hits files, (and those reported by `findBNT.py`) are based on static program values, i.e., when recording hits in a shared library the runtime address values are reduced by the image base. The RESim IDA plugin provides a `GoTo` function to jump to a program address computed by adding the image base to the given value. This is invoked using Ctrl-Shift-g.

5.14 Generating new input data sets

A review of the watch marks generated by an input file may reveal multiple string comparisons. Rather than manually creating new input file variants, RESim includes a set of utilities to automate generation of new input files based on string comparisons found in watch marks. Results of these scripts are placed in the user's home directory relative to the `newio` subdirectory. The starting point is an `io` file and a snapshot from `prepInject`.

Use of the scripts (found in `RESIM_DIR/simics/simicsScripts/bin`) is described below. **NOTE:** These scripts only currently support data tracking for a target process other than the one that ingests data. TBD expand scripts to include tracking of references by ingesting process.

See the `startNewIO.sh` script which uses `injectIO` to create an initial watchmark json beneath the `trackio` subdirectory from a given input file, and will copy the input file to a `queue` subdirectory. It also uses `playAFL` to generate coverage data for the input file. It then uses `createNewIOFiles.py` to generate a new input file for each string comparison found in the watchmarks. It places these into a new `next_level/file/queue` subdirectory and uses `playAFL` to create corresponding coverage files for each new queue file in `next_level/file/coverage`. Each queue file is then run with `injectIO` and the resulting watch marks are recorded in the `trackio` subdirectory. Another `next_level` directory is created under `file` and then a subdirectory for each queue file, beneath which the directory structures repeat, with `queue`, `coverage` and `trackio` subdirectories.

5.15 Selecting checkpoints

Use of checkpoints can significantly simplify and speed up analysis, e.g., by not having to wait for simulated systems to boot each time. The performance and ease of analysis can depend on the state of the simulated system when a checkpoint is made. Assume you wish to analyze a particular service. After using the `debugProc` command to select it and gather its information, consider running the simulation forward for a while until you see that other system initialization has completed. Creating a snapshot after initialization may speed up subsequent analysis by reducing the amount of extraneous execution.

Watching the log at `logs/monitors/resim.log` can let you know when initialization has largely completed. While you are waiting for initialization to complete, the target process may be waiting in a kernel call, e.g., an `accept` or a `read`. RESim retains that information in its checkpoint. For example, when running from such a checkpoint, a `trackIO` function on the bound FD will catch the return from `accept` and alter the tracked FD to the new FD returned by the kernel.

6 Fuzzing with AFL

RESim is integrated with a forked version of AFL from `github/mfthomps/AFL`. The AFL fuzzer was modified to provide its fuzzed input to RESim via a network socket. RESim uses that socket to send AFL results of the fuzzing session in terms of basic block edges hit. The `-R` switch tells AFL that this is a RESim session.

RESim either injects data received from AFL directly into application memory, or it injects the data into a kernel read buffer, depending on the setup steps used to prepare for data injection as described in 5.12.3.

Parallel fuzzing is supported as described in section 6.14. But you must first prepare and test a single fuzzing instance as described below. If you are limited to running a single instance of Simics, consider using the `--dirty` option to the `runAFL` command.

The following steps describe fuzzing a network service.

- Clone the AFL repo (<https://github.com/mfthomps/AFL.git>)⁶ and use `make` to build the executable.
- Create a RESim workspace for the fuzzing session. The name of the workspace will be used to name fuzzing artifacts, so make it meaningful.
- Confirm your `bashrc` file defines the AFL paths per section 5.1.
- Create a `target` directory beneath the AFL output directory having the name of your workspace.
- Create a `seeds` subdirectory beneath that.
- Use tracing or static analysis to identify the FD the service binds to and whether packet processing requires a minimum sized packet.
- Use the `prepInject` or the `prepInjectWatch` function as described in 5.12.3.
- Set values in the `ENV` section of your ini file per section 4.1. These may include a UDP header, and the backstop cycles that determine how long an AFL session will continue to run after hitting the most recent basic block.

⁶Older environments may need to update binutils.

- From a shell, use the `runAFL` command, naming the target (e.g., your workspace name).
- You should see AFL run some initial sessions for calibration and then begin its fuzzing.
- Output from the Simics command line is redirected to `/tmp/resim.log`, check that for errors if AFL hangs. Also look at `resim_db.log` in the workspace.
- Review code coverage as described in 6.3, adjust seeds accordingly and repeat.

A set of RESim ini environment variables affect the fuzzing sessions. These include:

- Each fuzzing session will run until some number of cycles have elapsed since the most recent basic block hit. This value is defined in the `AFL_BACK_STOP_CYCLES` environment variable within the ini file (in the ENV section). A session will also end if there is not more data to inject into application memory upon hitting the read call address.
- The `HANG_CYCLES` value determines when a session is considered to be hung, e.g., the program is in an infinite loop. Hangs may reflect more than just DoS vulnerabilities, e.g., corruption of a return address might lead to a viable code location that leads to a loop.
- The `BACK_STOP_DELAY` value will delay setting backstop or hang detection until the given count of basic block hits has been reached. The intended use is applications that do not commence parsing inputs until after performing some function. This value can be used instead of setting a very large backstop, which can cause AFL to run very slowly.
- The `AFL_STOP_ON_CLOSE` env value will end the fuzzing session if the FD is closed, and `AFL_STOP_ON_READ` will end it if a read system call is encountered on the target FD after all fuzzed data is consumed.
- Variables that have an effect on tracking data injection as described in 5.12.

6.1 Seeds

Well chosen seeds can save a lot of fuzzing time. For example, if all packets must start with a specific string there is no sense in making AFL discover that string. On the other hand, AFL can quickly discover many protocol values that affect execution paths from simple data. This is particularly true when the data sizes are small.

There is no one right way to select seeds. Sometimes a sample of PCAPs provide suitable seeds. However there are other cases where AFL would be more efficient with a smaller degenerate data set, e.g., a few key fields and a small set of a constant character. An example of such a case might be a protocol in which delimited fields are relatively long within the PCAP, but could be very much shorter in the protocol. In such a case, AFL might struggle to determine the field delimiters (not that it explicitly does any such thing) from the longer PCAP fields.

It is often the case that smaller seeds yield quicker results. There are though plenty of exceptions to that. An understanding of the protocol can be very helpful when selecting seeds.

6.2 Injecting fuzzed data

RESim receives fuzzed data from AFL and injects that data just as it does with the `injectIO` function. See 5.12.5

For single UDP packets, we simply truncate the AFL data to conform to expected UDP packet size (trusting that AFL will not focus future resources on fuzzing truncated data.) For multiple packets, RESim catches subsequent calls to `recv`, injects the new data, adjusts the size in the return register and skips execution forward to the return from the system call. Handling multiple packets currently requires that we assume the `recv` calls are made from a common user space address, and the returns from `recv` all end up at the same user space address.

Splitting multiple UDP packets for each coverage session currently takes one of two approaches. If the application is known to have a fixed UDP header, e.g., `RrUDP`, then we can split AFL data at those strings using the `AFL_UDP_HEADER` environment definition in the ini file. Seed packet combinations can be concatenated to accommodate whatever packet quantity is desired. RESim will reject any packet that lacks the prescribed header for subsequent packets, which should cause AFL to see reduced coverage for such data sets. The second approach is to inject the UDP packets into the driver computer as described in 5.12.

One potential challenge is where the application expects a minimum packet size. That complicates creation of minimal seeds. If the minimum size is 1400 bytes, a 2 packet sequence would require a seed of 2800 bytes, which is a lot for AFL to fuzz.

6.3 Fuzzing code coverage

After a fuzzing session, the `runPlay` shell script will create coverage files for each “unique” code execution path found by AFL. The `runPlay` script uses the RESim `playAFL` command and will execute parallel session if fuzzing was so configured. Coverage files generated by RESim when replaying AFL sessions tend to have a lot of redundancy because of the way that AFL determines uniqueness. See 11 for information on where RESim stores fuzzing and code coverage artifacts.

The `dedupCoverage.py` script creates a list of coverage files that are, in some sense, unique. First, it removes duplicate coverage files resulting from parallel fuzzing. Next it removes any file whose hits are a subset of another coverage file. When UDP headers are used, the script first assesses all files from sessions having a single UDP packet. It then considers multi-packet sessions that result in new hits. And where these match or are subsets, it removes files giving preference to those whose hits occur on the earliest session.

After coverage files have been de-duped, use the `runTrack` shell script to automate use of `injectIO` on each of the unique sessions. This optional step generates watch mark artifacts for each session, and these watch marks are then used by RESim to highlight BNTs from blocks the refer to know input data. If `runTrack` is not run, you can still find BNTs, but RESim will not annotate them with watch mark information.

6.3.1 BNTs from fuzzing

The `findBNT.py` utility displays BNTs resulting from fuzzing sessions. It assumes you’ve played the AFL sessions, e.g, via the `runPlay` utility or `playAFL` at the Simics command prompt. It also assumes you’ve run `dedupeCoverage.py`. The resulting list of BNTs can be explored using the `injectToBB` RESim command. This will find a fuzzing input that reaches the given BB, play it and break at that BB. If the `runTrack` command was run to generate watch mark artifacts, then use the `-d` option to the `findBNT.py` command to view watch marks that occur in basic blocks leading to BNTs. You can then use the `injectToWM` RESim command to find an corresponding session and run to that watch mark. This function can be useful when there are a large number of BNTs and you want to prioritize those with a higher likelihood of being affected by inputs. Use the optional `ws` option to name the AFL workspace if it differs from your current Simics workspace name.

An example workflow for this might be:

- After fuzzing, use `runPlay` to replay and record the BB’s from all the paths found by AFL. And run `dedupeCover.py` to generate a list of unique coverage files.
- Use the `findBNT.py` utility to list all of the BNTs and the BBs which lead to them.
- Run the RESim `injectToBB` command to take the simulation to the BB leading to the selected BNT.
- Start IDA and look at the branching conditions to identify data that would cause the program to take the BNT and guess if that appears to be a function of inputs.
- Use the IDA Client `reverse track data` function to find the source of the data. Note that if the input data includes multiple packets, RESim may still identify earlier packets as the source by tracing the data back through watch marks.
- Assuming you have an alternate input value to test, and an offset within the input file as reported by RESim, use `hexedit` to modify the input file copy created by `injectToBB` at `/tmp/bb.io`.
- Use `injectIO` to confirm that the modified input file reaches the target BNT. For example: `debugSnap; doBreak; injectIO`.
- Run the `addInput.py` utility to add your modified `/tmp/bb.io` to the set of input files discovered by AFL, which will then be included in the next `cycleSeeds.py` command..
- Run `addLive.py` to add a modified file to a running AFL system.
- After the BNTs have been explored, use `runPlay` and `dedupeCoverage.py` to update the coverage records.
- Use the `cycleSeeds.py` utility to add all of the input files corresponding to unique coverage to the target seed directory. This will include your manual additions.
- Delete the `afl/output/< target>` files and restart AFL to see if it can use the new paths you discovered to discover more paths.
- Use the `arch-tars.sh` utility (edit to name your archive path) to archive the AFL results and the workspace.

6.4 Data filters

Sometimes you may wish to focus the fuzzing on a specific code path determined by data within the packet. This may be of particular interest in multi-packet UDP cases where you know a specific command will lead to interesting paths when the 2nd packet is consumed. Left alone however, AFL will find interest in code paths generated by the first packet having other commands. Currently, you can define a `AFL_PACKET_FILTER` as a python program to reject AFL data that does not contain desired content, e.g., a specific byte at a specific offset. The program must have a `filter(data, packet_number)` method that returns data to be injected. For example, return nulls if the packet is to be rejected. The intention is that will yield poor coverage, causing AFL not see new paths. Alternately, return an altered packet, e.g., with a computed CRC injected into the data. An example AFL data filter can be found in

```
simics/workspace/sample.filter
```

When creating filter modules, you may need to modify the python module search path to include the python distribution, e.g.,

```
sys.path.insert(0, '/usr/lib/python3/dist-packages')
```

6.5 Address jumpers

Execution paths can be dynamically altered, e.g., to avoid CRC computations using the `addJumper` command. This takes as inputs the basic block addresses of a from-block and a to-block. When code coverage hits the from-block, the IP is modified to jump to the to-block. The jumpers are managed in a file with a `.jumpers` suffix along with the IDA databases and block lists for applications.

Jumpers are also useful when you discover very slow fuzzing sessions caused by large quantities of iterations driven by an input value. A `Data filter` as described above might mitigate such a problem, however that can artificially constrain data whose value may have effects beyond the iteration count. Review of the iterated code should identify whether use of a jumper might affect program execution.

These jumpers are only intended for use with code coverage and/or fuzzing are different than those used with the `jumper` command described in 3.8, which provides more general alterations to execution control flow.

6.6 Fuzz another library

Use the optional `fname` parameter to name a shared library file (relative to the `RESIM_ROOT_PREFIX`). You must have first opened that file with IDA and used the `findBlocks.py` script to save a database of its basic blocks.

6.7 Fuzz another process

Consider the case where one process receives data and sends a derivative of that data to another process via a pipe or internal network, and you want the fuzzing guidance (i.e., code coverage feedback) to be driven by the 2nd process. You would set up for fuzzing using `prepInject` as if the 1st process were the target. Then provide the `afl` command with the optional `target` and `targetFD` parameters to name the 2nd process and FD from which it reads the data.

6.8 Fuzz using the driver

As noted in 5.12.1, you may wish to inject json data into the driver, and observe its effect on the some target process. This can be extended to fuzzing as follows:

- Create an `AFL_PACKET_FILTER` that inserts fuzzed data into a selected entry in the json file. See the example at `simics/workspace/json.filter.py`
- Create an AFL seed file, e.g., the original data in the json entry you wish to fuzz.
- Use the `playAFL` or `afl` command with a `test_file` to create a `commence_params` file, providing a count to reach the data entry you selected in the json filter. This command would include a target naming your target component and process (full path), and the targetFD.
- Provide that file name as the `--COMMENCE_PARAMS` option to the `runAFL` command.

Note this approach may be slow because each AFL cycle requires data to be sent from the driver to the target; and it requires RESim to run the target forward prior to commencing code coverage.

6.9 Thread isolation and skipping breakpoints

Sometimes some other thread is in a poorly constructed receive loop, generating lots of breakpoints and slowing down execution. Unless otherwise directed, the afl function causes the coverage breakpoints to be on physical addresses, which are shared amongst threads of a process – or amongst all processes sharing a library that is being tracked. Given the costs of tracking context switches, breaking on physical addresses is often the best alternative. You can record the basic blocks hit by any other threads, and subsequently keep those blocks from generating breakpoints using the `dead=True` parameter (the `-d` flag on `runAFL`). This causes the coverage function to generate list of basic blocks to ignore. See the `IGNORE_BB_LIST` ini parameter for naming. This may take several minutes to complete. When it has found no new blocks hit by other threads for 2 minutes, it stores the dead zones and exits. When starting a new afl session, if such a file exists, it will be used to ignore selected basic blocks when establishing code coverage. The `-d` option should be run before multiple AFL clone copies are created using `clonewd.sh`. An alternate method of creating these lists to use the `backstop` and `dead_zone` options to `mapCoverage` command. You would do this from a snapshot without injecting or providing any outside input to the process of interest. You may want to increase the `BACK_STOP_CYCLES` value when recording these background bb hits to ensure you cover the period of time that may elapse once you’ve provided input to the process.

One symptom of problems with thread isolation is that AFL reports poor stability.

If performance appears to drop way off after a while, consider running with the dead switch again, this time using found paths instead of your seed. Of course this situation could suggest discovery of input data that causes other threads to consume fuzzed data, i.e., yet another shiny object. It would be nice if RESim could provide notice that some other thread is consuming breakpoints, however the act of reading modeled state during a breakpoint slows down the execution cycles by multiples. One clue may be the stability displayed by AFL. A low value can suggest interference by other threads.

TBD: The `runTrack` artifacts reflect if multiple threads are referencing data. Use that to exempt selected threads from generation of dead zones.

6.10 Crash analysis

If AFL finds crashes, use the `crashReport` command to generate reports on each crash. These reports include stack traces, reverse data tracking and indications of ROP or SEGV occurrences. A single command will generate reports on all crashes under a `afl-output` target directory. If the fuzzing inputs potentially include multiple packets, use the `crashReport` utility (not from the Simics prompt) and provide the target FD in the `-f` option. This will use a driver that is assumed to be listening on local port 4022. The ini file must identify the target IP address and port number in the `TARGET_IP` and `TARGET_PORT` ENV variables. The init file should not name the snapshot created with `prepInject`, it should name a snapshot reflecting state prior to receipt of test data because the driver will send the data rather than having data injected directly into memory.

The original fuzzed data for most recent crash in any Simics instance is written to a file in the workspace called `icrashed`. This data can be injected using `injectIO` (ensure the original filter is applied within the ini file.)

Note: Crash analysis and fuzzing have different criteria back-stop cycles. Crash analysis uses the `BACK_STOP_CYCLES` value. It may be necessary to increase this value to allow time for the crash analysis to encounter the crash.

6.11 Blacklist crashing instructions

The crash report funtions look for a file named `ignore_crash_ip.txt` in the workspace, and will skip processing of crashes that occur at instruction addresses listed therein. This can improve the speed of crash analysis.

6.11.1 False crash reports

The primary means of detecting SEGV relies on looking for unresolved page faults when an AFL cycle ends, e.g., due to a backstop. False crash reports may be the result of a backstop that is too small, i.e., a large backstop will let the kernel complete the mapping of the page.

6.12 Update code coverage

The new code paths discovered by AFL can be added to your aggregate code coverage file using the `playAFL` command (or the `runPlay` utility. This is intended to aid manual analysis to see if the new path leads to other interesting paths that can then be fed back to AFL. See 6.3 for information on fuzzing code coverage and analysis of branches not taken.

6.13 Fuzzing performance

The following notes may aid in improving fuzzing performance.

- The number of breakpoints hit for coverage has a large effect on performance. Avoid references to the target model from within the coverage HAP – a call to `getTID` reduces execution iterations from 20/sec to 5/sec. And see Thread isolation above.
- The default is for the `afl` command to set breakpoints on physical addresses. Use `linear=True` to force linear addresses, which will result in use of the `contextManager` to maintain execution context. Depending on the application, this may have different performance properties than use of physical addresses.
- One thing that may affect performance when using physical addresses is paging. If a significant amount of application code is not paged in at the start, RESim must dynamically break on page table structures and dynamically add breakpoints and Haps in order to track the basic blocks. Depending on the application and your goals, consider running the program to a steady state prior to creating the snapshot for fuzzing.
- Use the `-n` option to the `resim` command to suppress the GUI and windows, this can be the different between 2 sessions per second and 10.
- The `AFL_BACK_STOP_CYCLES` value might cause AFL sessions to run longer than needed. On the other hand, too small a value may cause some execution paths to be missed.
- When using the `UDP_HEADER` to fuzz with multiple packets, AFL may run amuck, generating lots of packets because it observes what it considers to be unique paths or unique crashes. Consider putting a cap on the number of packets with the `AFL_MAX_PACKETS` ENV value.
- Use parallel fuzzing as described in 6.14.
- Basic blocks hit while during processing of a selected function can be masked to improve performance by disabling breakpoints. Use the `SUSPEND_COVERAGE` ENV value in the ini file to name a file containing the address of a function (TBD expand to a list) start and end. RESim will suspend tracking coverage when the function start is hit and will not resume until the function end is hit.
- AFL sessions can be stopped when a selected function is hit by using the `FUNCTION_BACKSTOP` env setting in the ini file.

6.14 Parallel fuzzing

AFL includes support for parallel fuzzing in which multiple instances of AFL all fuzz the same binary. RESim builds on AFL's parallel fuzzing, and supports deployments having multiple computers, each running several instances of AFL/RESim pairs.

Assuming you have (and are currently in) a RESim workspace directory that contains an ini file and checkpoint, use the `clonewd.sh` command to create some number of copies of the workspace. For example:

```
clonewd.sh 6
```

will create six numbered subdirectories with a prefix of `resim_`. Each of those subdirectories is a RESim workspace having its own `logs/` subdirectory, all other files are simply links.

Then use the `runAFL` command, naming the ini file to be used in the fuzzing sessions. The system will spawn multiple pairs of AFL and RESim, and create an AFL xterm for each to display the AFL status screen. Simics is started without the gui or windows. Pressing **Enter** will terminate all sessions, (the AFL Xterminals will linger for ten seconds.)

AFL results are stored relative to your `AFL_DATA` environment variable (defined in the `.bashrc`) in subdirectories named by the workspace, the hostname and the instances, e.g.,

```
$AFL_OUTPUT/myworkspace/rb9_resim_1
```

The stdout and stderr from each of the Simics sessions is currently dumped to a file at `/tmp/resim.log`. Logs from the `runAFL` command are found in `/tmp/runAFL.log`

Use the `runPlay` command to replay the inputs found by AFL in parallel.

6.14.1 Fuzzing with multiple computers

Multi-computer fuzzing is achieved using scripts that copy your local RESim workspace to a set of *drone* computers, and execute `runAFL` on each drone. This is initiated and managed from the **master** RESim instance, i.e., where your workspace is located. Results from each drone are periodically retrieved and distributed to each other drone.

Create a `drones.txt` file within the workspace directory, containing a list of drone computer host names. Each must be reachable via SSH without passwords, e.g., by use of an SSH Agent. Each drone must be provisioned with Simics and have an account with the user ID used on the master, which must be the host containing the RESim workspace.

- Use the `sync-resim.sh` command to push the local copies of RESim and AFL to the drones and ensure their license managers are running.
- Push copies of your workspace to the drones using the `sync-drones.sh` script.
- Start the multi-computer fuzzing session with `start-drones.py`, passing in the ini file name.
- Pressing **Enter** will stop all fuzzing sessions.
- Get status using `status-drones.sh`
- Kill all Simics and AFL processes with `sync-kill.sh` (usually not needed, but stuff happens.)
- Remove all output data and seeds (for this workspace) from drones with `clean-drones.sh`

The `runAFL` command monitors free memory on each computer and will restart RESim if memory gets too low due to memory leaks in Simics. The AFL sessions persist during these restarts so that they do not lose context. **NOTE:** restarts of RESim inherit the current python scripts. Those should not be edited on hosts having long-running fuzzing sessions.

6.15 Fuzzing in background

The `runAFL` command creates an xterminal for each session when run on a single server. Use `nohup` and the `--background` option to run in the background, e.g., if your X11 server is a local VM that may be interrupted.

```
nohup runAFL some.ini -b &
```

Then use the `fuzzhappening.py` script to observe the status of the fuzzing sessions.

6.16 Fuzzing application notes

6.16.1 Why fuzz with full system simulation?

Fuzzing a binary often requires the binary to be led to a state in which it is prepared to consume fuzzed data. A general fuzzer such as AFL makes no provision for brining the binary to such a state. AFL is designed to exec the target program at the start of each session. Getting a binary into the desired starting state so that it can be fuzzed is sometimes accomplished using a custom test harness designed for that binary.

Creating a test harness to fuzz a non-trivial binary can be error prone and time consuming. Instead of constructing a harness, a full system simulator allows you to commence interaction with a target at the locus of execution of your choosing.

Applications that require specific interaction with external entities can be satisfied and checkpointed such that fuzzing sessions start precisely when/where the application is about to consume fuzzed data. Consider a thread that processes selected data provided by another thread that receives network data and writes some version of the data to a pipe.

Fuzzed data is injected directly into application or kernel memory, bypassing data reception by the kernel.

The fuzzer, e.g., AFL, relies on feedback from instrumentation on the target to tell it which code paths were hit while consuming a specific input. So long as that feedback indicates new code paths, the fuzzer will pursue those paths in attempts to find more branches on those paths. Sometimes however, we would like to avoid some paths, e.g., to focus on some areas of interest. If the inputs leading to different paths are well enough understood, applying filters to fuzzed data can deter the fuzzer from following unwanted paths. However, sometimes the inputs leading to unwanted paths is not known. While theorem satisfaction tools might help identify such inputs, it may be simpler to simply break on them and feedback a poor coverage report.

6.16.2 TCP timing and packetizing

Sometimes programmers make assumptions about TCP reads, e.g., they assume a relationship between the quantity of bytes written in a single write to the quantity of bytes read in a single read. Thus, changes in things like MTU can affect the application protocol...

6.16.3 State of the target system

Stateful services such as FTP typically require some amount of interaction to get the process into the state at which you'd like to fuzz it. In the example of FTP, this may include logging in, e.g., as *anonymous*, and issuing the PORT command. Keeping with the FTP example, you would create a driver component and ssh to that as part of the setup. If you wish to fuzz commands that use PORT, e.g, LIST, you would login to the ftp server and then stop the simulation. Knowing that a command such as LIST will first send the port command, you would use `prepInject` with `count=2` to pause the simulation after it has first read the PORT directive and then the LIST directive. The fuzzing injection would then be overwriting the LIST directive.

Using AFL to fuzz a sequence of FTP commands is not currently possible.

6.16.4 False paths

Use the `dedupCoverage.py` utility to identify unique coverage sets. If you find massive reductions, e.g., AFL finds 200 paths and deduping yields four, it is worth going back and looking if AFL is observing extraneous processing, either in another thread or crude read loops. TBD if the latter, add switch to force ending at next read? However that precludes byte-at-a-time receives.

When fuzzing UPD with a defined `AFL_UDP_HEADER`, the quantity execution paths that AFL thinks it is seeing can explode, e.g., if AFL gets 2 UDP packets for one session, and each result in hit sets already covered by different other sessions, AFL will treat that as a new execution path. The quantity of UPD headers processed by RESim can be bounded using the `MAX_AFL_PACKETS` environment variable in the ini file.

6.16.5 CADET01

The CADET01 example is fuzzed by the `simics/testing/cadet-test`. This simple CGC service lives up to a reputation it achieved for illustrating subtle issues related to TCP. The service reads a character at a time, until it sees a newline at which point it processes the buffered data. Each fuzzing iteration will catch each read system call and their returns to determine when the injected data has been consumed, and it will set the `read` syscall return value to zero following the consumption of the buffer. In cases having the newline, this will happen after all buffer processing and after the subsequent read for the next round of the service. From a fuzzing perspective, this is wasteful and slow because we could have stopped fuzzing upon the next call to read instead of waiting for that zero value to be set and processed. One alternative would be to set `STOP_ON_READ` and set a filter to force the newline in each input.

7 Example workflows

This section describes steps an analyst might take when using RESim to reverse engineer and analyze target systems.

7.1 Analysis of a specific service behavior

This example describes the use of RESim to understand why a service is generating an unexpected response to an input. Assume the target has a web server that you believe should respond to a specific URL, e.g., by proxying the request to another service, and yet you only see a 404 error. A first step is to use `traceAll` to generate a trace of all system calls made by the service when it receives the selected input. Things you might look for in the trace output include attempts to open files; attempts to connect to sockets; and diagnostic messages provided by the programmer. In many cases, programmers provide debugging messages that can aid understanding of the program behavior. You may see messages that are written to `/dev/null`, or other some other FD. Using the `traceFD` function will redirect all of those messages to a file for your review.

A next step is to use `trackIO` and/or `injectIO` to generate watch marks denoting execution points at which the target application references ingested data. This step typically uses the `drive-driver` utility to send test data via the driver component. This is necessary to permit reverse execution, which is required by `trackIO` and `injectIO`. Bringing the target up in the IDA or Ghidra debugger lets you see the execution context at the various watch marks and interactively debug the program. Things you might look for include calls to logging subsystems that do not appear to generate any system calls, i.e., do not generate output and thus did not appear in the system trace. This may be due to configuration values within logging software, e.g., setting a

“debug level”. Look for `sprintf`-type functions and calls to generate timestamps. You may be able to identify execution paths that you can alter with RESim jumpers or via the use of `regSet`.

Other things to look for while using `trackIO` or `injectIO` include references to that portion of the URL that you think should lead to the function that is not occurring. You may find `strcmp`-type functions or configuration data that inhibit the expected paths. You can then consider running AFL on the target, using HTTP that names your URL as the only seed. That might expose execution paths that you were unable to foresee using the debugger.

7.2 Watch consumption of a UDP packet

- Open a pcap with Wireshark. Select the data of the packet, right click and export the selected packet bytes into your simics workspace.
- Use the `driver-script.sh` to configure the driver computer network to share a subnet with the target computer such that the driver can reach the target. And use the `mapdriver.sh` to make sure you can ssh to the driver from your host.
- Create a *directive* file within your workspace for use with `drive-driver`. Create a 2nd terminal with a shell in your workspace and tee up the `drive-driver` command.
- Debug the desired process e.g., `debugProc('some-program')`, or `debugSnap()` if the snapshot had been made of the process subsequent to a `debugProc`. Be sure the program state is beyond the `bind` system call and you know the FD upon which the `bind` occurred.
- Use `trackIO(FD)` to track input on the FD returned by the `bind` call.
- Run the `drive-driver` command from the other terminal and wait for RESim to report the data watch has completed.
- Start IDA with the desired program (e.g, use `runIda program`. Use shift-r to attach to the debugger and load the RESim plugin. Then go to the “data watch” window and refresh. That will create a list of watch marks, i.e., program references to the content of the UDP packet.
- Double click on watch marks to jump to that execution state.

7.3 Reverse engineer a service

This example assumes you want to understand how a program consumes data on a specific TCP port.

- Run RESim on a configuration having the target and a driver.
- Use `traceAll` to generate a system call trace and the `saveTraces` to generate trace artifacts.
- Use the `traceReport.sh` script to generate trace reports, and look at the resulting network information noting the program that binds to a port of interest.
- Modify the workspace script-driver.sh script to set driver ethernet IP addresses to values to enable communication with the target.
- Restart RESim and `debugProc` the target program.
- Use `writeConfig` to save a snapshot so that you can return to this point. If you exit RESim, modify the ini file to reflect this snapshot and subsequently use `debugSnap` to return to debugging.
- Use `runToBind` and observe the FD (either in the log or using `reMessage`)
- Use `runToAccept`, which will not return until the client connects to the service.
- Use `drive-driver` to cause a data file to be sent to the target, which should then cause RESim to stop on return from the `ACCEPT` call. Observe the resulting FD.
- Use `trackIO` to note where input is processed. The stack may reflect a call to `libc` from some other library. Note you may be in a thread that started in that library. Use `showSOMap` to determine the address at which the library is loaded.
- Open the program or the library file in IDA. If analysis artifacts have not yet been created for the program or library, use `idaDump.sh` to do that and then restart RESim, use `debugSnap` and redo starting at `runToAccept`.

- If a library was loaded, use `edit/segments/rebase` to rebase the program from the load address observed via `showSOMap`.
- Attach IDA's debugger to the service and work through the `dataWatch` list of Watch Marks.
- Modify data to alter execution paths using either `modifyMemory` followed by `retrack`, or the `injectIO` function (the latter is only available as a `RESim` command.)
- Use `prepInject` or `prepInjectWatch` to create a snapshot for use with `injectIO`.
- Use `injectIO` to iterate through many permutations of input data with the intent of finding new code paths.
- Use your newly identified input data files as seeds for AFL and fuzz the service.
- Replay the AFL sessions using the `runPlay` script. This will create coverage artifacts.
- Reduce the number of AFL results using the `dedupeCover.py` script.
- Replay the unique session using the `runTrack` script to create watch mark artifacts reflecting basic blocks in which input data was referenced.
- Use `findBNT.py` to find branches not taken (BNT).
- Explore BNTs using `injectToBB` or `injectToWM`, find new branches and feed the resulting data files back to AFL.

7.4 Find source of log messages

Reviewing results of `traceAll`, you find an interesting message output from `Write` or `WriteFile` that looks like a log entry. You believe the thread that did the write is simply a logging thread, and thus it is not the source of the message. Use `runToWrite` to get to the execution point of the write while in debugging. Then use `revTaintAddr` to reverse to the source of the output buffer. Provide the address of log content that is interesting, e.g., if you supply the address of a timestamp or other boilerplate, it may simply reverse to part of the logging thread.

View the resulting bookmarks (`listBookmarks()`) and go to the first bookmark that reflects a data move (`goToDebugBookmark(num)`). Note that the TID of the current thread should be different than the TID of the thread that generated the log message. Use `stackTrace` to find the DLL that generated the content of the log message, e.g., via an `sprintf` type of function. Open that DLL in IDA or Ghidra and explore.

7.5 Observe changes in outputs

Use `traceAll` followed by `traceFD` to observe program output in response to varying inputs injected via `injectIO` with `stay=True`. TBD, combine commands or make modal? – for now, must repeat: `injectIO`; `traceAll`; `traceFD`. This can be more efficient and less complicated than trying to alter inputs of a client and then observing responses from the server.

7.6 Track buffer accesses

You've found a buffer populated with data, and you'd like to track access to the buffer just like you `trackIO`. Use the IDA "add data watch" to add the buffer, and then use `retrack`.

7.7 Find code divergence

Imagine you've found two similar data inputs, e.g., as the result of fuzzing, and one crashes and the other does not. You'd like to know at what point the program execution diverges.

- Use `injectIO` to run each of the sessions, providing the `instruct_trace=True` flag. Provide the TID if tracing is to be limited to a single thread.
- Use the `traceDiff.py` utility with the `-d` option to find where the two traces diverge.
- Rerun one of the `injectIO` sessions, but this time precede that command with the `doBreak` command. Note `traceDiff` tells you if the divergence happens on the `nth` execution of the identified instruction. Provide that value to `doBreak` so that it breaks just before the divergence.
- Use IDA to observe the condition of the branch.

7.8 Branches not taken

See [S.1](#) and [6.3.1](#) for workflows related to finding branches not taken.

7.9 Packaged example with public images

See `RESim/simics/examples/network_file_system` for an example use of RESim, including IO tracking and fuzzing. The `README.md` in that directory provides instructions for running the example, and for obtaining the disk images and binary files used within the example.

7.10 Fuzzing a web service

These steps describe how to stop a web service in the middle of a conversation to fuzz a specific command.

- Create a snapshot in which the web server is running and initialization has completed. Be sure to use tracking to record the process shared libraries.
- Use a derivative of the `simics/workspace/webproxy.sh` to script let a local browser reach a target web server via the driver.
- Record a web conversation using `traceAll` and `traceFD` with `web=True`. Use a fresh browser to ensure stale cookies are not passed to the server (these would break the cookie replacement functions.)
- Use the resulting file as input to `genJson.py` with the `-w` and `-c` options to create a json of the browser side of the conversation. This will replace cookies with the string `RESIM_COOKIE`.
- Reference the `traceFD` output (or a trace) and find a string unique to the data read immediately prior to the data you wish to replace with an inject. For example, the timestamp preceding the HTTP in the POST tokens.
- Start RESim and use `debugProc` or `debugSnap` to debug the web server.
- Use `drive-driver` to send the json content to the web server.
- Use `noReverse` to speed things up and then `runToRead` to find the read of the unique string you selected.
- Enable reverse execution with `allowReverse`.
- Step forward a few steps to be sure you are out of the kernel.
- Use `prepInject` to create a snapshot of the next read.
- Note `prepInject` displays the name of a file that now contains a copy of the data that was read and would be replaced by injected data.
- Use `injectIO` with that file to confirm it is processing the data as expected.
- Use that file as a seed for AFL

8 Example targets

8.1 CADET01 Example

Using the `resim-ws.sh -e` command will create a workspace that includes files used in the CADET01 example. See the `README-cadet01.txt` files in `simics/workspace`. The example requires two disk images located relative to your `$RESIM_IMAGES` directory. The target image from:

```
https://nps01-my.sharepoint.com/:u:/g/personal/mfthomps_nps_edu/IQC-hSPskpwKRqvVSauYb_14AU6FLcnA4awK
```

should be copied to:

```
$RESIM_IMAGES/cadet01/cadet.disk0.hd_image.craff
```

And the driver image at

```
https://nps01-my.sharepoint.com/:u:/g/personal/mfthomps_nps_edu/IQCV_MiNKTuQULQgny7z3A0RAc51gxYUK3Zw
```

should be copied to:

```
$RESIM_IMAGES/driver/driver_22.disk0.hd_image.craff
```


8.2 Network File Service

Another example from the Cyber Grand Challenge can be found at:

```
$RESIM_DIR/simics/examples/network_file_system
```

This example requires the target image and driver whose links are above under the CADET01 example. Follow its README.

8.3 Cyber Grand Challenge Services

Vulnerable services from the DARPA Cyber Grand Challenge are available as described in the README in `simics/examples/network_file_system` in your repo. The images described therein are located at: <https://nps.box.com/s/ffuz7fgyn770xcgrdur0uf1bo1tur2gk>

8.4 ARM Cortex A9 Sample

A sample ini file and parameter file for the ARM QSP platform is in `simics/examples/armA9`

8.5 ARM5 Sample

A sample ini file and parameter file for the ARM5 Integrator platform is at:

```
simics/examples/integrator
```

8.6 ARMv8-A FVP Foundation

```
simics/examples/armv8a
```

Information on building the kernel image and building a ram disk of a root file system are at

```
simics/image_build/arm_fvp_jun26
```

8.7 Custom devices

Sample custom devices can be found in `simics/modules/sample-i2c-device`. Add `export SIGN_ON_BUILD` to the `config.mk` before making (new in March 24 release of Simics).

9 Implementation strategy

This section discusses our approach to implementing RESim, and some implications for the analyst. RESim primarily gathers information about a system through monitoring of events, i.e., observed via callbacks (*HAPs* in Simics parlance), tied to breakpoints. Two key features of RESim enable its flexibility and performance.

1. Other than basic task record structures, the implementation has very little knowledge of kernel internals. This is a key design goal.
2. RESim only monitors events when directed to do so.

Implications of these design properties can be seen by considering example sessions. Assume you boot a system in RESim and let it run a bit without directing any analysis. The `tasks` directive will list currently running tasks. However, RESim would have no knowledge of full program names and arguments provided to `execve`. In this example, directing RESim to debug a currently running TID results in a debug session with limited stack traces because it would not have information from the ELF header⁷ or shared object map information. It would not have information about open files. That information would be collected if the `traceAll` directive were used, or, for a single program, the `debugProc` directive were used prior to the process start.

RESim maintains information it has gathered, and does so across debug sessions and across checkpoints written via `writeConfig`. For example, if you use `debugProc` to isolate a program, and then stop debugging that program and then return to it, the shared object information is maintained.

Other than IDA analysis, we do not maintain state across different sessions other than state used by the `writeConfig` and `RUN_FROM_SNAP` snapshot directives. For example, the shared object maps within two different

⁷Unless used with the IDA client

sessions may vary due to aslr. Simics includes some support for recording and replaying identical sessions. RESim does utilize those.

As can be seen in the implementation scripts (starting with `simics/monitorCore/genMonitor.py`, RESim relies extensively on breakpoints and callbacks (HAPs). Those are the basic building blocks on which most of the functions are implemented. Most of the HAPs are managed by the `genContextManager.py` module. That module also keeps track of processes selected by the user for observation, e.g., via `debugProc`. It is where process death is detected. That module manages two Simics contexts, the default context associated with the cell of the processor, and a distinct resim context associated with processes selected by the user. These contexts are how RESim can ignore events occurring in other processes. For example, while debugging a process, you may wish to run forward until that process reads from a given FD, and you don't want to be bothered by other processes reading from the same number.

Analysis during reverse execution occurs primarily within the `reverseToCall` and the `findKernelWrite` modules. The implementations appear (and are) convoluted as a result of how Simics implements reverse execution (and due to limitations of the implementer!). The fundamental issue is that while running backwards, HAPs may be invoked in non-deterministic ways, and thus they must be removed before reversing. The `Core.Simulation.Stopped` HAPs are used to detect when a reverse has hit a breakpoint (or the beginning of the recording). Those HAPs then must determine if the desired event was reached going backwards, and if not, continue backwards.

Details of the implementation and the use of HAPs can be found in this paper: <https://www.sciencedirect.com/science/article/pii/S1742287618301920>

10 Troubleshooting

RESim is an ongoing development and has a limited regression testing system. So there will be bugs. Check the logs, in `workspace/logs`. RESim bugs, e.g., Python errors, are often masked when driving from the IDA Client. When things seem broken, they may well be. Redo what fails using the RESim command line, and you may see the error.

IDA Pro's debugger at times may get lost. You can exit IDA and restart it using the `runIDA.sh` command followed by the `shift-r` hotkey to reconnect to the simulation.

If you find yourself in the kernel, you can usually use the `run2User` command or its IDA debugger equivalent.

The simulation may reach a state that you cannot proceed from. Use bookmarks (or watch marks) to return to a known state.

Information about share object file use by processes is only gathered when directed by use of the `debugProc` command. While you can attach to any process at any time, this key shared object information will not be available. See 9 for more information.

If you observe divergence between different runs of what should be identical simulations, consider whether the real world is leaking into your simulation in any way – see section P.1.

Double-check your ini file snapshot. Recall there are often 2 snapshots used for any given target; one for data injection and one for driver-driven data.

If you observe non-repeatable crashes, e.g., SEGV, confirm you are not injecting data past the end of a read buffer.

If RESim appears to no longer run, e.g., `runAFL` just hangs, check that there are no zombie simics processes running. Use the `kill-simics.sh` script to kill them (but do not kill the `lmgrd` instance).

Reverse track of data might go off the rails for a variety of reasons. When this happens, restart and try reverse-wrote-to-address

If an AFL session seems to hang, stopping it will cause the most recent AFL data to be written to the `last_afl.io` file, which can then be analyzed. instead of the full tracking.

If Simics stops for no apparent reason, e.g., while doing in `injectIO`, view the PC and the breakpoints (`list-breakpoints -all`) and look at the logs to see if some Hap is defined for that breakpoint. Use `showHaps` to see if the breakpoint is part of a hap managed by the `genContextManager`. It may be an ad-hock break/hap, such as those used by the `writeData` module.

IDA sometimes gets confused about memory segments. See `simics/ida/missingSegs.py`.

If a reverse results in RESim quitting simics, it may be because the reversible recorded time frame is too long. How long depends on the simulation. Look for complaints such as "BUT A FUTURE CYCLE" in the log, indicating simics is sick and is going forward instead of backwards.

11 Data Stores

This section describes how RESim names and accesses different data stores.

12 IDA data and hits files

One goal is to simplify sharing between the RESim platform and the IDA platform, which may be the same, or different. IDA and RESim need to share binary files; function and block lists; and hit lists. To avoid NFS conflicts, only read-only files should be on NFS. We trade this off against the desire to share basic data such as IDA's functions and blocks lists.

- Binaries are stored relative to `RESIM_IMAGE`. The RESim ini file defines a `RESIM_ROOT_PREFIX` value that typically begins with `RESIM_IMAGE`.
- The `.funs` and `.blocks` files generated by IDA or Ghidra are stored relative to `IDA_ANALYSIS` in a subdirectory named by the root of the `RESIM_ROOT_PREFIX` value, and extended by the subpath of the binary relative to `RESIM_ROOT_PREFIX`. These analysis artifacts are on an NFS share so that the same ini file can be used on multiple RESim platforms without recreating function and block artifacts.
- The subpath described above avoids name conflicts, e.g., for systems that may have multiple DLLs having the same name at different paths.
- The IDA `idb` and hits files will be per-user, stored in a local writable directory relative to path named by the `RESIM_IDA_DATA` environment variable, using the same subpath as that used for `.funs` and `.blocks` artifacts. Ghidra databases are stored where ever Ghidra put them.
- Files will be copied between IDA and RESim platforms using `scp`.

If IDA and Simics run on different computers, and the `runIDA.py` command is given the name of a remote computer, i.e., the IDA computer and the Simics computer communicate via an SSH tunnel, then the system will copy the RESim-generated hits files to the local `RESim_IDA_data` directory. Otherwise, you are responsible for running `syncIda.sh` from the IDA computer. Provide the optional user name if the user IDs differ on the two machines.

12.1 Coverage files

The hits files are stored as lists of basic block addresses. When `playAFL` (or `runPlay`) is run the hits file includes the name of the target. Per session coverage files are also stored in the AFL output directory. Coverage files are dictionaries keyed with the basic block address and the values are the cpu cycle at which the hit occurs. When the `playAFL` command is given a single file, RESim will display the location of the resulting hits file and the resulting coverage file, as well as any new hits that were found.

The addresses in hits files will be based on the static program addresses, i.e., not adjusted by the load offset. Other data sets that include instruction pointers, e.g., trackio records, should include SO map information so that their instruction pointer values can be normalized.

The `dedupeCoverage` utility reads all of the coverage files under an AFL target and generates a file with a suffix of `.unique` in the target directory containing a list of sessions that create unique sets of execution paths.

13 Utility scripts

The following is a summary of a set of utility scripts in the `RESim/simics/bin` directory. These are typically run from a bash shell in the workspace directory.

- `resim-ws.sh` – Create a Simics workspace for use by RESim.
- `runIda.sh` – Start IDA for a given program. Run this from the Application Root directory named in the ini file.
- `runGhidra.sh` – Start Ghidra for use with RESim.
- `idaDump.sh` – Perform IDA analysis on a given program and create analysis artifacts for functions and basic blocks.
- `idaThis.py` – Run an IDA Python script for a given program, e.g., ad-hoc scripts to find gadgets.
- `runAFL` – Start AFL sessions for a given target.
- `fuzz-summary.py` – Display a summary of AFL status, e.g., for parallel fuzzing.
- `pathcount.py` – Display number of paths and crash files found by AFL (subset of `fuzz-summary.py`).

- `clonewd` – Create a given number of cloned workspace subdirectories to run AFL/RESim in parallel.
- `start-drones` – Start a set of RESim/AFL pairs locally and on remote computers named in the local `drones.txt` file. See [6.14.1](#)
- `stop-drones`
- `sync-resim` – Push RESim code to the list of drones in `drones.txt`
- `sync-drones` – Push the current workspace to the list of drones.
- `syncIda.sh` – Push the current workspace to the list of drones.
- `status-drones` – Get AFL status of the drones.
- `runPlay` – Replay all path inputs generated by AFL to create RESim coverage files for each. Will run in parallel based on the number of clones created with `clonewd`. These files are created in directory structures parallel to the AFL queue files.
- `runTrack` – Replay all path inputs generated by AFL to create track IO watchmarks for each, or to create system call traces using the `--trace_all` option. Sessions will run in parallel based on the number of clones created with `clonewd`. This utility uses `injectIO`, restarting Simcis for each session. If the `trace_all` option is selected, tracing will be of all processes on the target.
- `crashReport` – Run automated analysis on a set of crash files found by AFL. When inputs may include multiple packets, the utility will send packets via the driver computer which is assumed to have port forwarding on port 4022. Use the `-f` option to identify the FD.
- `crashTriage.py` – Parse crash reports and provide a summary. to avoid redefinitions of the origin that would occur if `trackIO` were used multiple times in a single Simics session.
- `findBNT.py` – find branches not taken within the program hits file in the `RESIM_IDA_DATA` directory.
- `findBB.py` – List the AFL session files that reached a given basic block.
- `findTrack.py` – Find a trackio file generated by `runTrack` that includes a watchmark at the given instruction address.
- `showTrackSyscalls.py` – Find trackio files generated by `runTrack` that includes a watchmarks for kernel calls, e.g., to see if your inputs are being passed to file create.
- `showTracktids.py` – Shows data values for watch marks TIDs for all tracks for a given afl target. Intended for use in determining if multiple TIDs reference input data.
- `showCompares.py` – Display the *compare* strings found in the watch marks generated by `runTrack`.
- `bntMerge.py` – Annotate a given BNT report with annotations previously made to corresponding entries in the `old.bnt/` directory.
- `rmLogs` – Remove log files from AFL clones within a workspace (assumes you are in the workspace directory.)
- `showCoverage.py` – Show basic block coverage of an AFL file, or all files.
- `dedupeCoverage` – Identify AFL sessions that resulted in unique sets of hit basic blocks. This can be very slow if there are a lot of paths found by AFL.
- `dataDiff` – Experimental, compare trackio output generated from post-AFL processing.
- `traceDiff` – Compare 2 instruction trace files and locate points of divergence (see the `instructTrace` command).
- `diffHits` – Show differences between 2 hits files.
- `heavyHitters.py` – Find basic blocks having many hits, e.g., to blackball code sections from AFL. See the script.
- `idaDiff` – Compare two coverage hits files in IDA, using basic block coloring to highlight differences. Use `diffHits` to find points of divergence. The hits files must be in the `$RESIM_IDA_DATA/program` directory.

- cycleSeeds – Populate the AFL seed directory with queue files and manually added files identified in the results of the dedupeCoverage utility.
- kill-simics.sh – Sometimes it is the only way, also use when runAFL is given the -background switch
- drive-driver – Send data from a driver computer to a target IP/PORT. A real network is used to push the data and scripts to the driver. There are several ways to send data to a target computer. The drive-driver utility is recommended. See 5.10.1.
- syncIda.sh – Copy hits files from the RESIM_Data_Dir on the Simics computer to a local IDA computer.
- addInput.py – Manually add an input file to the AFL queue files. These will be included as seeds if cycleSeeds is run.
- wmMerge.py – Merge a watch marks report (from saveWatchMarks) into a named system call trace output (from traceAll).
- ckptMerge.py – Use the Simics checkpoint-merge -c command to create a sharable checkpoint. The script first removes all RESim pickle files and then restores them and copies them to the merged directory.
- genJson.py – Package input data into a json file to provide to drive-driver, e.g., for a TCP_JSON session.
- showFDs.py – Show FD assignments at a given cycle from a RESim system call trace file.
- dumpCoverage.py – Dump the content of a coverage file, e.g., generated by playAFL, displaying the hits addresses and cycles in hex.
- winProgSizes.py – Use this to create a file you will name with the WORD_SIZES target value in an ini file. The utility will parse a log file to determine the names of programs that are created, and it will then look in your ROOT_PREFIX directory for those to determine whether the program is 32 or 64 bit. Use the runToCall('CreateUserProcess', linger=True) command to generate a suitable log file for parsing.
- buildExecDict.py – Builds a dictionary of executable files relative to ROOT_PREFIX. This is an optimization for Windows systems that tend to have zillions of files, aiding the system in locating files named in a debugProc. Run this from the workspace naming the ini file.
- findProgRename.py – Read a trace file to find SET_NAME operations that change the comm of a thread. Creates a json of those in the analysis directory which is then used by RESim to equate the comms.
- arch-tar.sh – Archive fuzzing artifacts and optionally a workspace. See the script for details.
- arch-restore.sh – Restore archived fuzzing artifacts and optionally a workspace. See the script for details.
- injectReport.sh – Create a report that merges watch marks, system calls and coverage from a data injection.

14 Testing

Many of the RESim functions are exercised by automated tests found in the simics/testing/cadet-test directory. These include trackIO, injectIO and fuzzing with AFL. The tests create a new RESim workspace relative to the directory from which the tests are run. Use the

```
$RESIM_DIR/simics/testing/cadet-test/tstcadet.sh
```

command to initiate the testing. The tests require the xdotool package. Windows functions are tested using:

```
$RESIM_DIR/simics/testing/windows/wintest.sh
```

15 License

```
* This software was created by United States Government employees
* and may not be copyrighted.
* Redistribution and use in source and binary forms, with or without
* modification, are permitted provided that the following conditions
* are met:
* 1. Redistributions of source code must retain the above copyright
* notice, this list of conditions and the following disclaimer.
```

* 2. Redistributions in binary form must reproduce the above copyright
* notice, this list of conditions and the following disclaimer in the
* documentation and/or other materials provided with the distribution.
*
* THIS SOFTWARE IS PROVIDED BY THE AUTHOR ‘‘AS IS’’ AND ANY EXPRESS OR
* IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
* WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
* DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT,
* INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
* (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
* SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
* HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
* STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN
* ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
* POSSIBILITY OF SUCH DAMAGE.

Appendices

These appendices are a mostly unorganized collection of design notes, rationale and ToDo lists.

Appendix A Analysis strategies

A.1 Analysis on a custom stripped kernel

Use of external analysis, (i.e., observation of system memory during system execution, to track application processes), requires some knowledge of kernel data structures, e.g., the location of the current task pointer within global data. While this information can be derived from kernel symbol tables, some systems, e.g., purpose-built appliances, include only stripped kernels compiled with unknown configuration settings.

Within 32-bit Linux, the address of the current task record can be found either within a task register (while in user mode), or relative to the base of the stack while in kernel mode. Heuristics can then be used to locate the offsets of critical fields within the record, e.g., the PID and comm (first 16 characters of the program name). While the current task record provides information about what is currently running – it cannot be efficiently used to determine when the current task has changed. For that, the RESim tool must know the address of the pointer to the current task record, i.e., the address of the kernel data structure that is updated whenever a task switch occurs.

Once we have the address of the current task record, a brute force search is performed starting at 0xc1000000, looking for that same value in memory. This search resulted in two such addresses being found, and use of breakpoints indicate the one at the higher memory location is updated first on a task switch.

On 64-bit Linux kernels, the current task pointer is maintained in GS segment at some processor-specific offset. This offset is not easily determined – even from source code (see the arch/x86/include/percpu.h use of “this_cpu_off”). A crude but effective strategy for determining the offset into GS is to catch a kernel entry, and then step instructions looking for the “gs:” pattern in the disassembly. The first occurrence of “`mov rax,qword ptr gs:[`” seems to be the desired offset. It is expected that this will vary by cpu. The `getKernelParams` utility needs to be updated for multi-processor (or multicore) systems.

Once the address containing the pointer to the current task record address is located, the `getKernelParam` utility uses heuristics and brute force to find the remaining parameters.

A.2 Detecting SEGV on a stripped Linux Kernel

This note summarizes a strategy for catching SEGV exceptions using Simics while monitoring applications on a stripped kernel, i.e., where no reliable symbol table exists and `/proc/kallsyms` has not been read. In other words, this strategy does not rely on detecting execution of selected kernel code, e.g., signal handling.

Simics can be trivially programmed to catch and report processor exceptions, e.g., SIGILL. However, the hardware SEGV exception does not typically occur in the Linux execution environment. Rather, a page fault initiates a sequence in which the kernel concludes that the task does not have the referenced memory address allocated, and thus terminates the task with a SEGV exception.

When a page fault results from a reference to properly allocated memory in Linux, there is no guarantee that the referenced address has a page table entry. In other words, alloc does not immediately update page table structures – it is lazy. Thus, lack of a page table entry at the time of the fault is no indication of a SEGV exception. Our strategy must therefore account for modifications to the page table.

When a page fault occurs, we check the page table for an associated entry. If there is not an entry, then we set a breakpoint (and associated callback) on the page table entry, or the page directory entry if that is missing. We also locate the task record whose next field points to the faulting process, and set a breakpoint on the address of the next field. If the fault causes a page table update, it is assumed the memory reference is valid. On the other hand, if a modification is made to the next field before a page table update occurs, we assume the modification is part of task record cleanup due to a SEGV error.

RESim’s kernel parameter gathering identifies the entry point hit for page fault processing and RESim uses breakpoints on that address (two in the case of ARM) to perform housekeeping on page faults. RESim uses the `CORE_EXCEPTION` hap to catch undefined instructions.

We do not currently rely on any knowledge of user space signal handlers. If user space is returned to while the page remains unmapped, we (sometimes) assume a fault. Exceptions are intended faults such as brk?

A.2.1 Faults on ARM

The x86 case seems simple compared to what is found in ARM, whose exceptions include “Data Abort”; “Prefetch Abort”; and “Undefined Instruction”. Data references to unmapped pages yield a Data Abort; while

instruction fetches yield a Prefetch Abort. The "Undefined Instruction" is not necessarily fatal – for example we see the vmrs (some floating point transfer) a lot.

Data Aborts lead to page handling, unless it does not. Of interest is that it can lead to references from the kernel to addresses provided by user space.

A.3 External tracking of shared object libraries

During dynamic analysis of a program, the program may call into a shared object library, and the user may wish to analyze the called library. This note summarizes how RESim provides the user with information about shared object libraries, e.g., so that the target library can be opened in IDA Pro to continue dynamic analysis. This strategy does not require a shell on the target system, nor does it require knowledge that depends on a system map, e.g., synthesizing access to `/proc/<pid>/map`

When the program of interest is loaded via an `execve` system call, breakpoints are set to catch the open system call. The resulting callbacks look for opening of shared library files, i.e., `*.so.*`. When shared objects are opened, breakpoints are then set to catch the next use of `mmap` by the process. We assume the resulting allocated address is where the shared object will be loaded. Empirical evidence indicates this simple brute force strategy works. These breakpoints and callbacks persist until the process execution reaches the text segment of the program.

RESim maintains maps of addresses of shared library files. Use the `showSOMap` command to view a list of libraries and their load addresses. The `stackTrace` command identifies the library of the current stack frame (or `show` for the current instruction address. Once you know the library and its load address, open the library in IDA (and use `dumpFuns.py` and `findBlocks.py` to generate databases referenced by RESim if not yet created). Rebase the program (`Edit/Segment/rebase` and then attach to the debugger. NOTE, depending on the library, rebasing may take a while. Wait for IDA to finish before attempting to attach the debugger. Switching between libraries currently requires that you exit IDA and then open the other library.

A.4 Analysis of programs with crude timing loops

Consider a program that reads data from a network interface by first setting the socket to non-blocking mode and then looping on a read system call until 30 seconds have expired. The program spins instead of sleeping. It calls "read" and "gettimeofday" hundreds of thousands of times.

Creating a process trace on such a program could take hours (or days) because the simulation breaks and then continues on each system call. This note describes how RESim identifies this condition as it occurs, and semi-automated steps it takes to disable system call tracing until the offending loop is exited.

While tracing system calls of a process, invocations of system calls are tracked and compared to a frequency threshold. When it appears that the program is spinning on a clock or an event such as `waitpid`, the user is prompted with an option to attempt to exit the loop. If the user so chooses, RESim will step through a single circuit of the timing loop, recording instructions at the outermost level of scope. It then searches the recorded instructions to identify all conditional jump instructions, and their destinations. Each destination is inspected to determine if it was encountered within the loop. If not, the destination and the comparison operator that controlled the jump is recorded. Breakpoints are set on each such destination address. We then disable all other breakpoints, e.g., those involved in tracing and context management, and run until we reach a breakpoint. This feature is called a *maze exit*. If you would like to avoid the prompts, use the `@cgc.autoMaze()` function to cause the system to automatically try to exit mazes as efficiently as it can. Use `noExitMaze()` to disable loop checking.

RESim includes an optional function to ensure that the number of breakpoints does not exceed 4 (the quantity of hardware breakpoints supported by x86). If more than 4 breakpoints are found the analyst can guide the removal of breakpoints. RESim will automatically execute the loop a large number of times in order to identify comparisons that may be converging. And the user is informed of those to aid the reduction of the quantity of breakpoints. (Note that in the context of this issue, there are less than or equal to 4 breakpoints and more than 4. There is probably a lot more than 4 as well, but we've not yet quantified its effects.)

This feature is currently only functional on X86 platforms.

A.5 Stupid close calls

Consider this common coding practice:

```
for (i = 0; i < 4096; i++) {
    close(i)
}
```

Some setup code may have dozens of such loops. If tracing all system calls, this can slow down the simulation and generate rafts of useless trace output. The `stupidClose` module detects these and suspends tracing of the thread until the calls have completed. It requires such processes to have analysis artifacts so that it may track basic blocks between the failed close and the next syscall in order to find an escape path. Name a list of offending programs in the file named by `STUPID_CLOSERS` in the ini file.

A.6 Breakpoints can be complicated: Real and virtual addresses

Use of a full system simulator enables *external* dynamic analysis of the system. The analysis is said to be “external” because the analysis mechanism implementation, e.g., the setting of breakpoints, does not share processor state with the target. A distinguishing property of external dynamic analysis is that the very act of observation has no effect on the target system. This lack of shared effects improves the real-world fidelity of the observed system, but it can also complicate the analysis, particularly when referencing virtual addresses.

This property of external analysis is illustrated by tracing an “open” system call. Assume the simulator is directed to break on entry to kernel space. At that point, we can observe the value of the EAX register and determine if it is an “open” call. We can then observe and record the parameters given to the open system call by the application. However, the name of the file to open is passed indirectly, i.e., the parameters contain an address of a string. How might we record the file name rather than just its address?

Requesting the simulator to read the value at the given virtual address of the file name will not always yield the file name because the physical memory referenced by the virtual address may not yet have been paged into RAM by the target operating system. If the analysis were not external, then the mere reference to the virtual address could result in the operating system mapping the page containing the filename. An external analysis has no such side effects.

The simulator includes different APIs for reading virtual memory addresses and reading physical memory addresses. The former mimics processor logic for resolving virtual addresses to physical addresses based on page table structures. Attempts to read virtual addresses that do not resolve to physical addresses result in exceptions reported by the simulator – they do not generate a page fault.

Waiting to read from the file name’s virtual address until after the kernel has completed the system call, i.e., until the kernel is about to return to user space, would ensure the virtual address containing the string will have been paged in. However, that strategy is susceptible to a race condition in which the file name is changed after the kernel has read it but before the trace function records it. This may occur if the file name is stored in writable memory shared between multiple threads, and could result in a trace function failing to record the correct file name used in an open system call.

Since we know the kernel will have to read the file name in order to perform the open function, we can set a breakpoint on the virtual address of the file name, and then let the simulation continue. When the kernel does reference the address, the simulation will break. In some implementations, the memory will still not have been paged in, (e.g., the kernel’s own reference to the address generates a page fault), but leaving the breakpoint in place and continuing the simulation will eventually allow us to read the file name as the kernel is itself reading it. Except, that is true for only for 32-bit kernels. 64-bit kernels only reference physical addresses when reading filenames from pages that had not been present at the time of the system call. In other words, in 64-bit Linux, breakpoints may never be hit when set on the virtual address of a file name referenced in an open call.

Even though the kernel is able to read the file name without ever referencing its virtual address, the kernel does need to bring the desired page into physical memory so that it may read the file name. It happens that while doing so, the kernel updates the page tables such that references to the virtual memory address will lead to the file name in physical memory, *even though such a reference may never occur*. RESim takes advantage of the kernel’s page table maintenance by setting breakpoints on the paging structures referenced by the virtual address. In some cases, the target page table may not be present at the time of the system call, so we must first break on an update to a page directory entry, and then later break on an update to the page table entry, finally yielding address of the page in physical memory that we can read.

Note this property of the 64-bit Linux implementation has implications beyond tracing of system calls. A reverse engineer may wish to dynamically observe the opening of a particular file name observed within an executable image. Setting a “read” breakpoint on the virtual address of the observed file name would fail to catch the open function on 64-bit Linux, while it would have caught the open on 32-bit Linux.

Appendix B Divergence Between Physical Systems and RESim Simulations

B.1 Overview

This appendix identifies potential sources of divergence between RESim models and real world systems and presents some strategies for mitigating divergence. Models that lack fidelity with target hardware may lead to divergent execution of software. Consequences can range from scheduling differences, (which generally also diverge between boots of the same hardware), to substantially different behavior between the model and the physical system. For example, on some boots of a simulation, a set of Ethernet devices may be assigned incorrect names, e.g., “eth0” is assigned to the device that should be “eth1”. In some cases, these divergences can be mitigated if detected. In our example, if the eth0/eth1 are found to have been swapped, then reversing their corresponding connections to switches might mask the problem, restoring fidelity between the simulation and the physical system. (See 5.8.)

B.2 Timing

Simics processor models execute all instructions in a single machine cycle. The quantity of machine cycles required to execute instructions varies by instruction on real processors. Most designs for concurrent process execution do not rely on cycle-based timing. However, race conditions that appear on real systems may manifest differently on simulated systems.

B.3 Model Limitations

It is not always practical to obtain a high fidelity model of every component in a target system. Models may lack specific peripheral devices expected by a kernel. In some cases, functionally compatible devices can be substituted for missing peripherals. For example, the kernel image from an X86 target may include drivers for existing Simics ethernet devices, and yet the initialization functions do not cause the corresponding modules to be loaded – rather, they load kernel modules for an ethernet device that is not modeled. Use of the Dmod function described in 5.8 can cause the kernel to load the desired module, without having to modify the disk image. TBD: Explore model building, e.g., to simulate an fpga accessed via a PCI bus device.

B.3.1 VLANs

While Simics has support for VLAN switches, the network interface models do not support VLAN definitions in which the VLAN subnet differs from that of the parent interface, and the model requires the parent interface to be defined. Wind River considers this common configuration of VLANs (with differing subnets) to be a “use case” beyond their intended features.

B.3.2 File descriptors

File descriptors (FDs) can diverge between boots of some systems in ways that seem non-deterministic. Additionally, divergence between modeled hardware and real hardware can contribute to FD divergence. For example, if a model lacks a peripheral device cause an `open` to fail, subsequent FD assignments on the model will not match those used on the physical system.

Appendix C What FD is this?

You have created a input stimulus and would like to understand the resulting behavior of a specific process that runs a long time as a service. Running a full system trace up to the stimulus as a means of getting context may not be practical. So you start a `traceAll` at some point prior to the external stimulus. You then see socket activity on a specific FD. However the current trace does not include a connection that maps to that FD. Use the `procTrace.txt` file to get the pid of the process as it existed in the full system trace. Then, if you assume the FD will match that from your full system trace (which may be a fine assumption if the connection is long-term), grep on the system call trace, e.g.,

```
grep "pid:1731" syscall_trace.txt | grep "FD: 11
```

That gives you the port and address information that you can then search for in the `netLinks.txt` file.

Alternately, if you have a trace file, use the `showFDs.py` utility.

Appendix D Context management implementation notes

RESim manages two Simics contexts: the default for the cell, and a “resim” context for each cell. Contexts are managed within `genContextManager.py`. The resim context is used when watching a specific process or thread family. Otherwise, the default context is used. The context of each processor is dynamically altered such that breakpoints are only hit when in the corresponding context. For example, assume we are debugging PID 95 (for simplicity, assume no threads). Whenever PID 95 is scheduled, the processor context is set to resim. The context is returned to the default whenever PID 95 is not scheduled. System call breakpoints set during debugging, e.g., `runToWrite`, will be associated with the resim context. These breakpoints will only be hit when the processor is in the resim context. Breakpoints may be set in the default context while debugging, e.g., handle a background Dmod (see 5.8.5). Those breakpoints will not be caught when processor is in the resim context. TBD: define multiple contexts to allow parallel debugging of different processes on the same cell. (Parallel debugging of processes on different cells is easier since they each have independent context managers.)

IDA breakpoints and the Simics debug server make assumptions about the current context. If you are in an arbitrary state, you will need to reestablish the debug context before IDA breakpoints will work. Use the `cgc.resynch()` or Run to user in IDA to run the simulation ahead to the debug context.

The `playAFL` function uses the resim context for page faults and `exit/terminate` syscalls.

Appendix E What is different from Simics?

RESim is based the Simics simulator, including the *Hindsight* product. It does not incorporate the *Analyzer* product OS-Awareness functions. RESim includes its own OS-Awareness functions. The Simics Analyzer features are primarily intended to aid understanding and debugging of *known* software, i.e., programs for which you have source code. RESim is intended to reverse engineer unknown software, and thus does not assume possession of source code or specifications.

Appendix F IDA Pro issues and work arounds

The IDA debugger in 7.2 requests more registers than Simics knows about. The Simics gdb-remote could be modified to simply return the value of the highest numbered register that it knows about. The suggested alternative is to modify xml files in `/ida-7.2/cfg` to only include desired registers. For ARM (the `qsp-arm`), the `arm-fpa.xml` (from gdb source tree) needs to be added to the `arm-with-neon.xml` file in `ida/cfg`. The `cpsr` register (number 25) is not updated by IDA unless registers are defined for each register value returned by Simics in the ‘g’ packet.

The IDA debugger in 7.2 uses thread ID 1 when performing the vCont GDB operation. Hex-rays modified the IDA gdb plugin to use ID 0, thereby fixing the problem.

The block coloring in IDA fails sometimes, possibly when the function includes unmapped pages. The coloring works fine prior to attaching the debugger. After attaching the debugger, some blocks revert to their default color. Use the Debugger/RESim/Recolor Blocks menu function to recolor the blocks. This will usually correct the problem. But by that time you already know something is messed up.

Appendix G Simics issues and work arounds

- New-style Simics console management causes an X11 error: “The program ‘simics-common’ received an X Window System error.” Use:

```
gsettings set com.canonical.desktop.interface scrollbar-mode normal
```

to avoid the exception.

- The ARMv5 model had a flaw that reports in incorrect fault status on some data aborts. Wind River has now issued a patched release of the Integrator (6.0.12).
If you lack the patch, see the `fixFaults` function in the `afl.py` module.
- The service nodes used for real-network interaction were not saving their ephemeral port numbers on write-configuration. As a result, the server could see reused port numbers, causing the TCP connection to fail after a restore from the snapshot. Wind River has fixed this. We currently have a patch that will be included in the next Simics 6 release in mid 2022.

Appendix H Performance tricks

Use `disable-reverse-execution` whenever you do not really need reversing, e.g., while moving forward to a known connect or accept within a program being debugged. Actually, use `cgc.noReverse()` and `cgc.allowReverse()`, which will disable the sysenter Haps maintained by the `revToCall` module. Programs can be pathological with their use of syscalls.

Instead of tracing all from a boot, consider running ahead until the `systemd-logind` or `rsyslogd` is created.

Pay attention to the `BACK.STOP.CYCLES` environment variable in the ini. A large value may be needed during initial testing, but that may need to be greatly reduced for fuzzing.

If you are limited to a single instance of Simics, try using the `-k` option to `runAFL` to force use of AFL's quick and dirty mode.

Some systems may take a long time to create the process you are interested in. This may be exacerbated by monitoring such as occurs during a `runTo` or `debugProc` (e.g., some script is in a loop failing on create of a process, which may happen millions of times.) Consider using `runToCycle` to run forward to a desired cpu cycle with the least amount of overhead.

Use `system-perfmer sample_time = N` where N is in seconds to view current performance. Use `-deactive` to turn it off.

Some programs behave crudely while waiting for events. E.g., calls to `waitpid` or `timeofday` can sometimes occur at ridiculous frequencies. If `RESim` is tracking context switches, e.g., a process is being debugged or `only/ignore` is employed, performance can become ridiculous. Usually, if you disable debugging or whatever is causing `RESim` to watch context switches, you'll see the system run quite fast. Try doing what you can without causing `RESim` to watch context switches. For example, if you want to run forward until some process accepts on an FD, do so without setting `debug`. Then, once the accept is done, you can switch to `debug`.

Simics 6.0.157 snapshots were about 30% faster than more recent versions, including Simics 7.

See [A.5](#) for handling bulk calls to close.

Appendix I New disk images

For x86, use `put` the `workspace/dvd.simics` in targets. Edit it to name an installation iso. You may need to change the date in the file, and run with `realtime` enabled. After the install, use `save-persistent-state`, and then rename the resulting `crash` as needed.

When using real networks, configure the image routing and `resolv.conf`, using your ip address:

```
route add default gw 10.10.0.1
echo nameserver 10.10.0.1 > /etc/resolv.conf
```

Appendix J Implementation notes

J.1 HAPs

Use the `VT.in_time_order(self.vt_handler, param)` when you need to know the memory transaction that led to a stop hap during reverse. See `findKernelWrite` for an example.

Take care when using `SIM.hap.add_callback_range` to ensure your breakpoints are truly in a contiguous range. Concurrency often results in dis-contiguous allocations of breakpoint numbers with the "holes" subsequently used for other purposes. Imagine your confusion when a hap is hit for the wrong event. If needed, issue multiple haps on multiple ranges, watching the breakpoint sequence numbers as they are allocated. Simics breakpoint numbers may look like handles, but the hap range allocation gives their values semantics. See the `coverage.py` module for an example.

HAPs are often hit after they are deleted. Always set a deleted HAP to `None` and check that at the start of the HAP. Deleting a breakpoint does not stop its HAP from being called when the deleted breakpoint's address is hit. As such, optimization schemes should not try to reduce breakpoint counts on the fly.

J.2 Thread tracking

Thread tracking is necessary to record address space information of threads for use in analysis, e.g., knowing which library is executed while referencing input data. The scope of threads being tracked is constrained by the `only/ignore` lists. If a process is being debugged, other process threads are tracked unless `track.threads` is set false. Thread tracking also maintains data on initial stack pointers for each thread as a means of optimizing stack frame generation. The context manager facilitates this by catching the scheduling of new threads and calling `top.recordStackClone`.

J.3 Fuzzing anther process

When the fuzzing target differs from the process that initially consumes the fuzzed data, we must first make sure the target gets scheduled so that we can record and set coverage breakpoints. We assume the physical addresses of the resulting code will not change, e.g., due to other processing that occurs prior to the target being scheduled in response to any given input. This assumption may not always hold, especially if the target is not even created yet. When a different target is named, the playAFL and afl modules will inject data and run forward using debugProc to get the target scheduled.

We must also watch page faults for the target, and we want to catch its exits. What if process does not yet exist?

Introduced a **primer** option to provide an input file known to cause the target to be rescheduled so its coverage can be mapped.

The primer must be added to runPlay and runTrack chains.

Appendix K ToDo

Note Some of the entries below have been at least partially implemented. The entries remain to encourage testing and completeness.

K.1 Msc

- Stack trace on x86 assumes use of bp register unless OS is windows. This is a poor assumption. Need automated way of looking at functions. Though libraries or compilation units may differ in same program.
- Dmod support for altering shadow entries, e.g., to root "nopassword" shadow entry:

```
root:$1$60RujciF$2Fz0vN50V4xLBYtEimdSn1:14277:0:99999:7:::
```

- dumpStack option to only show reasonable candidate entries.
- Trace params of pctl and catch setname, which is used by systemd to change thread name to rsyslog. Should be able to break on this like we do for execve.
- Catching kill of process uses breakpoints on task record next field that points to the subject process's task rec. Fails if the process whose record is watched is killed. Need to also watch "prev" on the subject process?
- Convert traces to csv and explore data presentation strategies for navigating processes and IPC.
- Feature to flexibly identify user-space libraries to be traced.
- Enhance the **dataWatch** function improve detection of generic C++ string allocator, and expand watch. Still missing some destroy/dallocation of string.
- Tracking memory-mapped IO is not directly supported. Perhaps catch the mmap function call and somehow determine it is mmio purposed. Then set breakpoints...
- sscanf data tracking not right for some arm, if 1 or 2 values, r2 and r3 will point to where those go rather than using a pointer list.
- The playAFL function could disable reverse execution, also have it set jumpers to not clear bookmarks.
- Confirm crashReport uses jumpers.
- Add read counts to data watches.
- dMod scriptReplace sets an lseek DMod to adjust future lseek calls. The dmod sticks around until the file is closed (even then?, would have to track pid/FD).
- Add fuzzing/coverage option to instrument main program and a lib(s).
- IDA hotkey to go to address adjusted by the start of the LOAD segment.
- Ad-hoc copies in dataWatch not robust for arm, ignoring register overwrite until we can eliminate case where register is overwritten with a value derived from its value (x86 has same robustness issue, but it is not ignored).

- Investigate ways of limiting IDA debug data sent between client and Simics. Causes slow responses on remote debugs?
- Extend maze exit to ARM.
- Error handling when running AFL. Filter failures cause a hang with no messages. Simics crashes sometimes seen in /tmp/resim.log
- writeData has no fallback for a page fault. Add a hap on the page table to complete the write? NO, force big prepInject data
- Use of debugSnap after having run a while loses state information about threads waiting in syscalls. Add warnings if simulation is continued without debugging if there is a snapshot. Use hap that catches continue?
- The ida .idb files are per-user, but the function dumps are shared. Thus function names may not match. Add tool to update idb based on function list?
- (DONE) runToWM may diverge if dataWatch code changes, e.g., the trackio artifacts differ from what you'd see now. Change the function to look for the address as it does its processing, and then jump to it rather than relying on the watch mark given by the trackio artifact.

K.2 Data Watch and Watch Marks

K.2.1 Data Watch ntohs

The ntohs type functions are common in some programs. RESim catches some (but not all) instances of the results of these functions getting copied into memory buffers:

```
call    _ntohl
0808A7E0 add    esp, 10h
0808A7E3 mov    edx, eax
0808A7E5 mov    eax, [ebp+arg_0]
0808A7E8 mov    [eax+68h], edx
```

K.2.2 Data watch ad-hoc copies

Treated as an ad-hoc copy?:

```
mov x, buf
push x
call foo
mov bar, x
```

K.2.3 Watch Marks and Parsing

Watch marks created by complex regular expression parsers such as perlMatcher may show multiple marks where you might expect to see one. The ad-hoc nature of RESims regular expression detection can result in several adjacent marks that are part of the same call chain. When RESim first sees a data reference that appears to be a regex character lookup, it finds the call to that function. However it may be that the calling function might also be within a parser and it may then make a data reference, and so on. One approach to making better sense of such watch marks (particularly when there are many of them) is to perform post-processing to consolidate multiple marks.

K.2.4 Tracking values returned in registers

Data returned in registers, e.g., eax, from functions such as NTOHL or a getc from a buffer may be pushed onto the stack for passing to some other function that will store (or otherwise do something of interest with) the data. Or it may store the data into memory. Or it may do both. Currently this is not completely handled.

K.3 I/O via threads

Network traffic is sometimes sent via a thread, making it challenging to track the source of the data that was sent. Breaking on a "sendto" may lead to a thread that only does the sendto, with the parameters coming from a clone setup. Thus, you must find which pid does the send to and then runToSyscall until that pid is created.

K.4 Tracing library calls

It should be straight forward to instrument selected relocatable functions or statically linked functions. May be tempting to do that for malloc – but on the other hand, if you have the address of the start of a buffer, then reverse tracking that will generally lead to the malloc call.

Appendix L Dynamically loaded Linux programs

We find the location of the loaded text segment by looking at entry point of the loader named in the target binary ELF header. We subtract that from the address at which we return from `execve`, (which we assume is the loader entry), and set a break from zero to the base of the loader. This assumes that no libraries are executed before the program start is hit. If that turns out to be wrong, we can track the libraries and set holes in the break range.

L.1 Watching process exit whilst jumping around time

We try to catch page faults, or other events that lead to process death. This is performed in the `ContextManager`. This mechanism is also central to catching access violations. The mechanism works fine moving forward from a clean state. But how does it behave if we jump backward to an arbitrary time, e.g., prior to the demise of a now dead process? A new `ContextManager` function will reset all process state to that currently observed. TBD, also modify tracers to ignore events that occur prior to end of recording?

L.2 Defining new targets

System names are defined by assigning "`name = whatever`" within the `...system.include` file's line that creates the component, typically the board. Simics parameters are typically **only** added to the script file in which they are used, and then sucked in by the calling scripts using `params` from. Systems scripts error reporting is sparse and tedious. Copy the needed Simics scripts from their distribution directory into `simics/simicsScripts/targets`, and change the `simics` env variable to scripts where needed – or remove preface and use relative file if local.

L.3 Fuzzing: too many crashes

Some applications are just too easy to crash. This can result in scores of “unique” crashes from AFL. Often, these are all the same crash with a small change in how we get there. Any heuristic that suppresses a crash seems like a bad idea, unless analysis convincingly demonstrates otherwise. Perhaps a tool to scan crash reports to eliminate duplicates, i.e., we usually don't care how we got there if a high confidence stack frame is the same? And `crashReport` function itself should run that tool when done, along with a `grep` on ROP.

L.4 Branches not taken

The `injectToBB` function simply finds any fuzzing input that leads to the given basic block, and then injects that data. If the bb includes a `select` or similar, the execution may never reach the branch logic. Alter the `findBNT` to filter out those that do not follow any of the branches.

Manually working through BNTs can yield manual inputs that reach new branches, but may always crash. These inputs are not helpful to fuzzing because crashes don't feed the guidance system. These manual inputs can also cause `injectToBB` to bring up a crashing input even though non-crashing inputs might yield the same bb. More reason to keep crashing inputs out of the manual batch. Create a new manual crashes? Manual crashes would have their own coverage, which would be referenced when running `findBNT`, but would not be referenced by `injectToBB`.

Similarly, manual BNT analysis might yield crashes. Do not let that stop you from looking at other inputs leading to the branching BB, i.e., to find inputs that yield new code paths without crashing, since these are helpful to fuzzing and coverage. Crashes are good, but sometimes they are dead ends. So use `findBB` to locate other inputs besides the one picked by `injectToBB`.

L.5 Skipping over kernel calls

The data injection scheme catches kernel reads and skips over them while injecting data. This affects timing/scheduling because the kernel never blocks.

L.6 More on kernel data buffers

On a 32 bit x86, there were many 14xx sized kernel buffers. A new option for trackIO (used prior to calling `prepInjectWatch`) was created for recording these buffers. This option, `kbuf=True`, requires drive-driver to send as large a data stream as you wish to support for future injectIO or AFL sessions, and this data stream must be crafted so that the application reads all of it. A hap is set on the read/recv call, and this hap sets a write break on the start of the application buffer. When that is hit, we assume that esi contains the start of the kernel buffer, and it is recorded. A hap is also set on read returns. The consumption of the kernel buffer is monitored along with the amount of data read by the application such that before a kernel buffer is consumed, a write break is set on the next application memory location that will receive the first byte from the next kernel buffer.

The snapshot will be created just before the kernel copies the first data from a kernel buffer into the application buffer. Setting the snapshot at the kernel entry will fail if the read/recv is blocking, i.e., the data may not be present in the buffer yet.

Kernel buffers should not be used for applications that request to read more than it will get. For example, a TCP application that provides a count of 40K should be handled by `prepInject` (similar to UDP fuzzing) rather than `prepInjectWatch`.

L.7 Computed data (e.g., CRC) considerations

And fuzzing design in general... In addition to basic problems that can be solved by filters, sometimes fuzzed data does not conform to system state values. For example, previously received data may contain a crc and/or length of the data to be fuzzed. Jumpers around crc calculations and memory modification should be avoided simply because it requires code analysis to determine completeness, e.g., is that the only reference to the predetermined code length? In general, it is preferable to start the fuzzing at the inject of the earlier data, e.g., by using a filter.

L.8 UDP without headers

The injectIO and AFL design for UDP relies on their being UDP headers to break data into multiple packets.

L.9 TCP conversations and fuzzing

Consider a case of fuzzing TCP. Assume the target is a client and the driver is a server. The driver component is directed to run a server that accepts on the port and sends a given file of data.

You use the `kbuf` option to trackIO and then `prepInjectWatch` which gives you snapshot on the call to recv with lots of data in the kernel buffers.

Fuzzing yields the start of a conversation, i.e., where the target makes a 2nd call to read (or poll/select). It is assumed the first call to read had a length larger than the count. (Find a way to distinguish a conversation from just reading more data. For example, one or more writes to the socket between the reads.)

We'd like to use this conversation start to advance system state, and then start fuzzing with the next read. Use a multifile driver directive to force a read between the two writes.

Assuming the target is a server, a work flow might look like:

- Create an initial snapshot ("snapshot0") in which the server is ready to accept connections. This snapshot will be the direct parent of all other snapshots. (Once a `prepInject` snapshot is created, it cannot be reliably advanced.)
- Set the ini file snapshot to `snapshot0`
- Use trackIO with `kbuf=True` on a driver directive that sends Zs.
- `prepInjectWatch(1, 'snapshot1')`
- Set the ini file snapshot to `snapshot1`
- fuzz (session 1)
- Find a dangling conversation queue file, e.g., the service does a write to the socket after consuming data, and then issues another read (or poll/select).
- Set the ini file back to `snapshot0`
- Create a directive that starts with that queue file, and is followed Zs, with an intervening read.
- Use trackIO with that directive, which should hit a backstop after processing the first part of the conversation.

- Again use trackIO, this time with kbuf=True
- prepInjectWatch(N, 'snapshot2'), where N is the watch mark of the 2nd kernel read.
- Set the init file to snapshot2
- fuzz (session 2)
- Find the next dangling conversation from fuzz session 1 and repeat
- When all fuzz session 1 conversations have been run, proceed to fuzz session 2 dangling conversations.

We are interested in whether fuzz session 2 reaches code not hit by session 1. There should be some data mapping between the afl output of the fuzz sessions, the snapshots, and the directive file that led to the snapshot. All snapshots descend directly from snapshot0.

L.10 Fuzzing crash detection

While fuzzing, we want to detect and report crashes and/or process exits. We watch crashes by watching for modifications of the next pointer in the link lists previous entry. The crashes are seen by AFL. Exits are watched during playback and are displayed to the user. For Linux, the `exit_group` and the `tgkill` calls are watched. For Windows, the `TerminateProcess` is watched. (TBD add `TerminateThread`?)

L.11 BNTs and compares

Catch blocks such as these within markCompare.

```
LDR    R3, [SP,#0x50+var_44]
CMP    R3, #0
MOVNE  R3, #0
STRNE  R3, [SP,#0x50+var_44]
CMP    R0, #0
BEQ    loc_6BEE8
```

Some BNTs are at cycles that are not likely to be affected by input data, e.g., occur prior to parsing. However we only record the first hit of a bb, so we cannot weed these out without running and seeing if early blocks are hit again.

L.12 Shared libraries and jumpers

The jumper loading functions takes the name of a library and offset into that code. If the library is not yet loaded, a watch is set in the SO manager. Once the SO is loaded, an attempt is made to get the physical address of the jump source. If not yet mapped, a callback is set using the `pageCallback` module. This scheme presupposes these things all happen while the process of interest is schedule. We want to set the jumper on physical memory so that it affects all processes. Since we do not know which process will be first in mapping the page containing the jumper address, we must set page callbacks on every process that loads the library.

Also, we must be able to load a set of jumpers from arbitrary snapshots, where the process of interest is not scheduled. If a library is already loaded at the point of the snapshot, but not by the currently scheduled process, we need a way to obtain the physical address. The cleanest approach would be to have the SO module find a task with the library loaded, and then get the page table address (what would be loaded into CR3) for that task. While Windows stores that value at a small offset within `EPROCESS`, Linux stores it within the `mm_struct` at a location that is not trivially found. The Windows implementation has been tested. More testing is needed for the Linux x86 and arm implementations.

A library shared between multiple processes does not necessarily have a shared physical memory address. Thus library load callbacks and page table callbacks must persist, even after some process has set a break on a physical address. And you must track all processes that might load the target library since they may end up with unique physical addresses.

Appendix M Driver platforms

This appendix describes the driver images used internally at:

```
$RESIM_IMAGE/driver/driver2.disk.hd_image.craff
$RESIM_IMAGE/driver/driver_22.disk0.hd_image.craff
```

The driver `driver2.disk.hd_image.craff` is also available at <https://nps.box.com/s/ffuz7fgyn770xcgrdur0uf1bo1tur2gk>
The driver `driver_22.disk0.hd_image.craff` is at <https://nps.box.com/shared/static/je7bhh145oilzhk1mjnrg3h78j4bzd>
`craff`

The driver platform defined by the `RESim/simics/workspace/ubuntu_driver.ini` example has the Simics Agent pre-installed. The `driver-script.sh` in that directory can be copied and modified within your workspace. The driver will download that shell script and execute it when booting. Your target will not boot until that script finishes (and creates the `driver-ready.flag` file). Note the simulation will hang while processing that script, so launch any long running programs in the background. The sample driver script loads an `authorized_keys` file from that directory. This allows ssh into the driver without passwords and uses rsa key files also found in that directory. The sample also includes commands for creating a DHCP server and a DNS server, which are commented out by default.

The username and password for the driver is `mike`. No password is required if ssh keys are used. Use `enable-real-time-mode` to avoid login timeouts and network timeouts when interacting directly with the driver.

The driver device has 4 ethernet interfaces. The mac addresses below are as defined in the ini file:

- `ens11f0` – mac: `00:e1:27:0f:ca:a8`
- `ens25` – mac: `00:19:a0:e1:1c:9f`
- `ens12f0` – mac: `00:1a:a0:e1:1c:9f`
- `enp7s0`: mac: `00:1a:a0:e1:1c:a0`

Note that use of the ethernet `i82543gc` device changes ethernet device names, e.g., `ens11f0` becomes `ens11` and `ens12f0` becomes `ens12`.

Use `ip addr` commands within the `driver-script.sh` to modify these addresses are required.

The mapping of RESim ethernet names to Linux device names in the driver is:

- `eth0` - `ens25`
- `eth1` - `ens11`
- `eth2` - `ens12`
- `eth3` - `enp7s0`

Use the `ETH` directives in the ini file to connect different ethernet devices to the different switches. The defaults are `ETH0-to-switch0`, etc.

The ini file maps multiple ethernet devices to `switch0`, which avoids the problem of tracking connections. This works so long as the networking is simple. The following Simics command provides real-network access to this sample driver:

```
connect-real-network 10.20.200.91 switch0
```

Once the driver is booted, you can then ssh to it using:

```
ssh -p 4022 mike@localhost
```

You should first put Simics in real-time mode (`enable-real-time-mode` to avoid timeouts. You may also need to clear out the ssh keys used by localhost, e.g.,:

```
ssh-keygen -f "/home/mike/.ssh/known_hosts" -R [localhost]:4022
```

M.1 Using the driver component

The driver is a simulated computer that is intended to interact with simulated targets. This is typically achieved by either using SSH to establish a shell on the driver, or by using the `drive-driver` script to send files to the driver along with directives describing where and how to send those files to targets.

Large scp operations may stall. Consider using the `simics-agent` to move files to the driver. However, these only happen one file at a time, so use `tar`.

Smaller scp and ssh sessions seem to work well with drivers. Add an `authorized_keys` file to the `driver-script.sh` to facilitate xfer of scripts/data to the driver during the simulation. Take care to reset the reversing origin after you are done with the real-networ (see the `trackIO` reset option.)

Setting up a proxy from your local host through the driver to the target can be done using ssh tunneling. For example, to proxy to a web server:

```
ssh -p 4022 -AfN -L 8081:10.0.0.10:80 -oStrictHostKeyChecking=no -oUserKnownHostsFile=/dev/null \
-o "ServerAliveInterval 60" mike@localhost
```

M.2 Notes on updating the driver

If updating the driver, use the `DRIVER.WAIT=NO` directive in the ini file to prevent RESim from waiting for the driver to finish (the `driver_ready.flag`) – at least until you finish configuring a driver.

The Wind River *real network* interfaces are not entirely stable, as a result, the real-network connect used to update the driver can often stall/hang. Be prepared to kill apt-gets and retry multiple times depending on your network setup. It may eventually work. If major package installation is needed, you may need to create a VM using VirtualBox or such.

The Python world uses SSL certificates in a manner that leads to breakage. If pip fails on SSL validation, use the `-v` option to find which new python.org server is causing the problem, and use the `--trusted-host dog.pythonwhatever.org` option on pip. The pip3 system is sicker still and requires 3 trusted hosts. This seems to work when installing the scapy package:

```
pip3 -v install --trusted-host pypi.org --trusted-host files.pythonhosted.org scapy --trusted-host pypi
```

To update the driver, e.g., with new packages, use the driver.ini configuration (after first adding the `DRIVER.WAIT=NO` directive. Set the default route on the driver to the service node address, e.g.,

```
ip route add default via 10.0.0.1 dev ens25
```

And connect to the real network:

```
connect-real-network 10.0.0.91 switch1
```

And set the service node as the dns server in `/etc/resolv.conf`, e.g., `nameserver 10.0.0.1`. You can then use apt-get to get new packages. And scp to push files onto the machine. Then use save-persistent-state and bin/checkpoint-merge to create a merged craft. **Do not** forget to shutdown or sync before saving state!. Give the driver craft a name that does not conflict with existing names. The files in `simics/examples/driver` can be put into a new workspace for purposes of updating the existing driver.

Creating a new driver:

- Create a new workspace using Simics 146 to avoid UEFI disks that do not work with RESim.
- Copy `simics/workspace/dvd6.simics` to the new workspace and edit it to point to the Linux installation iso image.
- Run `./simics dvd6.simics` and use the vga window to perform the installation.
- Stop simics and use save-persistent-state to save the disk. This will take a while.
- use bin/checkpoint-merge to save a craft image.
- Copy the `simics/workspace/driver.ini` to the workspace
- Edit the driver.ini to point to your merged craft image and set TARGET to None.
- Use `resim driver.ini` to boot the driver and login via the serial port terminal.
- Add this to the `/etc/sudoers` file: `mike ALL=(ALL) NOPASSWD:ALL`
- Define a network device for temporary use, using IP that will be named in the real-network command below.

```
sudo ip addr add 10.0.0.91/24 dev ens25
```

- Add the `simics/workspace/authorized_keys mike/.ssh/` on the driver using vi/copy/paste.
- At the simcs prompt, create a real network service node:

```
connect-real-network 10.0.0.91 switch0
```

If creating a new driver, add the `simics-agent` from `RESim/simics/simics-agent` to `/usr/bin`. And create a systemd service to run the `driver-init.sh` script from the `RESim/simics/workspace`. That script bootstraps the driver's ability to pull and execute the `driver-script.sh` from the workspace.

To create a param file for the driver, edit the driver.ini file to rename the component from *driver* to anything else. Otherwise special handling for driver components will send you in circles.

M.3 Notes on driver-server.py

The drive-driver utility interacts with the driver-server.py service running on the driver. UDP is used to avoid timing issues that arise with TCP and the disparate clock rates between the simulation and the real world. The version id for the server is kept in `simics/bin/driver_server_version`, which is stored in snapshots and copied to the workspace for reference by drive-driver to determine if the driver-server needs to be replaced prior to data xfer. The normal drive-driver protocol is used to replace the driver-server. If that is not possible, then the `update-driver-server.sh` script should be run, and a new snapshot taken.

The driver-server.py service has a `driver_server_version` hardcoded into its first ack that drive-driver then uses to determine if driver-server.py needs updating, in which case the usual drive-driver protocol is used to send an install an update. The version value is recorded in RESim snapshots, and drive-driver uses this as the

Appendix N Simics user notes

This section includes ad-hoc suggestions for users with little Simics experience.

- Simics documentation is browser-based and can be generated using the documentation command in the workspace director. On older Simics versions, e.g., 4 and 5, documentation is in PDF in the doc directory relative to workspace directories.
- See the *ethernet...* manual for connecting real networks.
- Use `wireshark <switch>` to see traffic going through one of the switches. Note that wireshark in the Simics environment is sensitive to changes made to the simulated system. For example, if Wireshark is running then writing to memory will cause Wireshark to not report traffic. If you want to watch traffic with Wireshark when using the `injectIO` command, use the `stay=True` option, then start Wireshark and then continue the simulation.
- The `enable-realtime-mode` prevents the simulation from running faster than real time. Useful when trying to login to a virtual console, or avoid network timeouts. Also useful when working remotely and you cannot get a `ctl C` in edgewise.
- The `"x"` command at the simics command line displays memory values. Use `"help"` and `"apropos"` to find other commands of interest.
- Use `pselect` to control which cpu you are viewing.
- The `pregs` command displays registers.
- To save changes made to an OS disk image, use `save-persistent-state`; and then exit simics and use the `bin/checkpoint.merge` command to create a new craf file (found in the checkpoint directory).
- To trace all instructions and memory access use:

```
load-module trace
new-tracer
trace0.start
```

Simics 7 changed the tracer:

```
new-tracer-tool name=mydog file=/tmp/goodinstruct.trace -connect-all
```

- The Simics command line uses standard bash history. The up arrow can save much time and syntax problems.
- Capture command line output (redirect a copy) to a file using the `output-file-start / output-file-stop` commands (`start-command-line-capture` replaced that).
- `list-port-forwarding-setup` to view current port forwarding

Appendix O Injecting to kernel buffers

TBD integrate all injection sections, and clarify different approaches: ioctl; select / read; x86 buffers

The injectIO function is hamstrung if the application makes many single-byte read calls, because writing data into the simulation forces a reset of the origin used for reverse execution. This may also arise when protocols are implemented with buffered reads, e.g., 80 bytes at a time.

One solution to this is to inject the data into the kernel buffer instead of into the application buffer. We can then use trackIO and not have to write additional data into memory. This strategy has several limitations, but is also suitable for some environments. In general, the goal is to provide the application with a realistic view of input data in the course of its processing. There are two potential strategies for injecting directly into kernel buffers. The first is to intercept and patch return values from calls such as ioctl and select, and just allow the kernel to return previously buffered data for each read/recv call. The second approach is to modify the kernel's data structure used to track data in the input buffer. Neither approach attempts to fully synthesize the kernel input processing, and thus running forward past the application processing is likely to reflect divergence or kernel errors.

Intercepting kernel calls such as select can get complicated, particularly if the application is using timeouts and relies on the intervening blocking. Altering kernel buffer pointers avoids some of those complications. The snapshot created for use by AFL or data tracking must then include the address of the kernel buffer, and the addresses of the two pointer values that the kernel references when responding to read calls. Each are found by using reverse data tracking within the prepInjectWatch command, which is intended to be used after the analyst does a trackIO and observes the system calls.

Appendix P Troubleshooting

P.0.1 Missing syscalls in logs

Syscall haps for the default context and the RESim context can co-exist. The presence of syscall log entries does not mean that all the syscalls you intend to catch are being caught.

P.1 Real networks: WARNING

Simics supports traffic between the simulation and a real network using **connect-real-network** commands and related commands using a *service node*. This can be useful to quickly generate new packets and send them to the target, but without needing to interact with a driver component. Note however that Simics has limitations on the use of real networks when reverse execution is enabled, or memory snapshots (**restore-snapshot** are used. Some of these limitations can lead to silent corruption of the simulation. Others to crashes. An obvious limitation is that you cannot replay periods in which real network traffic was received (though Simics supports a separate IO replay feature).

Most problems related to real network can be avoided by connecting and completing use of the real network prior to enabling reverse execution, e.g., via a debugProc command. Or using **resetOrigin** after all real-network IO is finished. See the **reset** option to the trackIO function. It should be used when real-networks interact with drivers, e.g., an ssh script to cause data to be sent to the target.

Automation, such as the **prepInject** function can make use of Simics magic instructions to allow a driver computer to cause a new origin to be established. See the **simics/bin/driver-driver** program as an example.

Use the **INIT_SCRIPT ENV** directive in the init file to specify a simics script to load at the start of each session.

Real networks should be avoided when debugging, fuzzing or replaying TCP services since they can lead to undefined behavior.

P.2 Failed cat to /dev/udp/localhost

The name **localhost** fails on the blade servers (old Linux?). For example:

```
cat fu.io > /dev/udp/localhost/6060
```

should be, instead:

```
cat fu.io > /dev/udp/127.0.0.1/6060
```

P.3 Real networks and UDP

Using **cat foo > /dev/udp/localhost/60060** is fine for a single packet. However multiple packets seem to get lost unless brief sleeps are added between the cat commands.

P.4 Backtracing malloc'd addresses

You have the address of a buffer you think was malloc'd; you back trace, which stops in malloc. You think you found it, look at the call stack and declare success. You may be wrong. You may be looking at the malloc that happened prior to the malloc of interest. For example, if you are looking for the source of a memory location whose value is 0xf4938, RESim may find that in malloc, and then happily continue to back trace until it finds the creation of value 0xf4930 – just in increment, right? So, look at your cycles and notice large gaps. Add a "value" field to the bookmark printout to make it more obvious when a permutation on the desired value is being reported.

P.5 Fork exit

A fork followed by an exit may result in the exit occurring before the child is ever scheduled.

P.6 Fuzzing

If the number of paths seems stuck low, look at the cycle backstops. If the application will loop looking for more data, that will lead to a perceived hang. Use the `STOP_ON_READ=True` to cause the session to stop when the application cycles back to the read. This has obvious limitations.

P.7 Fuzzing TCP

Programs may be sensitive to packetizing that occurs under TCP. All testing and operations may have occurred with complete data transfers in which each read call returns all data needed for a transaction. If the fuzzing session were to force the client to perform multiple reads, untested code might be exercised.

P.8 Multipacket workflows

Often, when working with multiple packets, you need to avoid use of real networks and data injection to avoid potential corruption of the simulation. This is generally true when reverse execution is required for the analysis. This is not the case for code coverage operations such as AFL sessions that have no requirement for reverse execution. Analysis over the full session of a multipacket simulation requires a driver computer that sends the desired packets. Any checkpoint used for such a session must reflect time prior to the return from the read/recv call. Sessions used for fuzzing require a snapshot that reflect the precise return from the first read/recv call.

We therefore expect many workflows to use two different snapshots, one for data injection and one for analysis over the full data period, e.g., via trackIO. The latter requires at least two computers in the simulation.

When TCP data is injected into kernel buffers, the state of the kernel becomes corrupted, thus preventing running forward, e.g., to a new checkpoint. In order to change state of the system, revert to a running snapshot and send the crafted data via a driver. Again, do not try to move a `prepInjectWatch` checkpoint forward.

P.9 Your target process is not scheduled in the snapshot

Consider debugging a service that loads before you want to create a snapshot, e.g., you wish to wait for other processes to start. You wish to debug the target's data consumption and you know it will hang on a select or a read. So after `debugProc`, you run a while without giving it data and then create a snapshot. When the snapshot is restored, your target process is not scheduled, and thus the `debugSnap` function will appear to hang until you provide data that causes your target to be scheduled. This is expected.

P.10 Backstops

If you see fewer basic block hits or fewer watch marks than expected, try increasing the `AFL_BACK_STOP_CYCLES` or `BACK_STOP_CYCLES` values and possibly the `HANG_CYCLES`. Some applications have delays while they go off and do something other than processing your input data. If you find that you need a very large backstop value that affects performance (e.g., causes AFL to be very slow), try using a `BACK_STOP_DELAY` value. To determine if this may help, use `playAFL` to generate a hits file and then use the `dumpCoverage.py` utility to observe the cycles of the hits. Look for a small number of hits, then a big delay, and then a lot of hits without large cycle gaps. If you see that, then set the delay value so that backstops and hang detection does not commence until some number of hits have occurred.

Appendix Q Windows

A preliminary implementation of Windows application tracking is under development in the win7 branch of RESim. These preliminary functions support analysis of 64-bit Windows 7. The build is 6.1 Support functions include:

- Tracing syscalls with `traceAll`. There is also a `traceWindows` command that traces a subset of windows system calls. This can be useful when performance is impacted by the raft of syscalls some applications make. Also consider making use of the `ONLY_PROGS` and `SKIP_PROGS` directives.
- Using `OpenFile/CreateFile` and `MapViewOfFileSection` to track DLLs similar to how we use `open/mmap` to map so files. This seems to be slow. Revisit for performance. For now, use `stopThreadTrack` after you think DLLs have been loaded.
- `trackIO` mostly works, with initial processing of libc type data move functions. Also try `trackRecv` to focus on windows `recv` calls for performance.
- `prepInjectWatch` will create a snapshot and record windows kernel buffers. The snapshot will be just before the kernel starts to copy data to user space.
- Ability to detect SEGV, including catching return to user space without mapping faulted addresses (excepting stack growth).
- Fuzzing windows applications is working, at least preliminarily.
- `getWin7CallParams` with `track_params=True` will generate a report on kernel references to system call parameters. This is useful when creating code to parse out the system call parameters.
- `runTo32` Intended to run a 32-bit application forward from some 64bit syscall library until the processor is restored to 32-bit mode.

Windows applications can be very system call intensive. Use of crypto DLLs seems to generate many system calls. A new function was introduced to suspend system call tracing while executing from selected DLLs. A file named by the `DLL_SKIP` environment variable in the ini file contains, as the first non-comment line, the name of the DLL whose system calls are to be skipped. System call tracing will stop when that DLL is entered. That name is then followed by a list of other DLL names, one per line. System call tracing will recommence when any DLL *other* than those in the list are entered.

Windows page table management differs from Linux in that directory table entries are updated individually? With linux, the coverage module can set a break on the start of a table directory. When that break it hit, we can then wait until a return to user mode at which time the table will be populated. Windows seems to update table entries individually. This requires divergence in the coverage module. Or maybe this is just a 32/64 bit thing.

Q.1 Windows program names

Windows case-insensitive file naming can lead to extremely slow responses to `debugProc`, which must find the program under the root prefix. Use the `buildExecDict.py` utility to optimize these searches.

Q.2 Windows Kernel Parameters

The `getKernelParams.py` program creates `param` files containing Windows kernel parameters required by RESim. These include the address hit by the `sysenter` instruction, the instruction from which the kernel returns to user space, and values needed to reflect the jump table used by the kernel to vector different system calls to their respective functions. Other parameters are hard-coded within the `w7Params.py` file, which is called by `getKernelParams.py`. These reflect values found through experimentation with different versions of the `w7Params.py` program, and through use of the Windows kernel debugger. These hard-coded values include linked list pointers for process records managed by the kernel, and the unique PID and program strings, (similar to Linux `comm` values. Future work includes extending this program to automatically gather these values, and do so for different kernel versions.

Many of the kernel parameters are derived from information published here: <https://www.geoffchappell.com/index.htm>. The parameters needed by RESim are as follows (using RESim variable names).

These offsets are currently hard-coded.

- `ts_next` Offset of the next record pointer in the head in `EPROCESS`.

- `ts_prev` Offset of the previous record pointer in the head in `EPROCESS`.
- `ts_comm` Offset of the program name (equivalent to `comm` in Linux records) in `EPROCESS`.
- `ts_pid` Offset of the process ID in `EPROCESS`
- `proc_ptr` Offset within `ETHREAD` to the pointer to the thread's `EPROCESS` record.
- `THREAD_ID_OFFSET` within `ETHREAD` to thread identifier.
- `THREAD_HEAD` within `EPROCESS` to thread head within `ETHREAD`.
- `THREAD_NEXT` within `ETHREAD` to pointer to next record.
- `ACTIVE_THREADS` within `EPROCESS` of count of active threads.
- `PEB_ADDR` Offset within `EPROCESS` of the address of the process environment block (PEB).
- `PROC_CREATE_OFFSET` Offset within the stack of the address of the process creation block for the `CreateUserProcess` call.
- `PROG_NAME_OFFSET` Offset within the process creation block of the path to the executable program.

These values are derived by `getKernelParameters.py`

- `gs_base` Base address of GS segment when the parameters were generated.
- `current_task` Offset within GS segment of pointer to the current thread.
- `sysret64` Kernel address from which the return is made to user space.
- `sysenter` Kernel address of syscall entry point.
- `win7_saved_cr3_phys` physical address of where the kernel saves its `cr3` value.
- `saved_cr3` Offset within GS segment where `cr3` is saved.
- `ptr2stack` Offset within GS segment where pointer to the user stack is saved.
- `syscall_compute` Kernel address at which the syscall jump table value is computed.
- `syscall_jump` Jump table value that is multiplied by the syscall number to compute the jump value.
- `page_fault` Kernel address of entry when page fault is encountered.

Q.3 PIDs and Threads

Windows threads share a common pid. Linux threads have distinct PIDs. Most instances of the variable "pid" within `RESim` actually (should) refer to a thread. Currently they are integers. These variables will be replaced by "tid" string variable. In Windows, these will be `pid-threadId`. For Linux, simply a string of the pid.

SO maps are per process and the Windows instance will continue to use pids, but its interfaces will expect tids.

All module interfaces will expect tids, and will sort them out from there if needed.

The Context Manager must be able to distinguish between threads of a process, e.g., for trace maze management.

If `traceAll` while debugging, the all threads of the process must be traced.

Q.4 CR3 and Windows

Windows saves the `CR3` value used by the kernel in a location relative to GS. The `CR3` used by the kernel stays constant until it doesn't. The `memUtils v2p` changes the saved value when it is found to have changed.

Q.5 Windows syscall parameters

Parameters to Windows system calls are determined by a combination of online resources, experimentation and use of the `getWin7CallParams` function. The latter is a RESim function that observes the kernel reading and writing user space, and records those observations within the log file. An example from a `DeviceIoControlFile` system call with a command operation of `SUPER_CONNECT` is below.

```
2024-12-11 16:20:25,910 - DEBUG - call: SUPER_CONNECT rcx: 0x3e4 rdx: 0x0 r8: 0x0 r9: 0x7c736c \
    sp: 0xd2ebc8 cycles: 0x4b1652ada9
1 addr: 0xd2ec18 read: b'0000000000000000' size: 8 best_base: rsp4 best_base_delta: 0x10
2 addr: 0xd2ec10 read: b'0000000000000000' size: 8 best_base: rsp4 best_base_delta: 0x8
3 addr: 0xd2ec08 read: b'0000000000000001a' size: 8 best_base: rsp4 best_base_delta: 0x0
4 addr: 0xd2ec00 read: b'000000000466ead4' size: 8 best_base: rsp3 best_base_delta: 0x0
5 addr: 0xd2ebf8 read: b'00000000000120c7' size: 8 best_base: rsp2 best_base_delta: 0x0
6 addr: 0xd2ebf0 read: b'000000000d2ec20' size: 8 best_base: rsp1 best_base_delta: 0x0
7 addr: 0xd2ec20 read: b'007c736c' size: 4 best_base: 0xd2ec20 best_base_delta: 0x0 ptr 0xd2ec20 \
    base rsp1 offset 0x0
8 addr: 0xd2ec28 read: b'0000000000000024' size: 8 best_base: 0xd2ec20 \
    best_base_delta: 0x8 ptr 0xd2ec20 base rsp1 offset 0x0
9 addr: 0xd2ec20 read: b'007c736c' size: 4 best_base: 0xd2ec20 \
    best_base_delta: 0x0 ptr 0xd2ec20 base rsp1 offset 0x0
10 addr: 0x7c736c read: b'00000103' size: 4 best_base: r9 best_base_delta: 0x0
11 addr: 0x7c7370 read: b'00000000' size: 4 best_base: r9 best_base_delta: 0x4
12 addr: 0x466eae6 read: b'0000000000000000' size: 8 best_base: 0x466ead4 \
    best_base_delta: 0x12 ptr 0x466ead4 base rsp3 offset 0x0
13 addr: 0x466eade read: b'f69b503450000002' size: 8 best_base: 0x466ead4 \
    best_base_delta: 0xa ptr 0x466ead4 base rsp3 offset 0x0
```

Wrote:

```
14 addr: 0xd2ec20 wrote: 0x7c736c size: 4 best_base: 0xd2ec20 best_base_delta: 0x0 ptr 0xd2ec20 \
    base rsp1 offset 0x0
15 addr: 0xd2ec28 wrote: 0x24 size: 8 best_base: 0xd2ec20 best_base_delta: 0x8 ptr 0xd2ec20 \
    base rsp1 offset 0x0
16 addr: 0x7c736c wrote: 0x103 size: 4 best_base: r9 best_base_delta: 0x0
17 addr: 0x7c7370 wrote: 0x0 size: 4 best_base: r9 best_base_delta: 0x4
```

The RESim `winSyscall` module tracks system calls by looking at numbered parameter frames. The first four parameters are the registers `rcx`; `rdx`; `r8`; and `r9`. The remaining four are pointers relative to the stack, named `rsp1`, `rsp2`, etc. In the above example, item 13 reflects the kernel reading the socket information, including the IP address and port. The record tells us the address of that sock struct is 10 bytes relative to the address found in the third stack parameter, and is thus `frame['param7']` in the RESim frames within `winSyscall.py`.

Another example (updated record format) from a `SEND`:

```
20 addr: 0x2ef6f8 read: b'000000ce' size: 4 best_base: 0x2ef6f0 best_base_delta: 0x8 \
    ptr 0x2ef6f0 base 0x2ef530 offset 0x0 ptr 0x2ef530 base rsp3 offset 0x0
```

Says the size was read from `0x2ef6f8`. And that address is found by starting with the address found in `rsp3` (`0x2ef530`). That address contains `0x2ef6f0`.

The `paramOffPtr` names parameters (starting at 1), so `rsp3` would be 7.

```
count_value = self.paramOffPtr(7, [0, 0], frame, word_size) + 8
```

Appendix R CADET01 networking

The `cadet01` example can serve as a rapid prototyping testbed to explore Simics networking and RESim functions. The networking used in the example is described below.

- The driver has address 10.20.200.22 on `switch0` via `ens11`.
- The `mapdriver.simics` script forwards port 4022 to port 22 on that interface.
- The driver also has 10.0.0.140 via `ens25`, which is on the subnet containing the "cadet" server, which has IP of 10.0.0.22 on `ens11`.
- Everything goes through `switch0`.

Appendix S More code coverage

The following described functions that have not been recently tested. Features to reflect code coverage should support human analysis as well as automated analysis, e.g., fuzzing. To support the human, coverage is highlighted within IDA using color-coded basic blocks.

Coverage tracking commences with use of the `mapCoverage` command and is intended to aid understanding of program response to inputs tracked via the `trackIO` or `injectIO` commands. Those basic blocks which are only hit between coverage commencement and the first input event are separately colored so as to not be confused with blocks hit subsequent to receipt of input. This distinction is intended to facilitate comparisons between *data sessions* defined as periods between receipt of input (or IO injection) and quiescence with respect to the backstop logic. Within the IDA display, coverage properties of basic blocks will be coded into five colors:

- Never executed.
- Never executed during a data session
- Executed during a data session, but not during this data session
- Executed during this session and some previous data session.
- Executed only during this data session.

The `coverage` module maintains three data files for each coverage unit: a running total set of hits from previous data sessions; the hits for the most recent data session; and hits which occur prior to the first input data reference (e.g., reads, selects, etc). The running total is only updated immediately prior to the start of a new data session. At the start of each data session, the coverage module will restore any breakpoints found in the recent session data file and then merge it into the running total. The coverage module will then clear its internal hits data. When the IDA client is started with the `color` argument, it will first reset all basic block colors, and will then read the hits files and color blocks accordingly. The running hits file is intended to persist across RESim sessions.

S.1 Branches not taken

The IDA client generates a list of basic block branches that were not taken, i.e., unused exits from hit basic blocks. This list is presented in the BNT window of the IDA client and is intended to help the analyst see places where changes in input data may have resulted in added coverage. When used following a RESim `mapCoverage` command, double-clicking on an entry will take the simulation to the corresponding cycle. Note this is only available for the very first coverage hit of each basic block, and of course only within a session that hit the from block (see the `bbAFL` function). This window is most useful when IDA is started with the `color` option.

S.2 Ad-hoc code coverage

The `color` option and BNT window can also be used outside of the `mapCoverage` context, e.g., to analyze branches not taken during AFL sessions. Double-clicking an entry in the BNT window will go to the source of the branch not taken (note the simulation state is not altered.) The following is a notional code coverage workflow.

- Use AFL to fuzz a target.
- Use the `playAFL` command to replay all of the sessions, generating a new hits file.
- Copy that hits file over the target hits file.
- Start a RESim session from the checkpoint used during AFL.
- Start IDA and use the `resetBlocks` script to clear block coloring; then use `Color blocks` to color all blocks hit during the AFL session.
- Find an interesting BNT in the BNT window.
- Use the `findBB.py` utility to search for all AFL data files that hit the source block.
- Restart RESim and use the `injectIO`, `aflInject`, `aflTrack` or `aflReplay` functions to play the data file identified above.
- Run to the source block of interest and investigate whether an altered input could result in a branch to the BNT.

Note in the above workflow, you may wish to use 2 different checkpoints. The `playAFL` function must run from the checkpoint used with AFL. However the other functions that use trackIO could run from checkpoints that include a driver component. And that checkpoint might be from early on in the process's life, e.g., to identify the sources of state values that do not seem to result from the input data.

Initially, no database of hits will be maintained tying data files to hit sets. A downside is that if a data file is consumed twice, the analyst no longer sees which basic block hits are unique to that data file. Each subsequent data session artifacts are additive, and the effects of any previous runs cannot independently viewed.

S.3 Data injection and system calls

The `writeData` module manages injection of data into simulated memory for both user space buffers and kernel buffers. When the application makes subsequent system calls, e.g., to read more data, the module must sometimes intercede and provide altered return data. For example, when injected kernel buffer data is consumed, the next `recv` should return zero bytes. Calls to `select` and `ioctl` also may require altered return values. The `prepInject` modules currently maintain "call ip" values of the user space address just prior to entering the kernel. While these addresses may vary by syscall, often they do not, i.e., all syscalls come from the same libc or related library. The current `prepInject` design presupposes different calling addresses for things like `select` and `recv`. That seems to be a poor assumption.

Appendix T VxWorks DKM

Some functions have been added to support analysis of applications written for VxWorks DKM environments. Target applications are hosted on the Simics zynqmp RPU processor. The VxWorks DKM kernel must be rebuilt per the Simics and VxWorks instructions, and the resulting elf image is then named in the `RESim` ini file in order to extract symbol addresses of kernel function. `RESim` uses these addresses to catch "kernel calls", which cover the gambit of common library functions of CLIB and POSIX IPC functions as well as VxWorks-specific functions. `RESim` also uses the address of the `taskIdCurrent` symbol for use in tracking which task is running. The ini file also includes parameters for the load address and size of the application, which must be manually entered based on results of the VxWorks command line "ld" operation.

T.1 File system

The Simics zynqmp platform VxWorks implementation maps the Simics workspace directory to the target file system such that the VxWorks command line and applications can name files relative to that workspace directory. Absolute paths fail, so the call parser is implemented to increment the path address by one when there is a leading slash. This should be extended to record the location of the pathname, e.g., in read-only memory so that the memory values can be adjusted when the program is subsequently loaded. The goal is to avoid dynamic modifications.

NOTE: The file system is in the real world, and thus is a potential source of real-world bleed-in that can silently corrupt the simulation micro-checkpoints. See section [P.1](#).

T.2 Legacy binaries

Initial development was performed with VxWorks version 24.03. The resulting DKM api interfaces may not be fully compatible with older applications. For example, the `ioctl` command to set a network device to blocking or non-blocking (FIONBIO) may use a command value of 0x10 in older software. Current VxWorks expects this value to be 0x90040010. The call parser automatically converts these values. Again, it would be preferable to identify those locations and patch the code on load to avoid dynamic modifications.

T.3 Execution context

VxWorks DKM applications are dynamically linked to the kernel functions and all the code executes in supervisor mode. As a result, the standard `RESim` approach for tracing system calls does not work. Instead, we set breakpoints on each desired kernel function, (or all of them if `traceAll` is used). Since these function addresses are often invoked by other kernel functions, we must disable all function breakpoints upon entry into the function called by the application. At that point, we set a breakpoint range of the entire set of module addresses, and use that to catch returns from the kernel function. We use the task ID to differentiate returns that occur from other tasks. This large quantity of kernel entry points also complicates reversing. For systems such as Linux, when a process is being debugged, `RESim` records every system call entry within the `ReverseToCall` module for purposes of skipping backwards out of the kernel. That does not work as simply with VxWorks DKM because there are many kernel entry points, and functions call other functions that are also entry points. If all such

functions have clean BL calls, then we could filter out function calls made from within the kernel. A downside is the possibility that a large numbers of breakpoints being hit will affect performance – but that is unknown.

T.4 VxWorks Performance

Starting the zynqmp platform for VxWorks can be extremely slow if any other components, e.g., a driver, are defined. Altering time quantum and the frequency of the driver cpu does not seem to significantly speed this up. As a work-around, the `ADD_FROM_SNAPSHOT` will load a snapshot and then add other components to it. First start only the zynqmp platform, without starting any application on it. Create a snapshot and then run a ini file that includes a driver and names the snapshot as the `ADD_FROM_SNAPSHOT`. The driver loads slowly, but will finish. Once both are loaded, performance improves considerably.

Appendix U Simics 7

Simics 7 deprecated reverse execution. RESim replaces much of the lost reversing functions with its "reverseMgr" module. See https://github.com/mfthomps/Simics7_Reversing Simics 7 will not run on pre-Haswell CPUs, e.g., Dell R620. However it should run on more recent servers. There seem to be issues with the i82543gc, leading to kernel errors requiring the board to be "reseated". Use a different nic.

Appendix V Progressive fuzzing

This section describes scripts and procedures to fuzz a target through a progressive set of states. As an example, consider a UDP service whose processing of data within the UDP depends the content of previously received UDP packets.

V.1 Overview

Broadly the approach is to:

1. Fuzz from the starting state
2. For each queue file found to be unique by the `dedupCoverage.py` utility:
 - Drive the service to ingest the queue file and save the resulting state
 - Create a `prepInject` snapshot from this new state.
 - Fuzz from the new state, using original unique queue files as seeds
3. Identify the queue files that resulted in newly hit blocks in the new fuzz sessions.
4. For each such queue file, starting from the state of its corresponding fuzz session:
 - Drive the service to ingest the queue file and save the resulting state
 - Create a `prepInject` snapshot from this new state.
 - Fuzz from the new state, using original unique queue files as seeds
5. Repeat from step 3 for each subsequent state

V.2 Limitations

No claim is made regarding the completeness of this approach. Its shortcomings include:

- Hitting new blocks does not necessarily mean the state machine has progressed.
- State variables can be set without hitting new blocks.
- New blocks may be hit simply because time is progressing and threads may be scheduled independent of fuzzed input.
- This form of progressive fuzzing does not work with TCP conversations.
- The number of states can explode if there are large numbers of unique coverage files.

Alternatives to progressive fuzzing include:

- Include multiple UDP packets in fuzzed data and use the `UDP_HEADER` control to inject each in turn. This becomes challenging if the input buffer addresses change, or if `select/ioctl`-type of operations precede the receive.
- Inject multiple UDP packets (or portions of TCP conversations) into the driver using JSON files, derived from fuzzed data.

V.3 Workflow and artifacts

This description assumes a UDP service. Start with a fuzzing session and run `runPlay` and `dedupCoverage.py` when it no longer finds new paths. Then create a bash script that defines the parameters described in `simics/fuzz_bin/progressive_fuzz.sh` prior to invoking that script, use `simics/fuzz_bin/auto_fuzz1.template` as a template. Also create `next_state.directive` and `next_prep.directive` files from the corresponding templates.

When the `auto_fuzz1.sh` script finishes, your workspace will have snapshots for each unique queue file, one named `next_state_N` and another named `next_prep_N` where N is the index into the unique queue list. There will also be a `auto_fuzz` directory containing workspaces for each unique index. And AFL artifacts for each of those workspaces will exist under the AFL output and seeds directories.

The `find_next_state.py` script can be run to see which fuzzing sessions resulting in new states.

The N suffix is referred to as the *state ID*. The ID will expand as states progress, e.g., it may become something like `4_3_9`, reflecting a state that was reached by ingesting the 5th unique index (they are zero relative), followed by the forth queue file found to have hit new blocks... After running `progressive_fuzz.sh` script finishes, any new states IDs will be integers. Copy your bash script and change it to invoke `progressive_fuzz_next.py`, and add the `state_id` of the state you wish to proceed from.

Use `merge_progressive_hits.py` to merge all the hits from progressive fuzzing into one hits file.

Use `auto_track.sh`, to generate trackio artifacts for each new coverage file.

Appendix W ROP gadgets

The following utilities can be used to find ROP gadgets. They are in various stages of usefulness and architecture completeness.

- `bytesInBin.py` – find a byte sequence in a binary file. Limited support for wildcards.
- `bytesInStatic.py` – Uses `bytesInBin.py` to search binaries listed in a `static.list`
- `findBinGadget.py` – Uses `bytesInStatic.py` to search for byte sequences created by assembling an input sequence of instructions.
- `asm.sh` – assemble x86
- `idaStaticGadgets.py` –

Appendix X PowerPC

Simics 6.0.146 includes a powerpc quick start platform with a 32-bit processor. PPC is Big Endian. Simics PPC QSP linux is 3.4 The `ppc sc` instruction leads to an interrupt handler in low memory that eventually uses `rfi` (return from interrupt) to enter the kernel. The kernel returns status in the condition register, whose value must be `0x1000`. If not, the value returned in `r3` is the kernel error. The interrupt handler runs in supervisor mode, complicating kernel mode checks.

```
[board.cpu[0]] v:0x0ff26f4c p:0x000cc7f4c sc
[board.cpu[0]] Exception: System_call
taskModeChanged32 new 1
mode changed
[board.cpu[0]] v:0x00000c00 p:0x000000c00 mtsprg0 r10
[board.cpu[0]] v:0x00000c04 p:0x000000c04 mtsprg1 r11
[board.cpu[0]] v:0x00000c08 p:0x000000c08 mfcrr r10
ooo
[board.cpu[0]] v:0x0000f380 p:0x00000f380 mtlr r9
[board.cpu[0]] v:0x0000f384 p:0x00000f384 rfi
[board.cpu[0]] v:0xc000f430 p:0x00000f430 stw r3,152(r1)
```

The kernel uses an `rfi` to return to the caller


```
[board.cpu[0]] v:0xc000f4e4 p:0x00000f4e4 rfi
[board.cpu[0]] v:0x0ff26f50 p:0x000cc7f50 bnsldr+
```

Simics mode hap reports the supervisor mode as 1, though it is zero in the msr PR bit.

The kernel is entered at 0xc000f430 above (and also from a different syscall). That address is hit many times.

On return to user space, r7 has same value as return PC; as does the ssr0 register

On page fault, ssr0 has the faulting address (at least for code?) page fault vectors to pc 0x400 return from page fault from 0xc000fbb8
sc vectors to 0xc00

X.1 current task pointer

While in the kernel, r2 points to the current task struct. So where does that come from? The r2 value is set in the low memory interrupt handler. But the page table is not right until the rfi into the kernel.

Using brute force search, current_task seems to be 0xc07fe39c determined from snapshot created without RESim. When booting with RESim, the offset of the record changes. But is consistent. 2nd try 0xc07c239c 3rd try 0xc07c239c Does not seem random. Because none of the other kernel addresses change.

Kernel does not use ASLR (for current task anyway)

X.2 syscall jump table

Syscall computation:

```
[board.cpu[0]] v:0xc000f458 p:0x00000f458 cmplwi r0,353
[board.cpu[0]] v:0xc000f45c p:0x00000f45c lis r10,-16384
[board.cpu[0]] v:0xc000f460 p:0x00000f460 ori r10,r10,0x84cc
[board.cpu[0]] v:0xc000f464 p:0x00000f464 slwi r0,r0,2
[board.cpu[0]] v:0xc000f468 p:0x00000f468 bge- 0xc000f4e8
[board.cpu[0]] v:0xc000f46c p:0x00000f46c lwzx r10,r10,r0
[board.cpu[0]] v:0xc000f470 p:0x00000f470 mtlr r10
```

After which r10 has the syscall-specific address. The "lis r10,-16384" loads r10 with 0xc0000000 after the or we get 0xc00084cc So getKernelParams likely should just record the orig value

X.3 Page tables

Grim. Block Address Translation (BAT) is used for kernel addresses. Inverted page tables with Segment/ hashing used for user space. See pageUtilsPPC32.py. Page mapping depends on set of segment registers (SR0, SR1..) and a single SDR that provides an origin.

The RESim pageCallback module is context independent, requiring us to find page table structures for processes other than the currently scheduled one. Thus we must know how to find SR and SDR regs for some other process.

X.4 instructions

A minus after a conditional branch mnemonic is a "hint" that the branch will likely to be taken.

Each of the cr# condition registers consists of four bits, numbered from most significant to least significant

Bit Name Mnemonic

0 Less than lt

1 Greater than gt

2 Equal to eq

3 Summary overflow so

For convenience, the assembler predefines the constants lt, gt, eq, and so to represent their respective

Uses bl to branch-and-load lr as call. Uses bctrl to branch to the count register. How different from a call? r3 is return value register r1 is sp bl sets condition register per returned r3 value.

X.5 Simics breakage

While fuzzing, we sometimes see: [PPC32.cpu[0] error] Failed to map breakpoint logical address to physical. Passing 0 to handler. which is odd because the breakpoint in question is set on a physical address. This is intermittent. Work-around is to delete the breakpoint, otherwise Simics reports too many errors and dies.

Data tracking breaks. It looks like the ppc32 model gets confused on cycles.

Appendix Y Windows XP (32 bit)

Parameters passed to the kernel via user stack. User space sp is in edx on entry.

Kernel addresses from 0x80000000 to light Kernel space frame at 0xffdff040?

```
[board.mb.cpu0.core[0][0]] cs:0x000000008053d540 p:0x00053d540 mov ecx,0x23
[board.mb.cpu0.core[0][0]] cs:0x000000008053d545 p:0x00053d545 push 48
[board.mb.cpu0.core[0][0]] cs:0x000000008053d547 p:0x00053d547 pop fs
[board.mb.cpu0.core[0][0]] cs:0x000000008053d549 p:0x00053d549 mov ds,cx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d54b p:0x00053d54b mov es,cx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d54d p:0x00053d54d mov ecx,dword ptr [0xffdff040]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d553 p:0x00053d553 mov esp,dword ptr [ecx+0x4]
...
Current task?
[board.mb.cpu0.core[0][0]] cs:0x000000008053d573 p:0x00053d573 mov ebx,dword ptr [0xffdff01c]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d579 p:0x00053d579 push 59
[board.mb.cpu0.core[0][0]] cs:0x000000008053d57b p:0x00053d57b mov esi,dword ptr [ebx+0x124]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d581 p:0x00053d581 push dword ptr [ebx]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d583 p:0x00053d583 mov dword ptr [ebx],0xffffffff
[board.mb.cpu0.core[0][0]] cs:0x000000008053d589 p:0x00053d589 mov ebp,dword ptr [esi+0x18]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d58c p:0x00053d58c push 1
[board.mb.cpu0.core[0][0]] cs:0x000000008053d58e p:0x00053d58e sub esp,0x48
[board.mb.cpu0.core[0][0]] cs:0x000000008053d591 p:0x00053d591 sub ebp,0x29c
[board.mb.cpu0.core[0][0]] cs:0x000000008053d597 p:0x00053d597 mov byte ptr [esi+0x140],0x1
[board.mb.cpu0.core[0][0]] cs:0x000000008053d59e p:0x00053d59e cmp ebp,esp
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5a0 p:0x00053d5a0 jne 0x8053d53c
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5a2 p:0x00053d5a2 and dword ptr [ebp+0x2c],0x0
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5a6 p:0x00053d5a6 test byte ptr [esi+0x2c],0xff
...
Computed jump:
esi_value = readWord(ptr_to_current_thread + 0x124)(
esi_adjusted = esi_value + 0xe0
edi_adjust = readWord(edi_adjusted)
edi_start = ((call_num >> 8) & 0x30) + esi_adjust
val_in_edi = readWord(edi_start)
call_to_addr = val_in_edi + 4*call_num
call_to = readWord(call_to_addr)
[board.mb.cpu0.core[0][0]] cs:0x000000008053d61e p:0x00053d61e mov ebx,dword ptr [edi+eax*4]

ESI value based on ebx from hardcode
[board.mb.cpu0.core[0][0]] cs:0x000000008053d57b p:0x00053d57b mov esi,dword ptr [ebx+0x124]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d581 p:0x00053d581 push dword ptr [ebx]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d583 p:0x00053d583 mov dword ptr [ebx],0xffffffff
[board.mb.cpu0.core[0][0]] cs:0x000000008053d589 p:0x00053d589 mov ebp,dword ptr [esi+0x18]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d58c p:0x00053d58c push 1
[board.mb.cpu0.core[0][0]] cs:0x000000008053d58e p:0x00053d58e sub esp,0x48
[board.mb.cpu0.core[0][0]] cs:0x000000008053d591 p:0x00053d591 sub ebp,0x29c
[board.mb.cpu0.core[0][0]] cs:0x000000008053d597 p:0x00053d597 mov byte ptr [esi+0x140],0x1
[board.mb.cpu0.core[0][0]] cs:0x000000008053d59e p:0x00053d59e cmp ebp,esp
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5a0 p:0x00053d5a0 jne 0x8053d53c
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5a2 p:0x00053d5a2 and dword ptr [ebp+0x2c],0x0
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5a6 p:0x00053d5a6 test byte ptr [esi+0x2c],0xff
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5aa p:0x00053d5aa mov dword ptr [esi+0x134],ebp
```

```

[board.mb.cpu0.core[0][0]] cs:0x000000008053d5b0 p:0x00053d5b0 jne 0x8053d400
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5b6 p:0x00053d5b6 mov ebx,dword ptr [ebp+0x60]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5b9 p:0x00053d5b9 mov edi,dword ptr [ebp+0x68]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5bc p:0x00053d5bc mov dword ptr [ebp+0xc],edx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5bf p:0x00053d5bf mov dword ptr [ebp+0x8],0xbadb0d00
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5c6 p:0x00053d5c6 mov dword ptr [ebp],ebx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5c9 p:0x00053d5c9 mov dword ptr [ebp+0x4],edi
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5cc p:0x00053d5cc sti

```

get edi value IS fuction of eax!

```

[board.mb.cpu0.core[0][0]] cs:0x000000008053d5cd p:0x00053d5cd mov edi,eax
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5cf p:0x00053d5cf shr edi,8
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5d2 p:0x00053d5d2 and edi,0x30
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5d5 p:0x00053d5d5 mov ecx,edi

```

Adjust edi based on esi value

```

[board.mb.cpu0.core[0][0]] cs:0x000000008053d5d7 p:0x00053d5d7 add edi,dword ptr [esi+0xe0]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5dd p:0x00053d5dd mov ebx,eax
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5df p:0x00053d5df and eax,0xffff
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5e4 p:0x00053d5e4 cmp eax,dword ptr [edi+0x8]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5e7 p:0x00053d5e7 jae 0x8053d332
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5ed p:0x00053d5ed cmp ecx,0x10
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5f0 p:0x00053d5f0 jne 0x8053d60c
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5f2 p:0x00053d5f2 mov ecx,dword ptr [0xffdff018]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5f8 p:0x00053d5f8 xor ebx,ebx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d5fa p:0x00053d5fa or ebx,dword ptr [ecx+0xf70]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d600 p:0x00053d600 je 0x8053d60c
[board.mb.cpu0.core[0][0]] cs:0x000000008053d60c p:0x00053d60c inc dword ptr [0xffdff638]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d612 p:0x00053d612 mov esi,edx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d614 p:0x00053d614 mov ebx,dword ptr [edi+0xc]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d617 p:0x00053d617 xor ecx,ecx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d619 p:0x00053d619 mov cl,byte ptr [eax+ebx]

[board.mb.cpu0.core[0][0]] cs:0x000000008053d619 p:0x00053d619 mov cl,byte ptr [eax+ebx]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d61c p:0x00053d61c mov edi,dword ptr [edi]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d61e p:0x00053d61e mov ebx,dword ptr [edi+eax*4]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d621 p:0x00053d621 sub esp,ecx
[board.mb.cpu0.core[0][0]] cs:0x000000008053d623 p:0x00053d623 shr ecx,2
[board.mb.cpu0.core[0][0]] cs:0x000000008053d626 p:0x00053d626 mov edi,esp
[board.mb.cpu0.core[0][0]] cs:0x000000008053d628 p:0x00053d628 cmp esi,dword ptr [0x805589d4]
[board.mb.cpu0.core[0][0]] cs:0x000000008053d62e p:0x00053d62e jae 0x8053d7dc
[board.mb.cpu0.core[0][0]] cs:0x000000008053d634 p:0x00053d634 rep movsd
[board.mb.cpu0.core[0][0]] cs:0x000000008053d636 p:0x00053d636 call ebx

```

Windows XP has no drivers for the x86 platfrom motherboard ethernet. It has drivers for the others, but they windows reports error 10, unable to use the device. Trial and error showed that the am79c973 controller is recognized, but it hangs during configuration. The dec21041 appears to work.

Windows XP changes syscall number mapping frequently. The ntdll.dll does the mapping. The RESim/windows/winxpSys reads the ntdll.dll and generates a json map of call numbers to calls.